



图像与视频处理

Image and Video Processing

第十二讲：图像压缩

人工智能与机器人研究所

Institute of Artificial Intelligence and Robotics

兰旭光



目录

- 编码冗余
- 源编码器和解码器
- 霍夫曼编码
- 算术编码
- LZW编码
- 符号编码
- 游程编码
- 无损预测编码
- 块变换编码



JPEG at 0.125 bpp (bits per pixel) 放大

像素深度





JPEG2000 at 0.125 bpp(bits per pixel)

像素深度





感兴趣区域(ROI) (1)

- 以高于图像其他部分的保真度重建图像的特定部分
- 两种类型:
 - 编码期间的ROI: 当编码前已知所需区域的几何形状时, 非常有用。
 - 解码过程中的ROI: 例如在交互式浏览远程数据库中的图像时非常有用。



































我们为什么需要压缩?

■ 数据存储

- $30\text{f/s} \times 720 \times 480\text{pixels/f} \times 3\text{bytes/pixel} = 31\text{MB/s}$
- $2\text{hour} \times 60 \times 60\text{s/hour} \times 31\text{MB/s} = 224\text{GB};$
- 27 DVD-8.5G to store;
- Compressed by 26.3/frame to store in one DVD

■ 1920×1080 , 186MB/s, 1343G-158DVD.

■ 数据传输

- 56kps-12Mbps
- 4534s-20.6s
- $186\text{MB/s} - (12\text{Mbps}) = 124\text{s}$



压缩的基本原理(1)

- 压缩可以节省存储空间和通道带宽
- 压缩技术的应用:
 - 多媒体计算，即在印刷、出版和视频处理中使用计算机
 - 成像传感器和电视的分辨率增强
 - 视频会议、遥感、文件和医疗成像、传真等



压缩的基本原理(2)

■ 两种类型的压缩方案

- **无损**: 解码后的图像在数字上与原始图像相同
 - 适度的压缩比: 最高可达4:1
 - 例子: 霍夫曼, 算术, Ziv-Lempel, JPEG-LS
 - 应用: 存储法律文件、医疗图像等。
- **有损**: 目的是在保持所需的信号质量水平的同时最大限度地提高压缩率
 - 在视觉上不太重要的地区允许有一些退化
 - 高压缩比: 最高可达80:1



压缩的基本原理(3)

- 有损压缩(续)
 - 例子：DPCM，分形，DCT转换，小波
 - 应用：消费者应用，远程浏览，等等。
 - 视觉无损是一种有损压缩的类型



- 让b和b'表示相同信息的两个表示中的比特数，相对数据冗余R为

$$R = 1 - 1/C$$

C是压缩率

$$C = b/b'$$

e.g., 较大的表示的相应相对数据冗余度为0.9, 表明其90%的数据是冗余的



数据冗余度

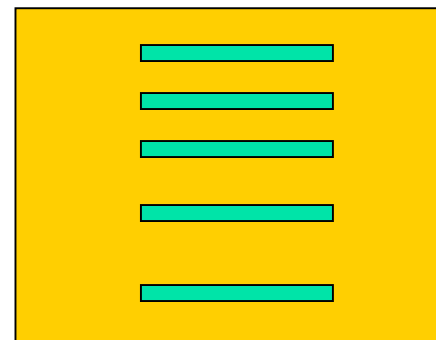
- 图像数据的冗余
 - 像素间的冗余
 - 频谱冗余
 - 心理视觉上的冗余
 - 编码冗余



压缩基本原理(4)

■ 像素间冗余

- 相邻像素的值是高度相关的，也被称为空间冗余
- 帧间冗余



■ 频谱冗余

- 不同颜色平面或光谱带之间的相关性，例如，RGB到YUV 4:1:1

■ 心理视觉上的冗余

- 人类视觉系统（HVS）对所有视觉信息的反应灵敏度不一样



压缩基本原理(5)

- 心理视觉上的冗余(续)
 - 一些视觉信息的相对重要性低于其他
 - 例如, HVS对极高和极低强度的敏感度低于中等强度。这可以在量化中加以利用
 - HVS对边缘或纹理比随机强度变化更敏感
- 编码冗余
 - 基于源数据的熵值
 - 可以通过使用可变长度的代码来加以利用
 - 例如, 哈夫曼编码、算术编码



我们怎样才能实现压缩？

- **编码冗余**

大多数二维强度(intensity)阵列包含的比特数超过了表示强度(intensity)的需要

- **空间和时间上的冗余**

大多数二维强度阵列的像素在空间上是相关的，视频序列在时间上是相关的。

- **不相关的信息**

大多数二维强度阵列包含被人类视觉系统所忽略的信息



冗余的示例



a b c

FIGURE 8.1 Computer generated $256 \times 256 \times 8$ bit images with (a) coding redundancy, (b) spatial redundancy, and (c) irrelevant information. (Each was designed to demonstrate one principal redundancy but may exhibit others as well.)



编码冗余



TABLE 8.1
Example of
variable-length
coding.

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	10000000	8	1	1
$r_{186} = 186$	0.25	11000100	8	000	3
$r_{255} = 255$	0.03	11111111	8	001	3
r_k for $k \neq 87, 128, 186, 255$	0	—	8	—	0

表示每个像素所需的平均比特数是

$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k) = 0.25(2) + 0.47(1) + 0.24(3) + 0.03(3) = 1.81 \text{ bits}$$

$$C = \frac{8}{1.81} \approx 4.42$$

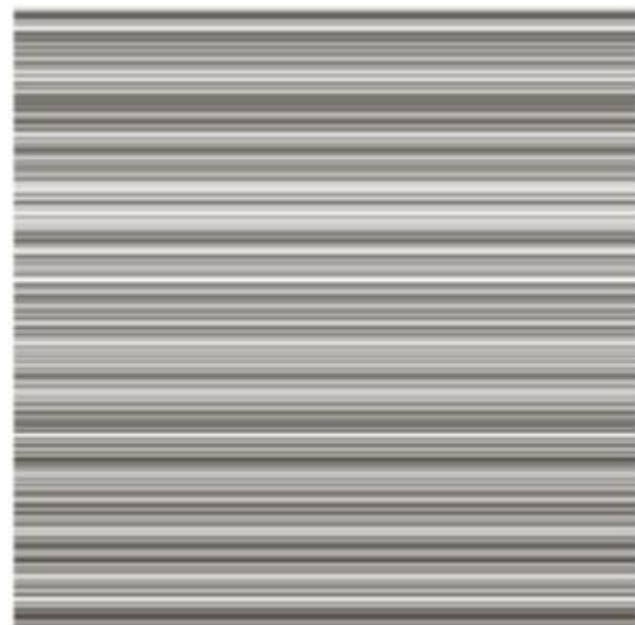
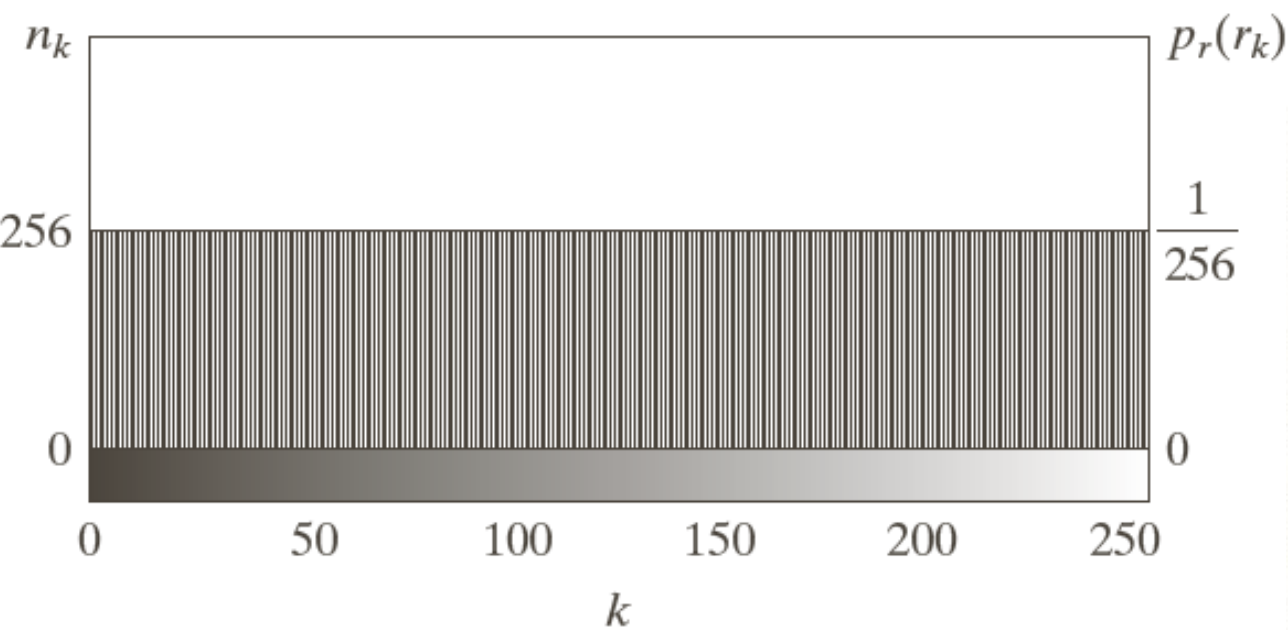
$$R = 1 - 1/4.42 = 0.774$$



空间和时间上的冗余

1. 所有256个强度值的概率相同
2. 每条线上的像素是相同的.
3. 每条线的强度值是随机选择的.

FIGURE 8.2 The intensity histogram of the image in Fig. 8.1(b).





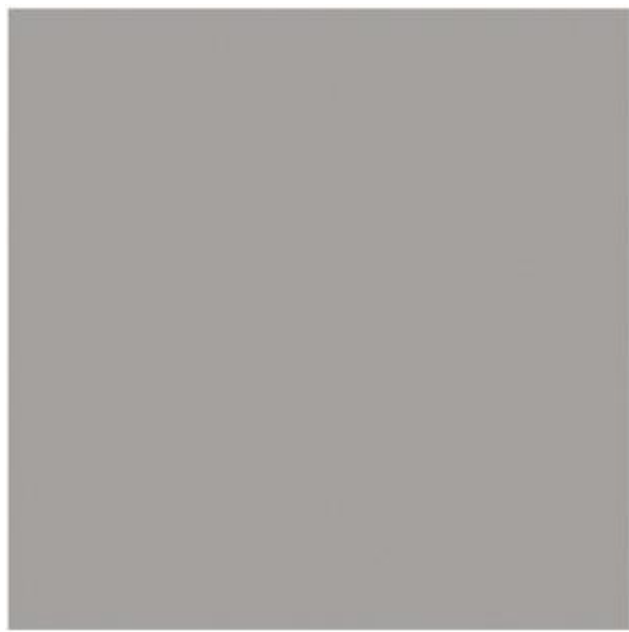
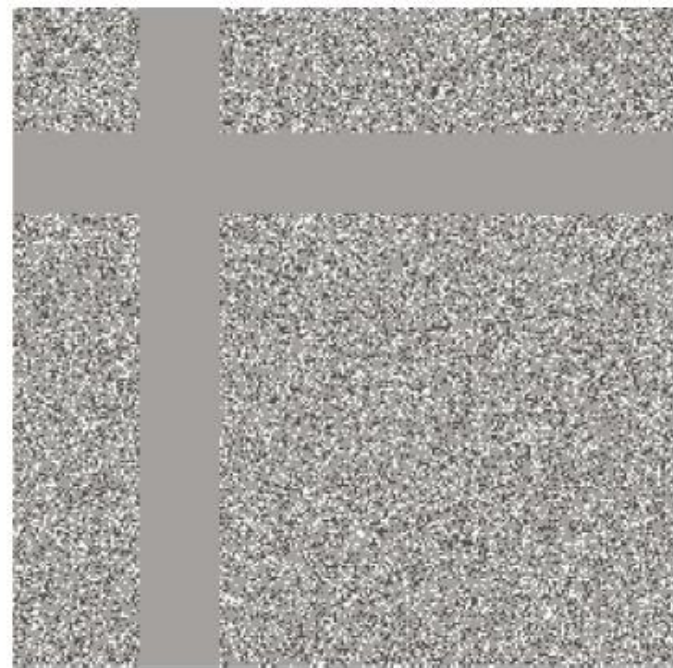
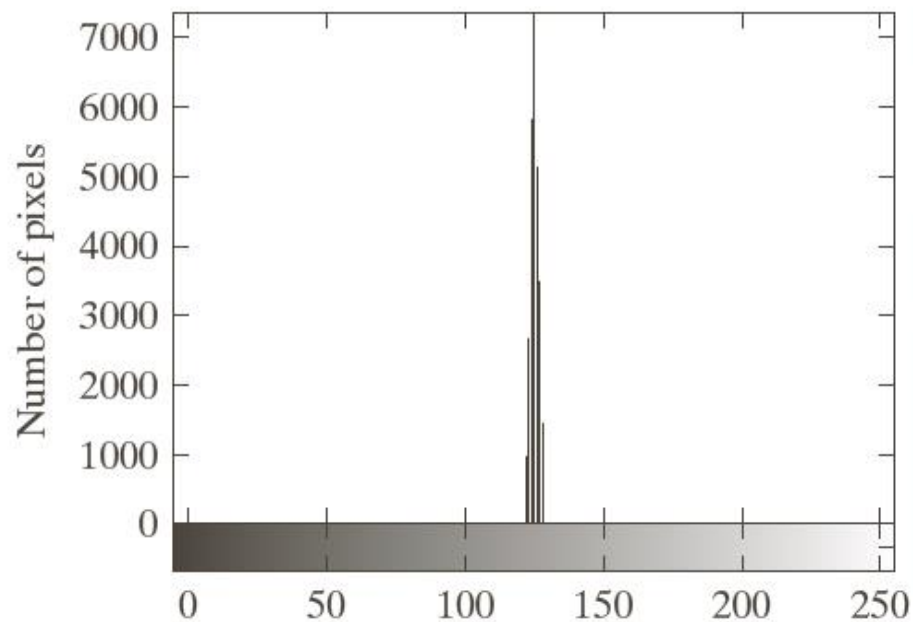
空间和时间上的冗余

1. 所有256个强度值的概率相同
2. 每条线上的像素是相同的.
3. 每条线的强度值是随机选择的.

游程对(*run-length pair*)指定一个新的强度值的开始和具有该强度值的连续像素的数量。在游程表示中，原始表示的每256像素行被一个8位强度值和长度256取代。

压缩率:

$$\frac{256 \times 256 \times 8}{(256 + 256) \times 8} = 128:1$$



a b

FIGURE 8.3

(a) Histogram of the image in Fig. 8.1(c) and (b) a histogram equalized version of the image.

$$\begin{aligned} & 256 \times 256 \times 8 / 8 \\ & = 65536 : 1 \end{aligned}$$



衡量图像信息

一个概率为 $P(E)$ 的随机事件 E 被称为包含:

$$I(E) = \log \frac{1}{P(E)} = -\log P(E)$$

单位信息.



衡量图像信息

给定一个统计上独立的随机事件的来源

Given a source of statistically independent random events from a discrete set of possible events $\{a_1, a_2, \dots, a_J\}$ with associated probabilities $\{P(a_1), P(a_2), \dots, P(a_J)\}$, the average information per source output, called the entropy of the source 每个源输出的平均信息

$$H = -\sum_{j=1}^J P(a_j) \log P(a_j) \quad \text{称为源的熵}$$

a_j is called source symbols. Because they are statistically independent, the source called *zero – memory source*.



If an image is considered to be the output of an imaginary zero-memory "intensity source", we can use the histogram of the observed image to estimate the symbol probabilities of the source. The intensity source's entropy becomes

$$\hat{H} = - \sum_{k=0}^{L-1} p_r(r_k) \log p_r(r_k)$$

$p_r(r_k)$ is the normalized histogram.

如果一幅图像被认为是一个假想的零记忆的强度源的输出，我们可以用观察到的图像的直方图来估计该源的符号概率。强度源的熵为：

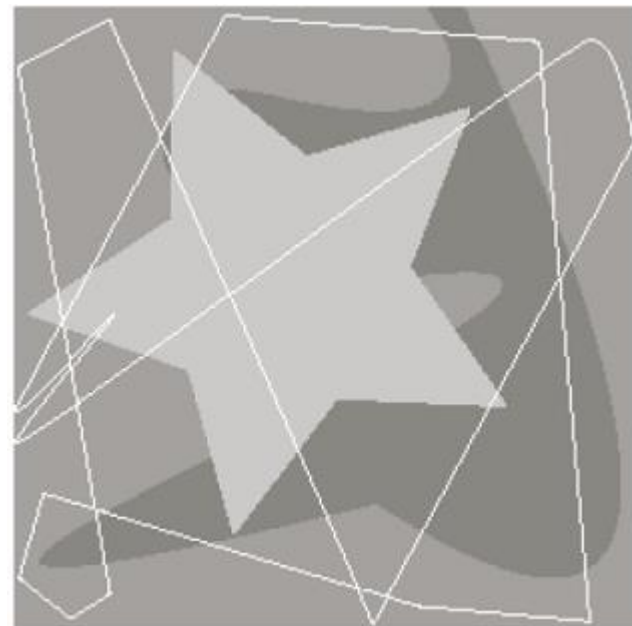
$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k)$$

归一化直方图



衡量图像信息

r_k	$p_r(r_k)$	Code 1	$I_1(r_k)$	Code 2	$I_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	10000000	8	1	1
$r_{186} = 186$	0.25	11000100	8	000	3
$r_{255} = 255$	0.03	11111111	8	001	3
r_k for $k \neq 87, 128, 186, 255$	0	—	8	—	0



For the fig.8.1(a),

$$\hat{H} =$$

$$-[0.25 \log_2 0.25 + 0.47 \log_2 0.47 + 0.25 \log_2 0.25 + 0.03 \log_2 0.03]$$

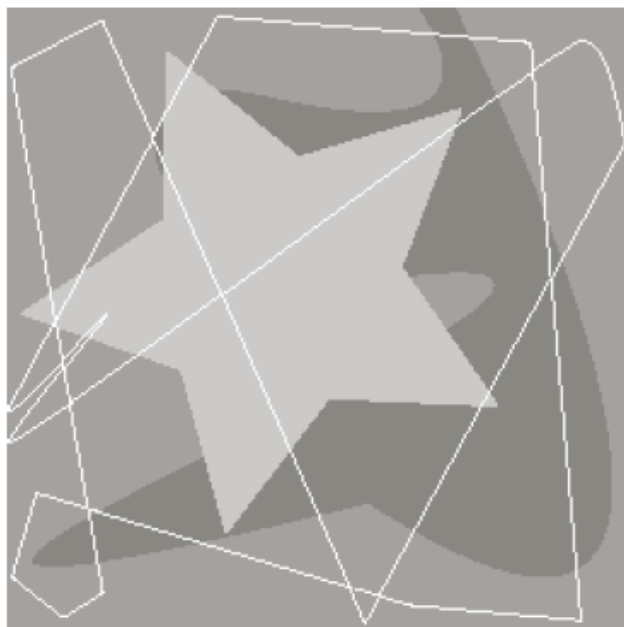
$$\approx 1.6614 \text{ bits/pixel}$$

$$L_{avg} = 1.81 \text{ bits}, 4.42:1$$



冗余的示例

$H=1.66$ bit/pixel
 $L_{avg}=1.81$ bits, 4.42:1



$H=8$ bit/pixel
 $L_{avg}=128:1$



$H=1.56$ bit/pixel
 $L_{avg}=65536:1$



a b c

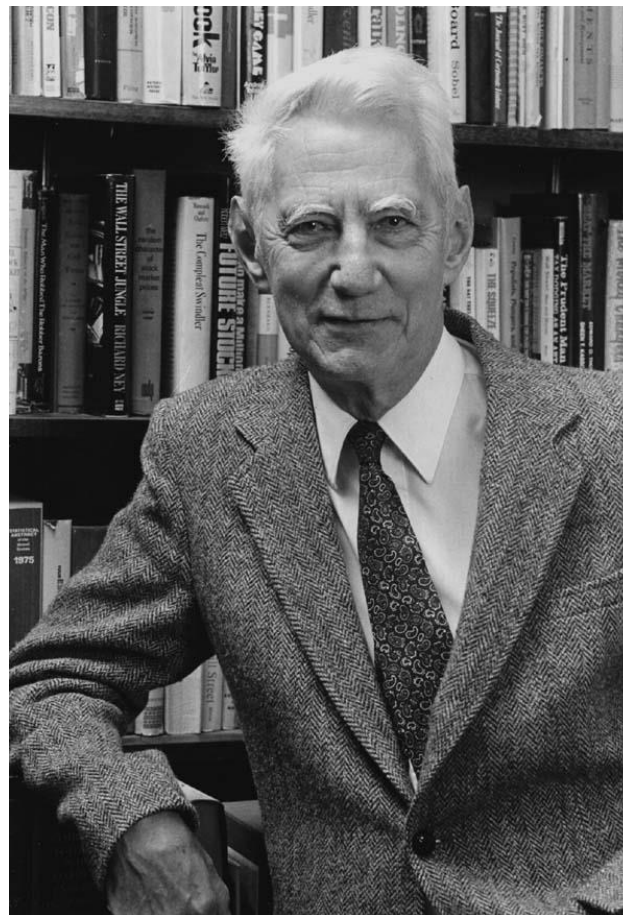
视觉信息丰富，但熵低；视觉信息少，但熵高

FIGURE 8.1 Computer generated $256 \times 256 \times 8$ bit images with (a) coding redundancy, (b) spatial redundancy, and (c) irrelevant information. (Each was designed to demonstrate one principal redundancy but may exhibit others as well.)



信息理论和熵

- 信息理论试图解决在一个嘈杂的信道上尽可能多地传递数据的问题。
- 衡量数据的标准是熵
- 克劳德-香农首次证明了在噪声信道上进行可靠的通信是可能的（启动了数字时代）。





"The fundamental problem of communication is that of reproducing at one point, either exactly or approximately, a message selected at another point."

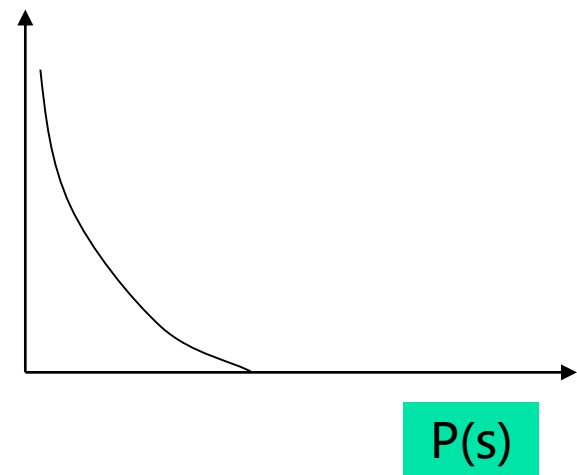
-Shannon, 1944

通信的基本问题是，在一个点上准确或近似地复制在另一个点上选择的信息。



信息的衡量

- s_i 的信息内容
 - (in bits) $I = -\log_2 p(s_i)$
- 示例
 - $p(s_i) = 1$ 没有信息
 - $p(s_i)$ 越小信息越多



A random event E with probability $P(E)$ is said to contain

$$I(E) = \log \frac{1}{P(E)} = -\log P(E)$$

units of information.



熵

- 通过其出现的概率来衡量每个源符号的信息含量：
 - value is called Entropy (H)

$$\sum_{i=1}^n -p(s_i) \log_2 p(s_i)$$

- 每个源输出的平均信息
- 产生用编码表示信息所需的比特数的下限



- Alphabet = {A, B}
 - $p(A) = 0.4$; $p(B) = 0.6$
- Compute Entropy (H)
 - $-0.4 \cdot \log_2 0.4 + -0.6 \cdot \log_2 0.6 = .97$ bits/source
- 最大不确定度(gives largest H)
 - 当所有的概率都相等时发生



■ Shannon entropy

$$H(p) = - \sum_{i=1}^k p(i) \log p(i),$$

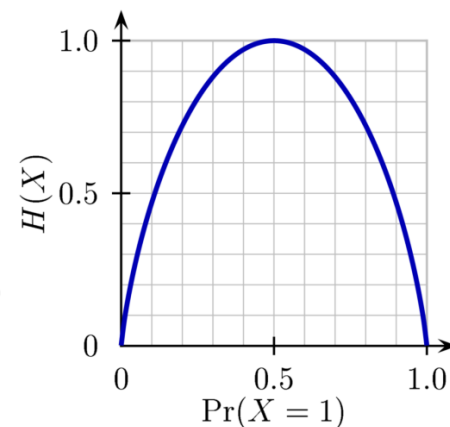
二元熵公式

■ Binary entropy formula

$$H_b(p) = H(p, 1 - p) = -p \log p - (1 - p) \log(1 - p).$$

■ Differential entropy 差分熵

$$h[f] = - \int_{-\infty}^{\infty} f(x) \log f(x) dx,$$





熵的属性

- 可以被表示为 $\log p(x)$ 的期望 (ie $H(X) = E[-\log p(x)]$)
- Is not a function of a variable' s values, is a function of the variable' s probabilities不是一个变量值的函数, 是一个变量概率的函数
- Usually measured in "bits" (using logs of base 2) or "nats" (using logs of base e)通常以 "比特" (使用基数2的对数) 或 "纳特" (使用基数e的对数) 衡量
- Maximized when all values are equally likely (ie uniform distribution)当所有的值都有同样的可能性时最大化 (即均匀分布) 。
- Equal to 0 when only one value is possible当只有一个值时等于0



香农第一定理

■ 无噪声编码定理

$$\lim_{n \rightarrow \infty} \left[\frac{L_{avg,n}}{n} \right] = H$$

$$\hat{H} = - \sum_{k=0}^{L-1} p_r(r_k) \log p_r(r_k)$$

$L_{avg,n}$ is the average number of code symbols required to represent all n – symbol groups. $L_{avg,n}$ 是代表所有 n 个符号组所需的平均代码符号数。

The average number of bits required to represent each pixel is

$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k) \quad \text{表示每个像素所需的平均比特数}$$



熵（理论极限）

$$H = \sum_{i=1}^N -p(s_i) \log_2 p(s_i)$$

$$\begin{aligned} &= -.25 * \log_2 .25 + \\ &\quad -.30 * \log_2 .30 + \\ &\quad -.12 * \log_2 .12 + \\ &\quad -.15 * \log_2 .15 + \\ &\quad -.18 * \log_2 .18 \end{aligned}$$

$$H = 2.24 \text{ bits}$$

Symbol	$P(S)$	Code
A	0.25	11
B	0.30	00
C	0.12	010
D	0.15	011
E	0.18	10



平均编码长度

$$L = \sum_{i=1}^N p(s_i) \text{codelength}(s_i)$$
$$= .25(2) +$$
$$.30(2) +$$
$$.12(3) +$$
$$.15(3) +$$
$$.18(2)$$

$$L = 2.27 \text{ bits}$$

Symbol	$P(S)$	Code
A	0.25	11
B	0.30	00
C	0.12	010
D	0.15	011
E	0.18	10



编码长度与熵的关系

$$L = \sum_{i=1}^N p(s_i) \text{codelength}(s_i)$$

$$H = \sum_{i=1}^N -p(s_i) \log_2 p(s_i)$$

- 当所有概率都是2的负数时，哈夫曼达到了熵的极限
 - i.e., $1/2$; $1/4$; $1/8$; $1/16$; etc.
- $H \leq \text{编码长度} \leq H + 1$



Fidelity Criteria保真度标准

图像输入

图像近似

Let $f(x, y)$ be an input image and $\hat{f}(x, y)$ be an approximation of $f(x, y)$. The images are of size $M \times N$.

均方根误差

The *root - mean - square error* is

$$e_{rms} = \left[\frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[\hat{f}(x, y) - f(x, y) \right]^2 \right]^{1/2}$$



Fidelity Criteria保真度标准

The *mean - square signal - to - noise ratio* of the output image,
denoted SNR_{ms} 均方信噪比

$$\text{SNR}_{\text{ms}} = \frac{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[\hat{f}(x, y) \right]^2}{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[\hat{f}(x, y) - f(x, y) \right]^2}$$

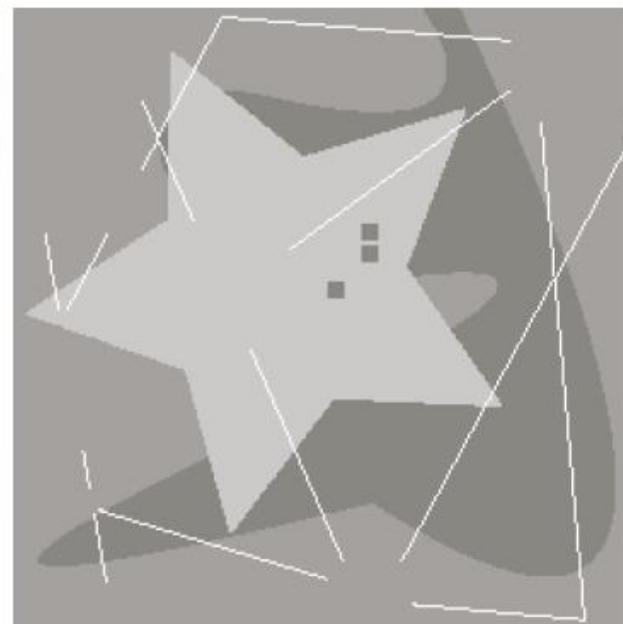
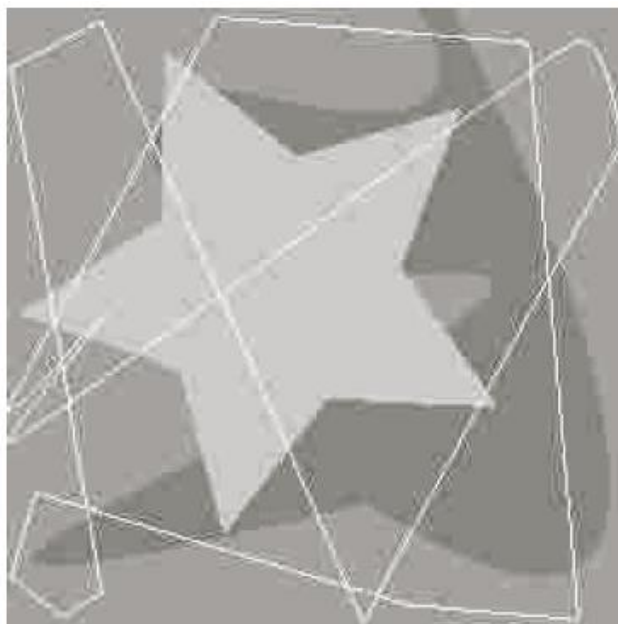
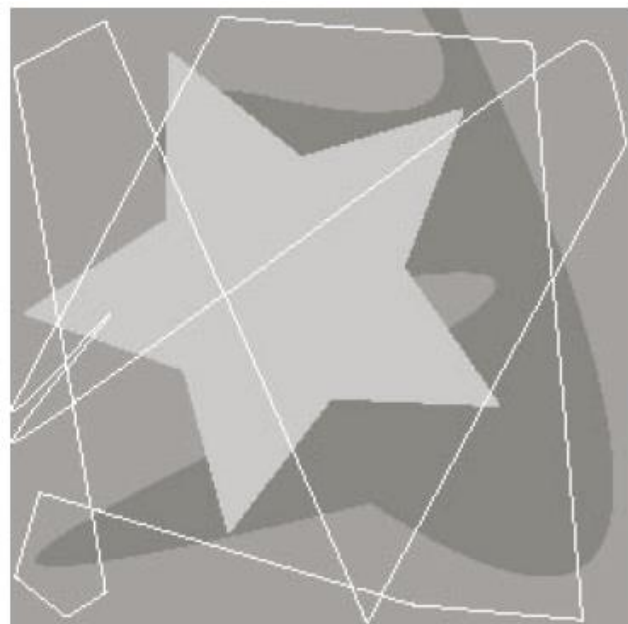
Value	Rating	Description
1	Excellent	An image of extremely high quality, as good as you could desire.
2	Fine	An image of high quality, providing enjoyable viewing. Interference is not objectionable.
3	Passable	An image of acceptable quality. Interference is not objectionable.
4	Marginal	An image of poor quality; you wish you could improve it. Interference is somewhat objectionable.
5	Inferior	A very poor image, but you could watch it. Objectionable interference is definitely present.
6	Unusable	An image so bad that you could not watch it.



RMSE = 5.17

RMSE = 15.67

RMSE = 14.17



a b c

FIGURE 8.4 图8.1(a)中图像的三种近似。



性能参数 (1)

- 两种用于比较各种图像压缩技术的误差指标是均方误差(MSE)和信噪比(SNR)。
- MSE是压缩图像与原始图像之间的累积误差平方，而信噪比是峰值误差的度量。
- 使用了两个客观的衡量标准:

$$MSE = \frac{1}{NM} \sum_{j=0}^N \sum_{k=0}^M [f(j,k) - f'(j,k)]^2$$

$$PSNR(dB) = 10 \log_{10} \frac{(\text{peak_signal})^2}{MSE}$$

Here, *peak_signal* represents *peak-to-peak value of original image data*

其中, peak_signal表示原始图像数据的峰峰值



性能参数 (2)

- 压缩比(CR)
 - 定义为原始比特率与压缩比特率之比
 - 也用每像素位(bpp)或每秒位(bps)表示

- 复杂性
 - 实现编码和解码算法所需的计算工作量
 - 根据算术能力和记忆需求来衡量
 - 也可以根据代码大小和功耗来衡量



性能参数 (3)

■ 通信延迟

- 复杂性也与编码器和解码器的处理延迟有关
- 在实时应用中非常重要



目录

- 编码冗余
- **信源编码器和解码器**
- 霍夫曼编码
- 算术编码
- LZW编码
- 行程长度编码
- 无损预测编码
- 块变换编码

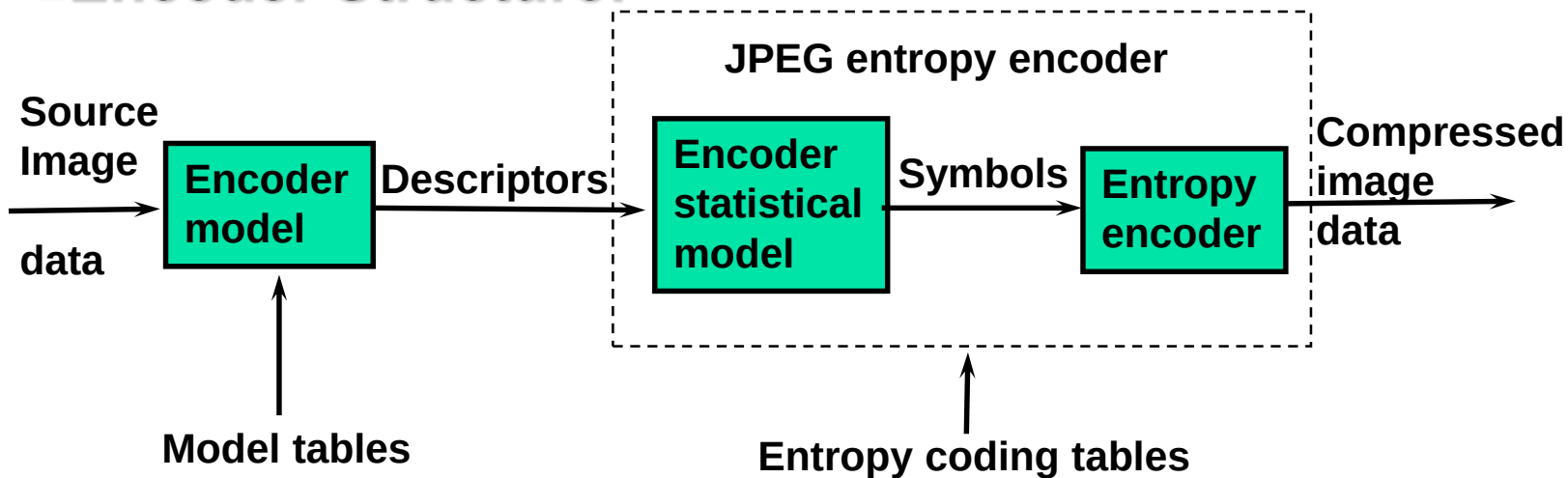


图像压缩系统(1)

■ 图像压缩系统的基本结构



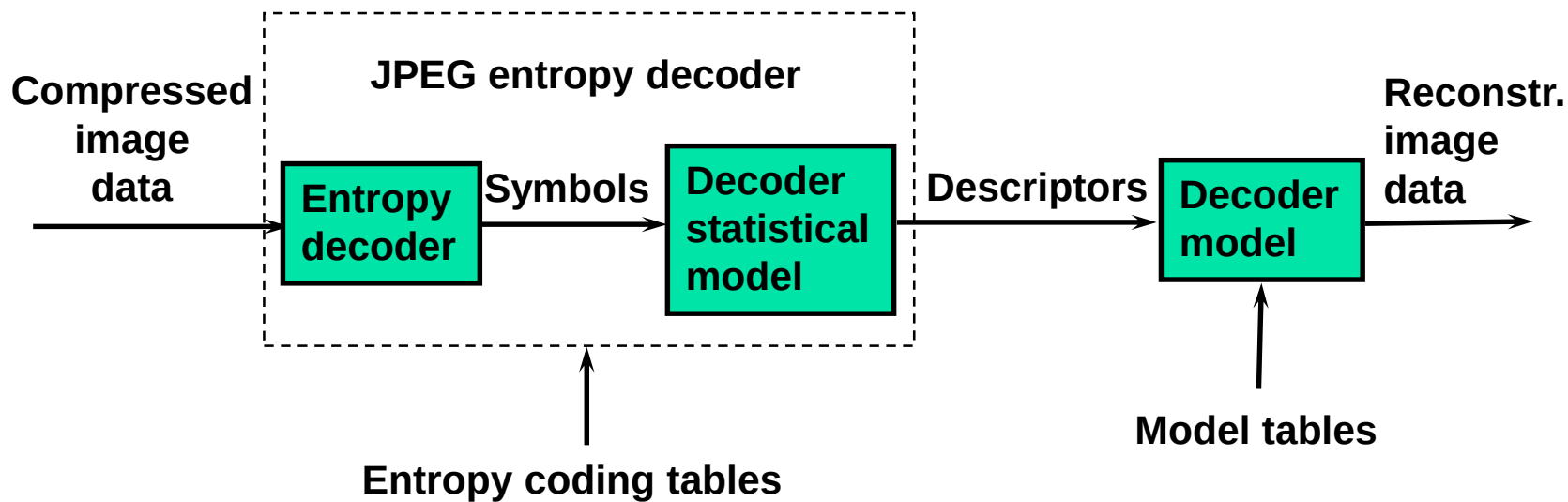
■ Encoder Structure:





图像压缩系统 (2)

■ 解码器结构:

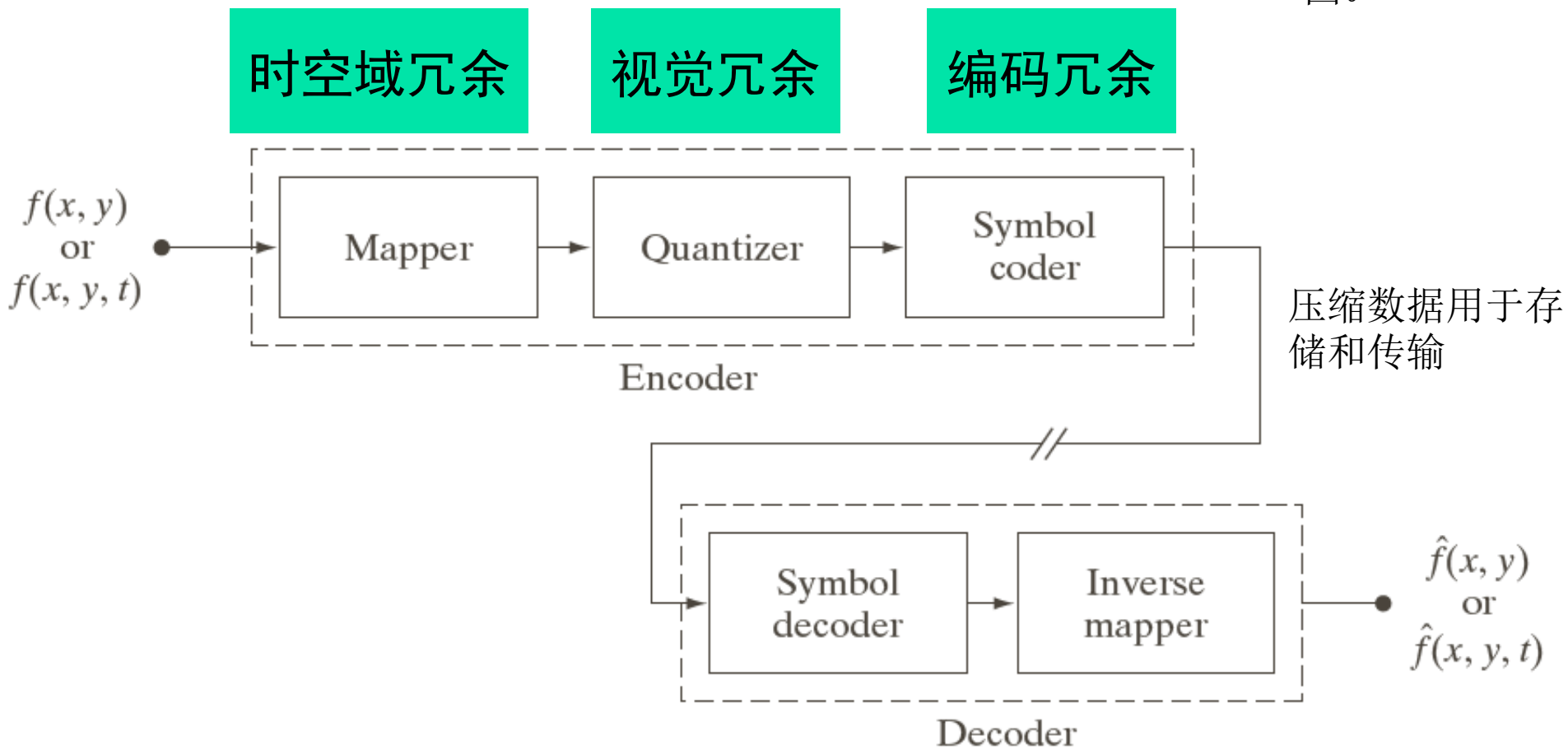




-图像压缩模型

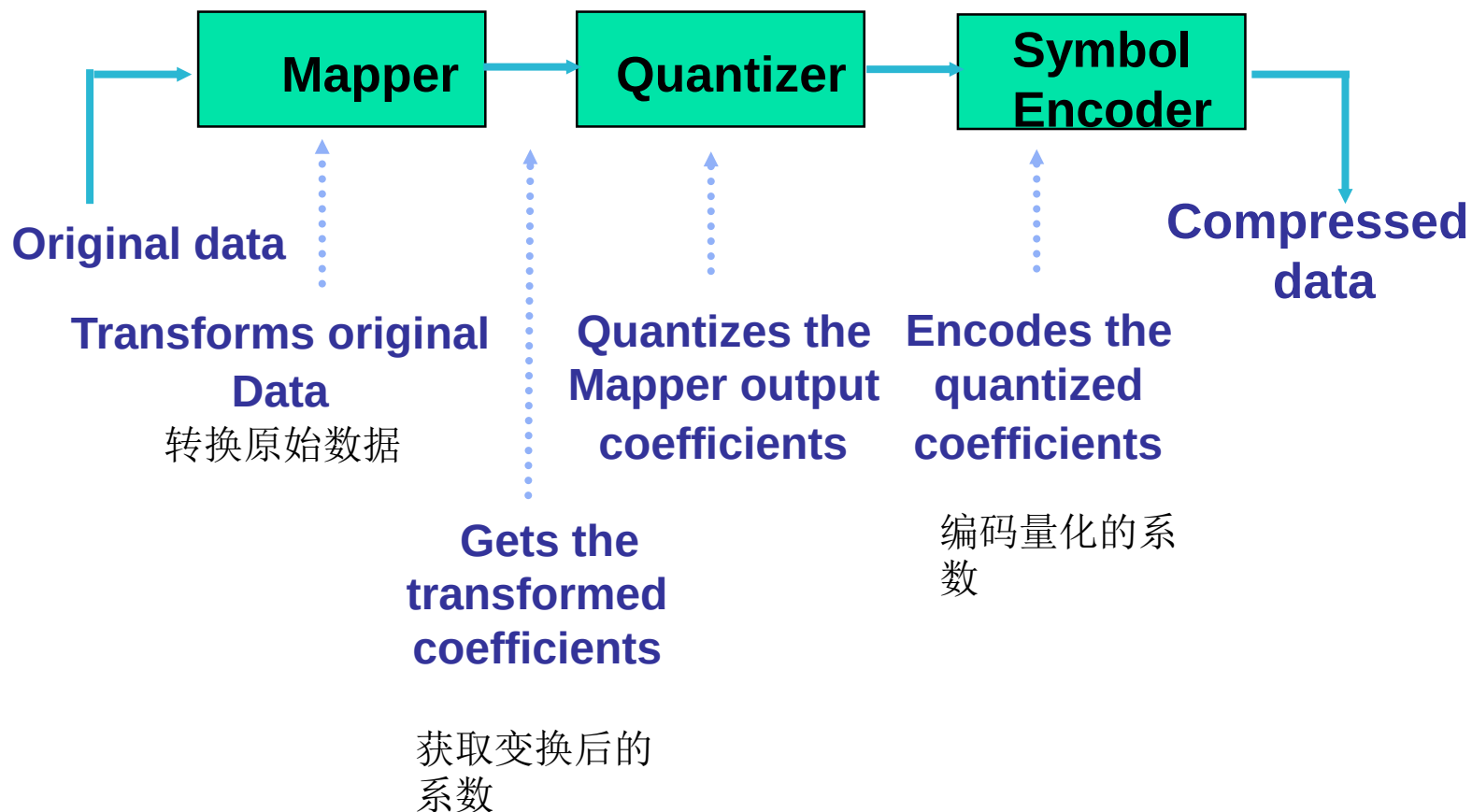
FIGURE 8.5

通用图像压缩系统的功能框图。





图像编码





熵编码器

- 在对数据进行量化之后，使用熵编码器进行额外的压缩。
 - By entropy, 通过熵，数据中存在的信息量，即最小比特数
 - 霍夫曼编码和算术编码。
-
- A good introduction to Huffman coding can be found in
 - "A Closer Look at Huffman Encoding" at <http://www.rasip.fer.hr/research/compress/algorithms/fund/huffman/>
 - A good introduction to Arithmetic coding can be found in
 - "Arithmetic Coding+Statistical Modelling=Data Compression" at <http://dogma.net/markn/articles/arith/part1.htm>
 - H.Witten, M.Neal and G.Cleary, "Arithmetic Coding For data Compression", [*Communications of the ACM*](#), Vol. 30, No. 6 (June 1987), Pages 520-540.



图像编码 (1)

■ Mapper

- 将输入数据转换为=减少像素间冗余的格式
- 便于在后期利用冗余
- 一般操作可逆
- 可能会或可能不会直接导致压缩
 - 行程长度编码
 - 离散余弦变换
 - DPCM中的预测和误差



图像编码(2)

■ 量化器

- 降低了映射器输出的准确性
- 根据心理视觉特性设计
- 不可逆操作，不用于无损压缩

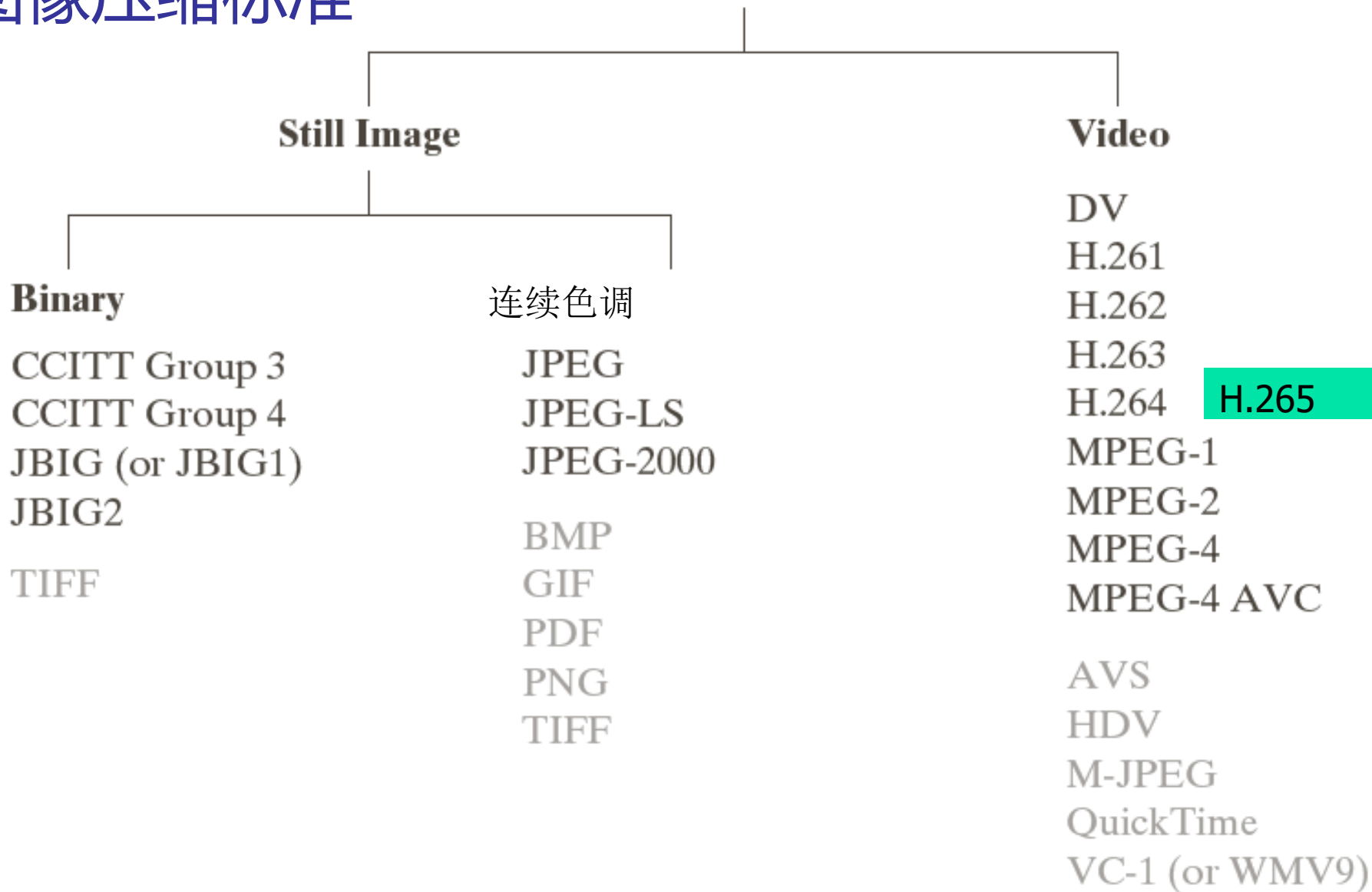
■ 符号编码器

- 用固定或表示量化器输出变长码
 - 例如，将最短的码字分配给出现频率最高的符号
- 可逆的



Image Compression Standards, Formats, and Containers

图像压缩标准





Compression Ratio Rules of Thumb

压缩比的经验法

- 压缩比保持好的质量:
 - – 10:1 for general images using JPEG
 - – 15:1 for general images using JPEG2000
 - – 30:1 for general video using H.263 and MPEG-2
 - – 50:1 for general video using H.264 / MPEG-4 AVC
 - – 100:1 for general video using HEVC/H.265



目录

- 编码冗余
- 信源编码器和解码器
- **霍夫曼编码**
- 算术编码
- LZW编码
- 行程长度编码
- 无损预测编码
- 块变换编码



一些基本压缩方法: 霍夫曼编码

1952-Huffman

FIGURE 8.7
Huffman source
reductions.

霍夫曼源缩减

Original source		Source reduction			
Symbol	Probability	1	2	3	4
a_2	0.4	0.4	0.4	0.4	0.6
a_6	0.3	0.3	0.3	0.3	
a_1	0.1	0.1	0.2	0.3	0.4
a_4	0.1	0.1	0.1	0.3	
a_3	0.06	0.1			
a_5	0.04				



一些基本压缩方法: 霍夫曼编码

FIGURE 8.8
Huffman code
assignment
procedure.

Original source			Source reduction			
Symbol	Probability	Code	1	2	3	4
a_2	0.4	1	0.4	1	0.4	1
a_6	0.3	00	0.3	00	0.3	00
a_1	0.1	011	0.1	011	0.2	010
a_4	0.1	0100	0.1	0100	0.1	011
a_3	0.06	01010	0.1	0101	0.3	01
a_5	0.04	01011				

这段代码的平均长度为

$$\begin{aligned} L_{avg} &= 0.4 * 1 + 0.3 * 2 + 0.1 * 3 + 0.1 * 4 + 0.06 * 5 + 0.04 * 5 \\ &= 2.2 \text{ bits/pixel} \end{aligned}$$

H=2.14bits/pixel



解码

- 010100111100
- a_3, a_1, a_2, a_2, a_6

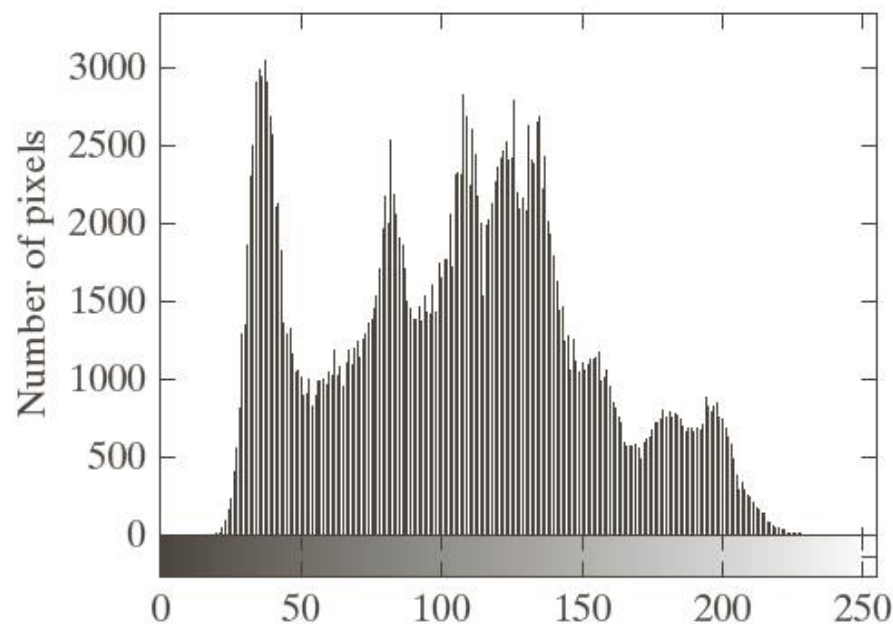
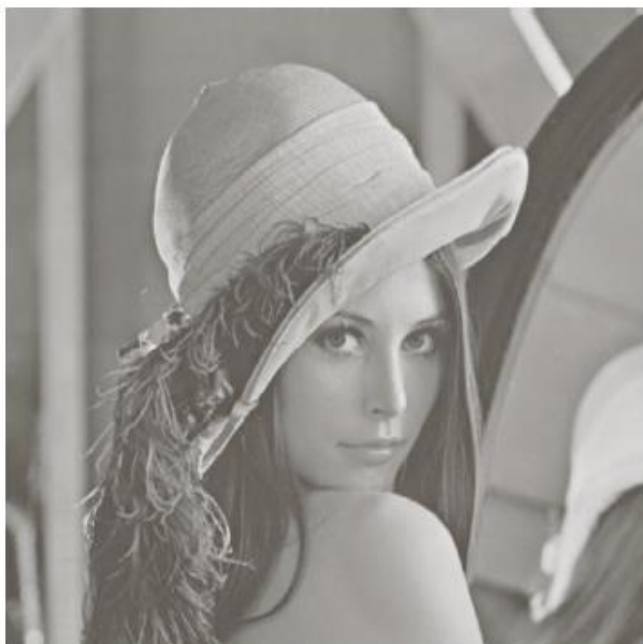
Original source	
Symbol	Code
a_2	1
a_6	00
a_1	011
a_4	0100
a_3	01010
a_5	01011



- 当对信息源的符号进行单独编码时，霍夫曼编码对每个源符号产生尽可能少的代码符号
- 这些符号一次编码一个。



FIGURE 8.9 (a)
A 512×512 8-bit
image, and (b) its
histogram.



$512 \times 512 \times 8$

$H = 7.3838$ bits/pixel

$$H = \sum_{i=1}^N -p(s_i) \log_2 p(s_i)$$

霍夫曼编码: 7.428位/像素, 包括霍夫曼表

$512 \times 512 \times (7.428 - 7.3838)$ bits/pixel = 11587 bits. 0.6%

$$C = 8 / 7.428 = 1.077$$
$$R = 1 - 1 / 1.077 = 0.0715$$



目录

- 编码冗余
- 信源编码器和解码器
- 霍夫曼编码
- **算术编码**
- LZW编码
- 行程长度编码
- 无损预测编码
- 块变换编码



算术编码

How to encode $a_1 a_2 a_3 a_4$?

Source Symbol	Probability	Initial Subinterval
a_1	0.2	$[0.0, 0.2)$
a_2	0.2	$[0.2, 0.4)$
a_3	0.4	$[0.4, 0.8)$
a_4	0.2	$[0.8, 1.0)$

TABLE 8.6

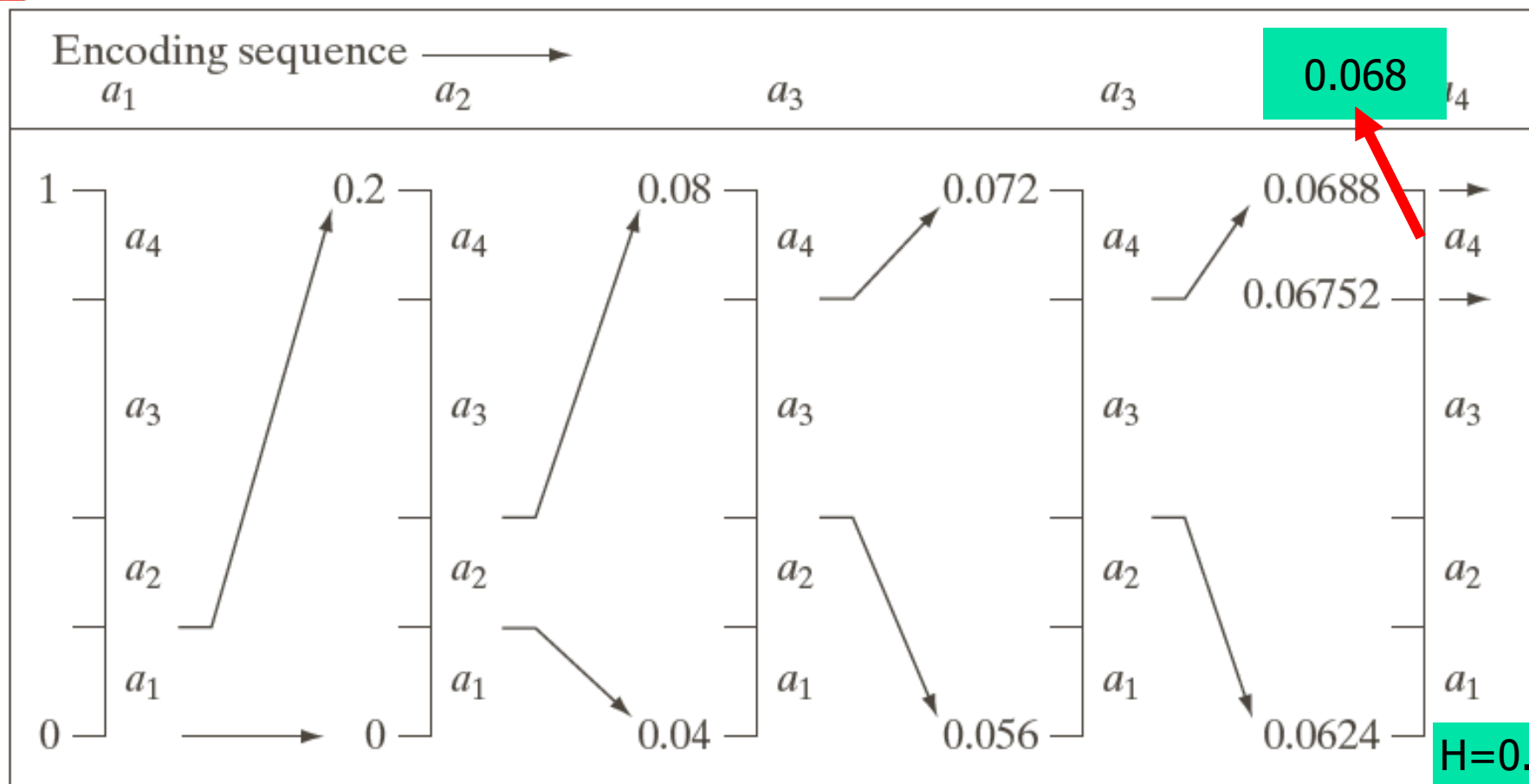
Arithmetic coding
example.

表8.6

算术编码示例。



FIGURE 8.12
Arithmetic coding
procedure.



Source Symbol

Probability

Initial Subinterval

a_1	0.2	$[0.0, 0.2)$
a_2	0.2	$[0.2, 0.4)$
a_3	0.4	$[0.4, 0.8)$
a_4	0.2	$[0.8, 1.0)$

1963 Abramson
Generates nonblock codes
No one-to-one correspondence

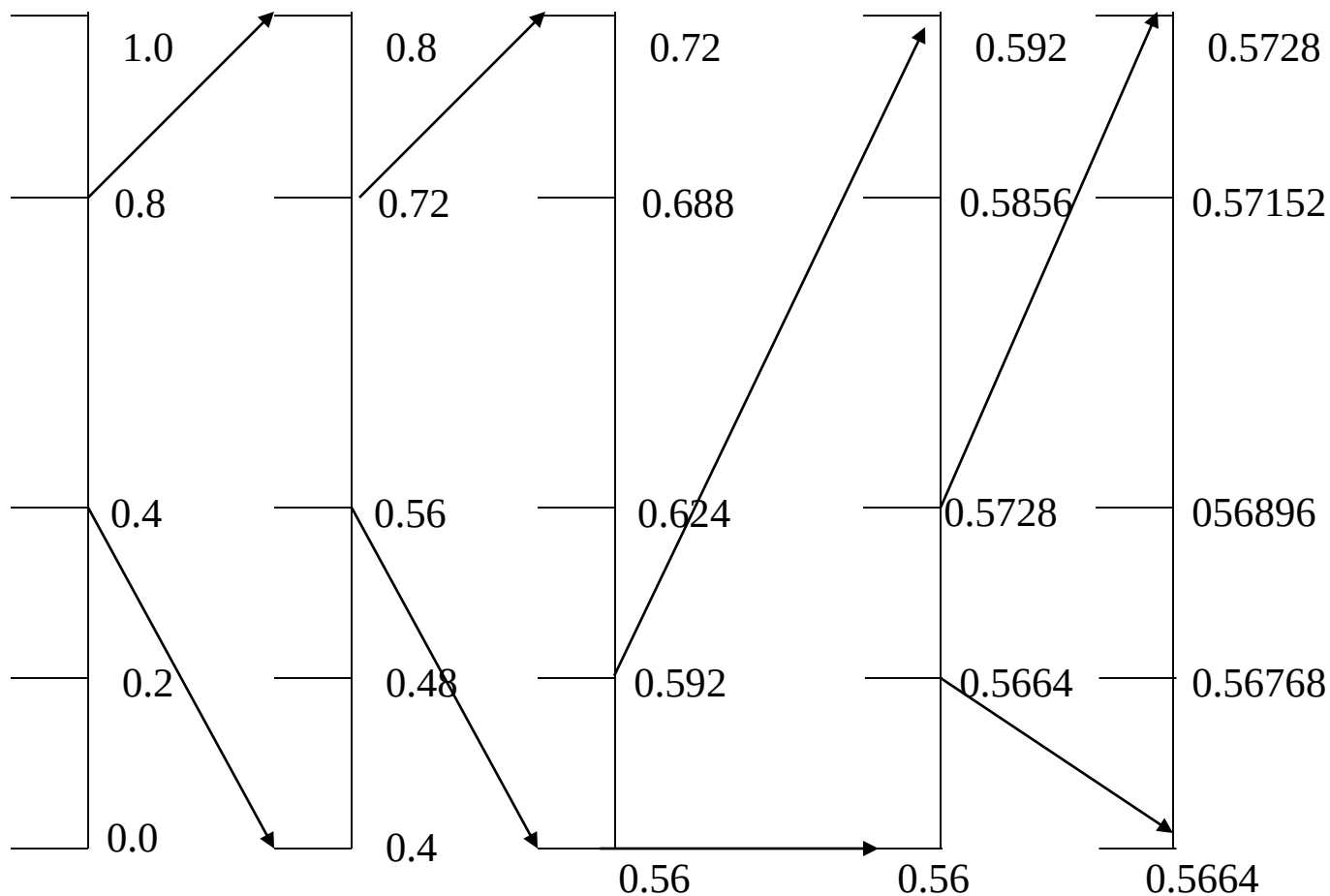
$3/5=0.6$ decimal digits/ source symbol

$3/5=0.6$ 个十进制 数字/源符号



0.572解码。消息长度为5。

因为 $0.8 > \text{码字} > 0.4$ ，所以第一个符号应该是 a_3 。



因此,信息是
 $a_3a_3a_1a_2a_4$

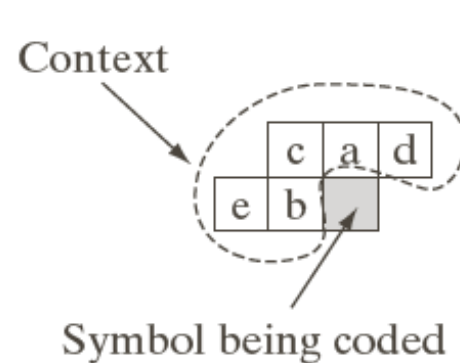
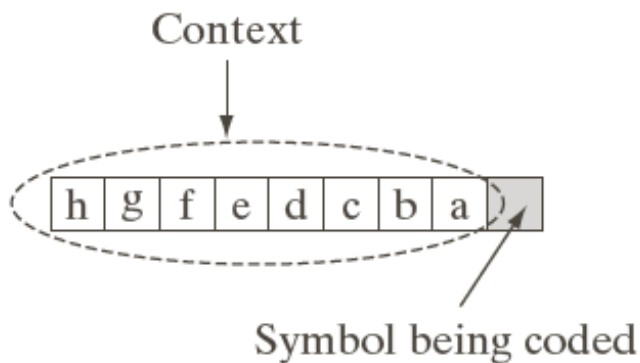
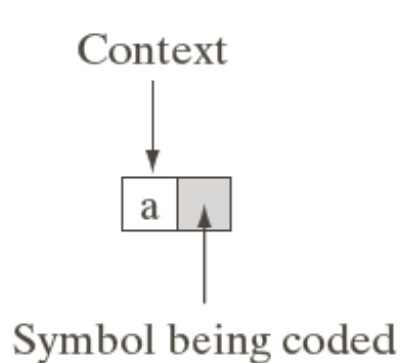
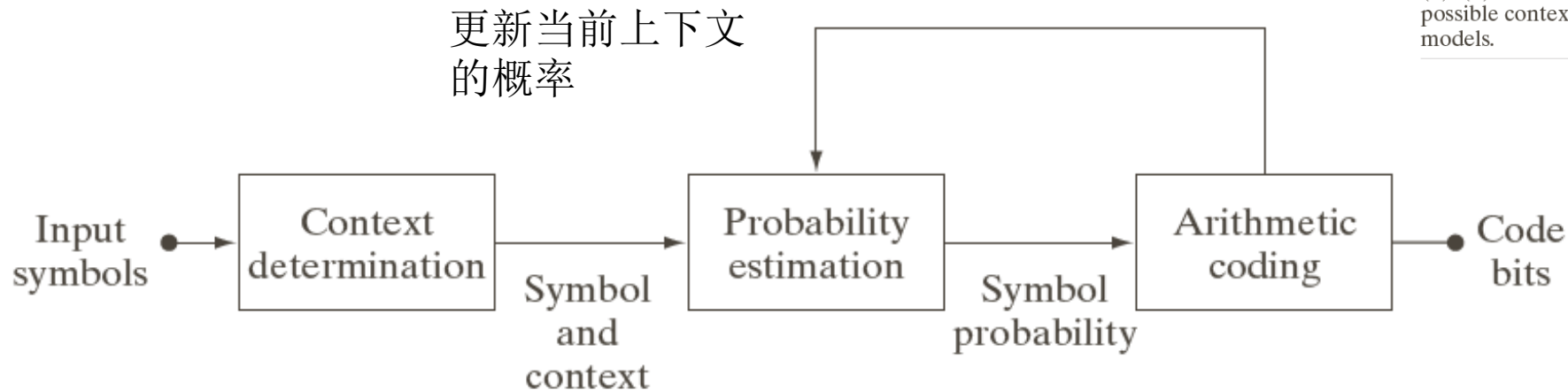


- 霍夫曼编码:每个源符号转换成一个整数的代码符号(一次编码一个);
- 随着编码序列的长度增加, 得到的算术代码接近香农第一定理所建立的边界。
- 性能下降:
 - 添加end-of-message indicator (消息结束指示器)
 - 有限精度算法。
 - 引入缩放策略($\text{renormalize}[0,1)$)和舍入策略



精确的概率模型
可以得到最优的编码

更新当前上下文的
概率



a
b c d

FIGURE 8.13
(a) An adaptive, context-based arithmetic coding approach (often used for binary source symbols). (b)–(d) Three possible context models.

Track $P(1|a=0)$ $P(0|a=0)$ $P(1|a=1)$ $P(0|a=1)$



目录

- 编码冗余
- 信源编码器和解码器
- 霍夫曼编码
- 算术编码
- **LZW编码**
- 行程长度编码
- 无损预测编码
- 块变换编码



LZW (Dictionary coding)(字典编码)

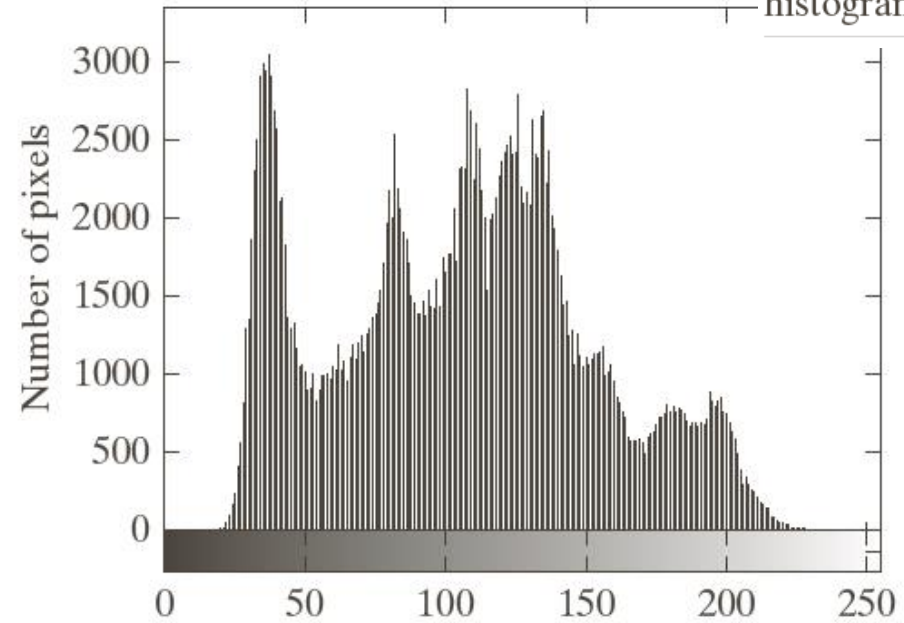
LZW was formulated in 1984

LZW (Lempel-Ziv-Welch)编码，将固定长度的码字分配给可变长度的源符号序列，但不需要源符号概率的先验知识。

LZW用于：

- 标记图像文件格式(TIFF)
- 图形交换格式(GIF)
- 便携式文件格式(PDF)

LZW于1984年制定



a b

FIGURE 8.9 (a)
A 512×512 8-bit
image, and (b) its
histogram.

$512 \times 512 \times 8$

$H = 7.3838$ bits/pixel

$$H = \sum_{i=1}^N -p(s_i) \log_2 p(s_i)$$

TIFF: 286740 bytes = 262144 for pixels + 24596 overhead
LZW: 224420 bytes

$C = 1.28$

Huffman: $C = 1.077$

去除空间冗余



算法:

- 一个包含源符号的码本或“字典”被构造。
- 对于8位单色图像，字典的前256个单词被分配到0-255的灰度级别
- 字典的剩余部分由灰度序列填充



LZW的重要特征

1. 在对数据进行编码时创建字典。所以编码可以在动态中完成
2. 字典不需要传输。词典将在解码过程中建立起来
3. 如果字典“溢出”，则必须重新初始化字典，并为每个码字添加位。
4. 选择较大的字典大小可以避免溢出，但会破坏压缩



例子:

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126



EXAMPLE 8.7: LZW coding.

Dictionary Location	Entry
0	0
1	1
⋮	⋮
255	255
256	—
⋮	⋮
511	—



39 39 126 126
39 39 126 126
39 39 126 126
39 39 126 126



Currently Recognized Sequence	Pixel Being Processed	Encoded Output	Dictionary Location (Code Word)	Dictionary Entry
	39			





- $4*4*8=128\text{bits}$

- LZW:

- $9*10=90\text{bits};$

- $C=1.42$



Recognized	Encoded value	pixels	Dic. address	Dic. entry
	39			





解码 LZW

设接收到的比特流为:

39 39 126 126 256 258 260 259 257 126

In LZW, the dictionary which was used for encoding need not be sent with the image. A separate dictionary is built by the decoder, on the “fly”, as it reads the received code words.

在LZW中，用于编码的字典不需要随图像一起发送。解码器在“fly”的过程中读取接收到的码字，构建一个单独的字典。



Recognized	Encoded value	pixels	Dic. address	Dic. entry
	39			



目录

- 编码冗余
- 信源编码器和解码器
- 霍夫曼编码
- 算术编码
- LZW编码
- **行程长度编码**
- 无损预测编码
- 块变换编码



一些基本压缩方法:

行程长度编码

1. 运行长度编码(Run-length Encoding, RLE), 是一种用于减少重复字符字符串大小的技术。
2. 这种重复的字符串称为运行, 通常RLE将运行的符号编码为两个字节, 一个计数和一个符号。
3. RLE可以压缩任何类型的数据Is connected to the Internet to identify and translation...
4. 与其他压缩方法相比, RLE不能实现高压缩比



一些基本压缩方法: 行程长度编码

- 5. 它易于实现，执行速度快。
- 6. 大多数bitmap文件格式(如TIFF、BMP和PCX)都支持运行长度编码

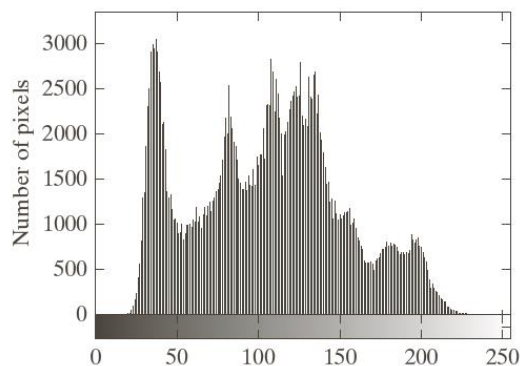
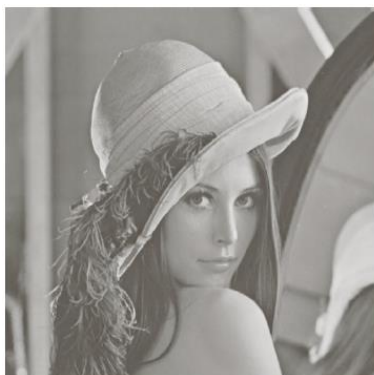
5. Effective for binary images, further VLC coding

6. $C=0.98$ for lena image,

$C=1.35$

没有足够的等
强度运行

No enough
Equal intensity
runs





Run-Length Coding 行程长度编码

WWWWWWWWWWWWWWWWWWBWWWWWWWWWWWWWWWWBBBW
WWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWBWWWW
WWWWWWWWWWWWWWWW

RLE coding:

12W1B12W3B24W1B14W



目录

- 编码冗余
- 信源编码器和解码器
- 霍夫曼编码
- 算术编码
- LZW编码
- 行程长度编码
- **无损预测编码**
- 块变换编码



Differential Pulse Code Modulation.

差分脉冲编码调制 (DPCM)

- 示例:

AAABBCDDDD encoded as A0001123333

- 如果增量太大，请更改参考符号
- 对于许多数字图像，工作效果比RLE更好



目录

- 编码冗余
- 信源编码器和解码器
- 霍夫曼编码
- 算术编码
- LZW编码
- 行程长度编码
- **符号编码**
- 无损预测编码
- 块变换编码



一些基本压缩方法:

Symbol-Based Coding

在基于符号或标记的编码中，图像被表示为经常出现的子图像(称为符号)的集合。

每个符号都存储在一个符号字典中

图像被编码为一组三胞胎

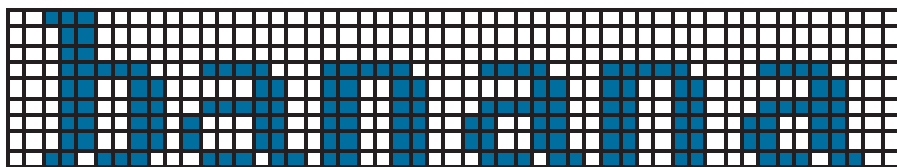
$\{(x1, y1, t1), (x2, y2, t2), \dots\}$

1974

文档存储和检索应用程序



Symbol-Based Coding



$$9 \times 51 \times 1 = 459 \text{ bits}$$

a b c

FIGURE 8.17

(a) A bi-level document, (b) symbol dictionary, and (c) the triplets used to locate the symbols in the document.

图8.17

(a) 两级文件,
(b) 符号字典
(c) 文件中用于
定位符号的三
元组。

Token	Symbol	Triplet
0		(0, 2, 0) (3, 10, 1) (3, 18, 2) (3, 26, 1) (3, 34, 2) (3, 42, 1)
1		
2		

$$6 \times 3 \times 8 + 9 \times 7 + 6 \times 7 + 6 \times 6 = 285 \text{ bits}$$

$$C = 1.61$$



大语言模型的本质是压缩器

- 数据的压缩和信息中每个字符出现的概率有关系
- 在实际场景中，每个字符出现的概率是不同的，比如汉语中的“稿”和“的”出现的概率明显不同，“的”出现的概率会高很多，在这种情况下，如果把出现概率高的字符编码成更短的二进制数，把出现概率低的字符编码位相对长的二进制数，可以节省整体编码的二进制数。
- 大语言模型本质上是一个概率预测模型，即在给定上下文的情况下，模型给出字符表中各个字符出现在下一个位置的概率，然后我们通常选取概率最高的那个字符作为预测值，所以预测模型依赖一个概率分布。
- 通常说大模型在训练过程中学习“知识”，这个“知识”是什么呢，其实就是概率分布。



大语言模型的本质是压缩器

- 找到解决感知问题的最小描述长度
- 生成模型是无损压缩器
- 大语言模型是 SOTA 的无损文本压缩器

Lossless Compression with LLMs

So let's put this together, Satya wants to send a dataset of the world's knowledge D to Sundar. Both have access to supercomputers, but there's a low bandwidth connection.

Satya → Sundar

Powered by Zoom 20

假设这么一个场景，Satya 想要传输包含世界知识的数据集 D 给 Sundar，他们都可以连接到超级计算机，但带宽连接很小。



大语言模型的本质是压缩器

Lossless Compression with LLMs: how!?

We can make use of arithmetic encoding:

We have a token x_t at timestep t from vocab V , and a probability distribution p_t over tokens.

1. $\text{arithmetic_encode}(x_t | p_t) \rightarrow z_t$ where z_t takes up $-\log_2 p_t(x_t)$ bits.
2. $\text{arithmetic_decode}(z_t | p_t) \rightarrow x_t$ i.e. still need p_t to decode.

Example 1 : if p_t is uniform i.e. $1/|V|$ this uses $\log_2 |V|$ space. Same as naive storage!

Example 2: if p_t is an oracle i.e. $p_t(x_t) = 1$, this uses zero space.

Jack Rae

Powered by Zoom

21

- 可以使用算术编码，假设我们在时间戳 t 有一个 token x_t ，并且概率分布是 p_t 。简单来说，算术编码就是将 token x_t 映射为 z_t （压缩后的表征）， z_t 占用 $-\log_2 p_t(x_t)$ bits。事实上真正实现算术编码可能需要无限精度的计算，而在现实使用时往往需要付出额外的 bit，但算术编码在编码方面大致可以算是最优的。解码 z_t 时仍然需要用到分布 p_t 。
- 以 p_t 是均匀分布为例，此时存储 x_t 需要 $\log_2 |V|$ 的空间，这跟直接二进制存储是一样的。当我们确切地知道 x_t 是什么（即 $p_t(x_t) = 1$ ）时，不需要耗费存储空间。这是两个极端的例子，说明了我们需要一个足够好的生成模型来塑造我们的数据，它可以使用更少的空间。



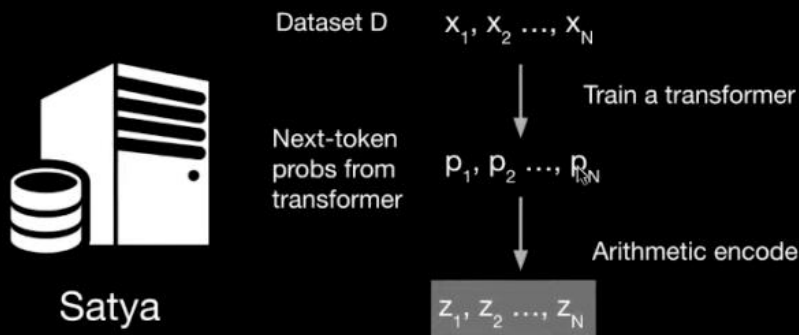
大语言模型的本质是压缩器

Lossless Compression with LLMs

Satya has the code to instantiate and deterministically train an LLM, it's ~1MB

He trains an LLM over the dataset and records next-token probabilities.

Then he encodes this with arithmetic encoding.



```
Code
import torch
...
n_layers = 96
d_model = 16384
....
```

所以在实践中我们应该怎么做？Satya 在数据集 D 上训练一个 transformer，然后记录 next-token 概率分布，然后使用算术编码来对数据进行编码。



大语言模型的本质是压缩器



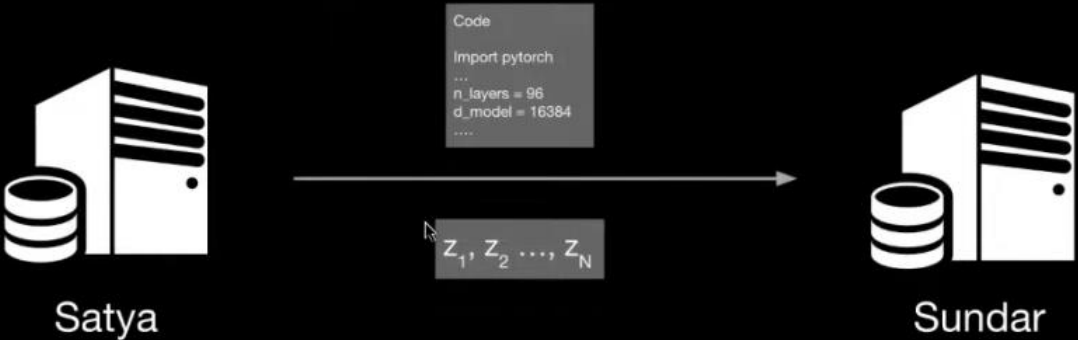
Jack Rae

Lossless Compression with LLMs

Satya sends the code, and the arithmetic encoded transcript.

Total bits: training code = ~1MB

+ encoded transcript = $-\sum \log_2 p_t(x_t)$ bits



```
Code
Import pytorch
...
n_layers = 96
d_model = 16384
...
```

$z_1, z_2, \dots, z_N$

Satya Sundar

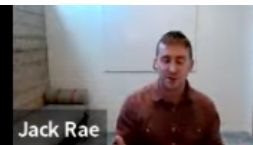
Powered by Zoom

23

Satya 把代码和算术编码后的 z 发送给 Sundar，训练代码约为 1MB，编码抄本为 $-\log_2 p_t(x_t)$ 。也就是说实际上，他并没有发送神经网络（可能几千万甚至上亿参数），而只是发送了训练的代码。



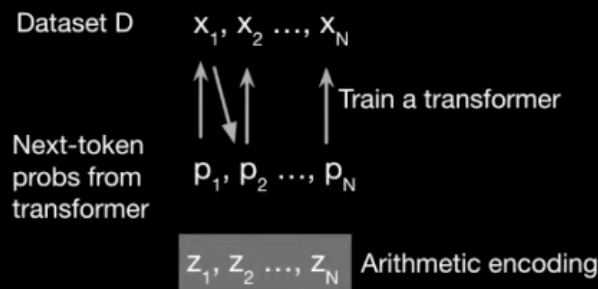
大语言模型的本质是压缩器



Lossless Compression with LLMs

Sundar runs the deterministic training code:

- It initializes the same initial network, at time-step 1 the model emits p_1
- Using arithmetic decoding: $\text{decode}(z_1 \mid p_1) \rightarrow x_1$
- Take a training step, network predicts p_1 and repeat... Until D is fully decoded.



Sundar

Powered by Zoom

25

在 Sundar 这端，他运行确定的训练代码：使用相同的初始化网络进行初始化，在时间戳 1 模型使用 P_1 对 Z_1 进行算术解码得到 X_1 ，然后经过一个训练步长使得网络预测出 P_1 ，之后迭代地遍历并恢复整个数据集。这看起来更像是一个思想实验，因为这需要大量的中间计算资源才能达到如此高的压缩率。有个有趣的事实是，利用大语言模型进行无损压缩的想法与香农的信息熵定理不谋而合。



大语言模型的本质是压缩器

A recipe for perception

1. Collect all useful perceptual information
2. Learn to compress it as best as possible with a powerful foundation model

You can use any research method to improve compression:

- Better architecture
- Scale
- Tool use
- Retrieval over the past
- Synthetic data



之前我们讨论了为什么大语言模型是最先进的无损压缩器，现在我们讨论如何解决感知问题并转向 AGI 的诀窍是什么。这主要分为两步：一是收集所有有用的感知信息，二是学习使用强大的基础模型来尽可能地压缩数据。这样做的好处是你可以从任意的角度，使用任意的研究方法来提高压缩的效率，例如改进网络架构、使用生成数据来辅助生成等。



目录

- 编码冗余
- 信源编码器和解码器
- 霍夫曼编码
- 算术编码
- LZW编码
- 行程长度编码
- 无损预测编码
- **块变换编码**



块变换编码

a
b

将图像划分为子图像(8×8)
使用二维变换对块进行独立处理

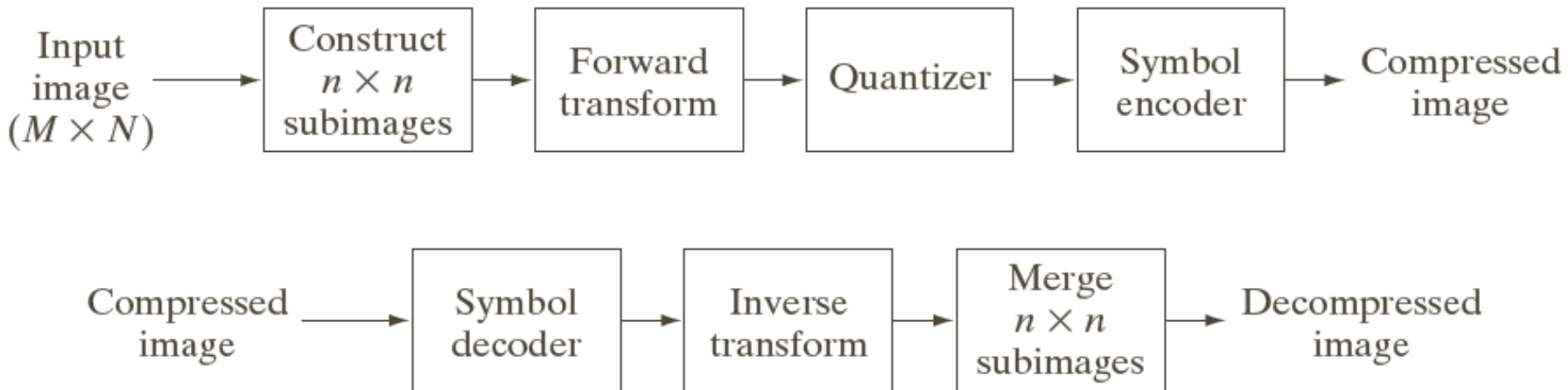
图8.21
块变换编码系统:
(a)编码器;
(b)译码器。

速度快, 内存少

解关联每个
子图像的像素

失真

编码冗余





块变换编码

Transform selection depends on the amount of reconstruction error that can be tolerated and the computational resources available

变换的选择取决于可容忍的重构误差量和可用的计算资源

考虑一个大小为 $n \times n$ 的子图像，它的前向离散变换 $T(u,v)$ 可以用下面的关系表示

$$T(u, v) = \sum_{x=0}^{n-1} \sum_{y=0}^{n-1} g(x, y) r(x, y, u, v)$$

基本图像/函数

for $u, v = 0, 1, 2, \dots, n-1$.



给定 $T(u, v)$, $g(x, y)$ 同样可以用广义逆离散变换得到

$$g(x, y) = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(x, y, u, v)$$

基本图像/函数

for $x, y = 0, 1, 2, \dots, n-1$.



Separable可分

DFT

$$r(x, y, u, v) = e^{-j2\pi(ux+vy)/n}$$

$$s(x, y, u, v) = \frac{1}{n^2} e^{j2\pi(ux+vy)/n}$$



图像变换

- Two main types:
 - orthogonal transform**:
e.g. Walsh-Hadamard transform, DCT

两种主要类型:

正交变换:

例如Walsh-Hadamard变换, DCT

- subband transform**:

e.g. Wavelet transform

部分波段转换

示例: 小波变换



正交变换

■ 正交矩阵 W

$$\begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \\ d_3 \\ d_4 \end{pmatrix}$$

$$\rightarrow C = W \cdot D$$

- Reducing redundancy
- 减少冗余
- Isolating frequencies
- 隔离频率



图像变化DFT

Given $T(u, v)$, $g(x, y)$ similarly can be obtained using the generalized inverse discrete transform 给定 $T(u, v)$, $g(x, y)$ 同样可以用广义逆离散变换得到

$$g(x, y) = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(x, y, u, v)$$

for $x, y = 0, 1, 2, \dots, n-1$.

$$G = \begin{bmatrix} \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, 0, u, v) & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, 1, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, n-1, u, v) \\ \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(1, 0, u, v) & & \cdots & \\ & & \cdots & \\ \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, 0, u, v) & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, 1, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, n-1, u, v) \end{bmatrix}$$

$$\chi(u, v) = \begin{cases} 0 & \text{satisfy a specified truncation criterion} \\ 1 & \text{otherwise} \end{cases}$$

Base images



图像变化DFT

$$G = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(x, y, u, v) \quad s(x, y, u, v) = \frac{1}{n^2} e^{j2\pi(ux+vy)/n}$$

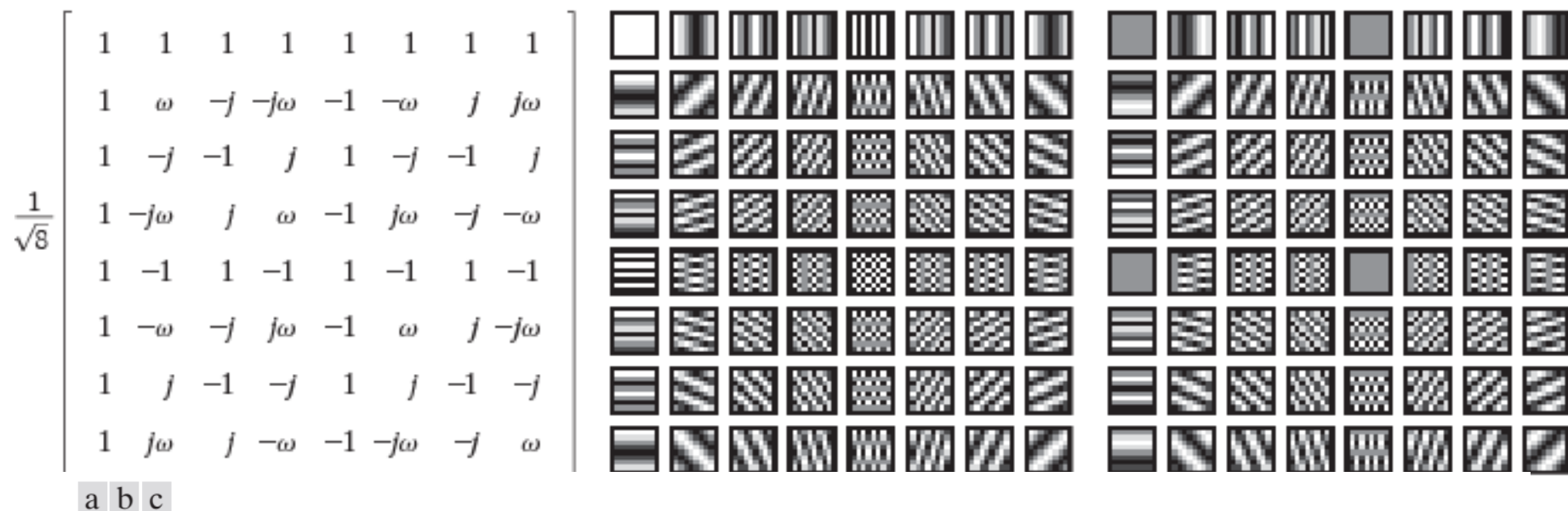


FIGURE 7.7 (a) Transformation matrix $\mathbf{A}_F$ of the discrete Fourier transform for $N=8$, where $\omega = e^{-j2\pi/8}$ or $(1-j)/\sqrt{2}$. (b) and (c) The real and imaginary parts of the DFT basis images of size 8×8 . For clarity, a black border has been added around each basis image. For 1-D transforms, matrix $\mathbf{A}_F$ is used in conjunction with Eqs. (7-43) and (7-44); for 2-D transforms, it is used with Eqs. (7-41) and (7-42).

Base images



块变换编码

Walsh-Hadamard变换(WHT)

$$r(x, y, u, v) = s(x, y, u, v) = \frac{1}{n} (-1)^{\sum_{i=0}^{m-1} [b_i(x) p_i(u) + b_i(y) p_i(v)]}$$

$b_i(z)$: i th bit from right to left in the binary representation of z .

$$n = 2^m$$

$$p_0(u) = b_{m-1}(u)$$

$$p_1(u) = b_{m-1}(u) + b_{m-2}(u)$$

$$p_2(u) = b_{m-2}(u) + b_{m-3}(u)$$

$\vdots$

$$p_{m-1}(u) = b_1(u) + b_0(u)$$

$$u = 0, v = 1, n = 4, m = 2, x, y = 0, 1, 2, 3;$$

$$u = 0, b_i(x) p_i(u) = 0$$

$$b_0(0) = 0, b_0(1) = 1, b_0(2) = 0, b_0(3) = 1;$$

$$b_1(0) = 0, b_1(1) = 0, b_1(2) = 1, b_1(3) = 1;$$

$$p_0(1) = b_1(1) = 0$$

$$p_1(1) = b_1(1) + b_0(1) = 1$$

$$\sum_{i=0}^{m-1} [b_i(x) p_i(u) + b_i(y) p_i(v)] = 0, 0, 1, 1.$$



Walsh-Hadamard变换(WHT)

$$r(x, y, u, v) = \frac{1}{n} (-1)^{\sum_{i=0}^{m-1} [b_i(x) p_i(u) + b_i(y) p_i(v)]}$$

$$u = 0, v = 1, n = 4, m = 2, x, y = 0, 1, 2, 3;$$

$$u = 0, b_i(x) p_i(u) = 0$$

$$b_0(0) = 0, b_0(1) = 1, b_0(2) = 0, b_0(3) = 1;$$

$$b_1(0) = 0, b_1(1) = 0, b_1(2) = 1, b_1(3) = 1;$$

$$p_0(1) = b_1(1) = 0, p_0(2) = 1, p_0(3) = 1$$

$$p_1(1) = b_1(1) + b_0(1) = 1, p_1(2) = 1, p_1(3) = 2$$

$$\sum_{i=0}^{m-1} [b_i(x) p_i(u) + b_i(y) p_i(v)] = 0, 0, 1, 1.$$

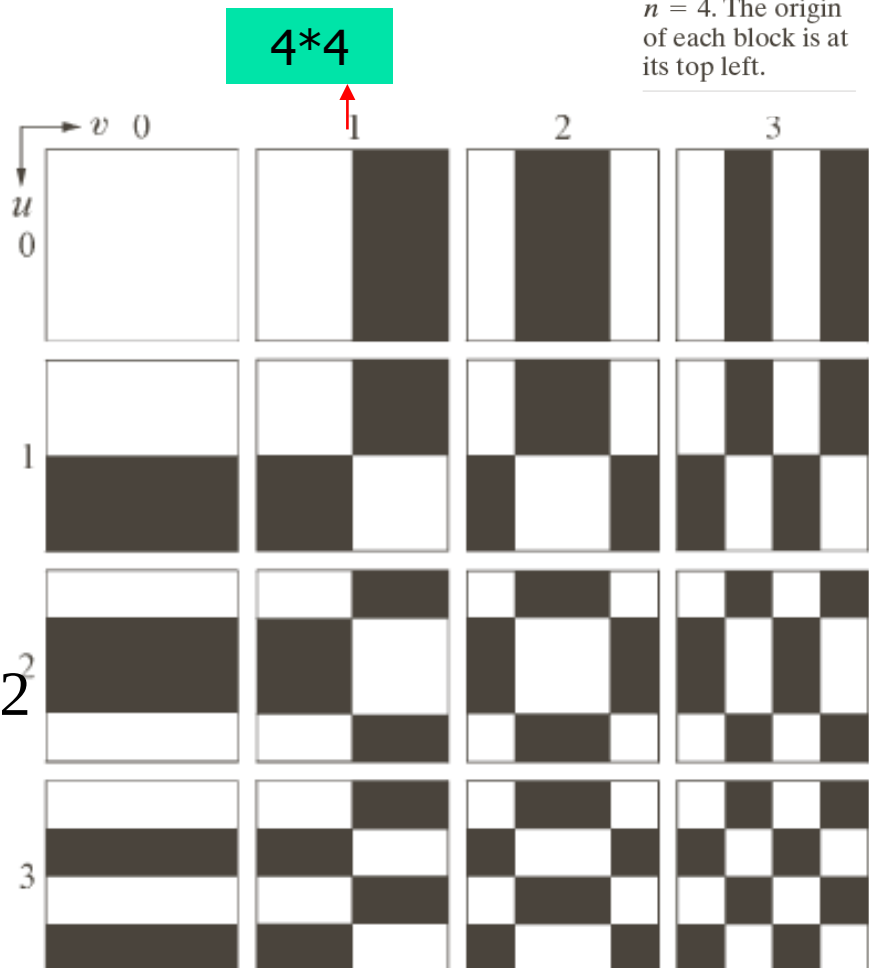


FIGURE 8.22
Walsh-Hadamard
basis functions for
 $n = 4$. The origin
of each block is at
its top left.



Walsh-Hadamard变换(WHT)

8*8

FIGURE 8.22
Walsh-Hadamard
basis functions for
 $n = 4$. The origin
of each block is at
its top left.

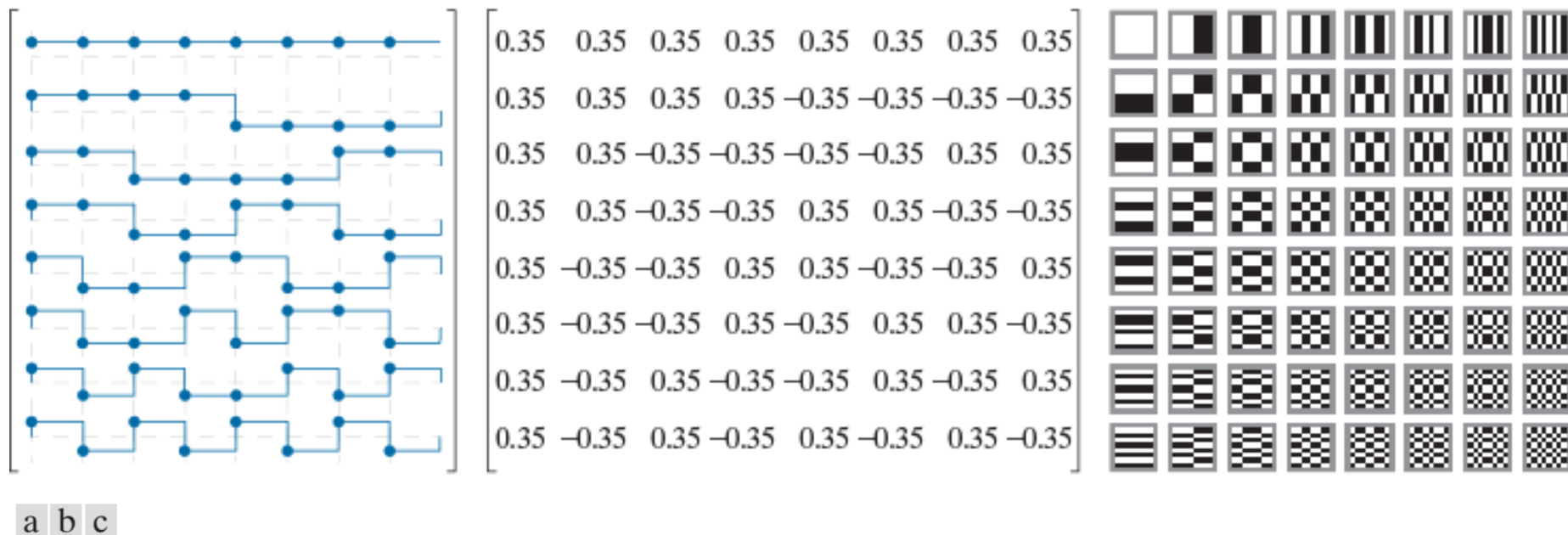


FIGURE 7.16 The transformation matrix and basis images of the sequency-ordered Walsh-Hadamard transform for $N = 8$. (a) Graphical representation of orthogonal transformation matrix $\mathbf{A}_W$, (b) $\mathbf{A}_W$ rounded to two decimal places, and (c) basis images. For 1-D transforms, matrix $\mathbf{A}_W$ is used in conjunction with Eqs. (7-28) and (7-29); for 2-D transforms, it is used with Eqs. (7-35) and (7-36).



块变换编码

离散余弦变换(DCT)

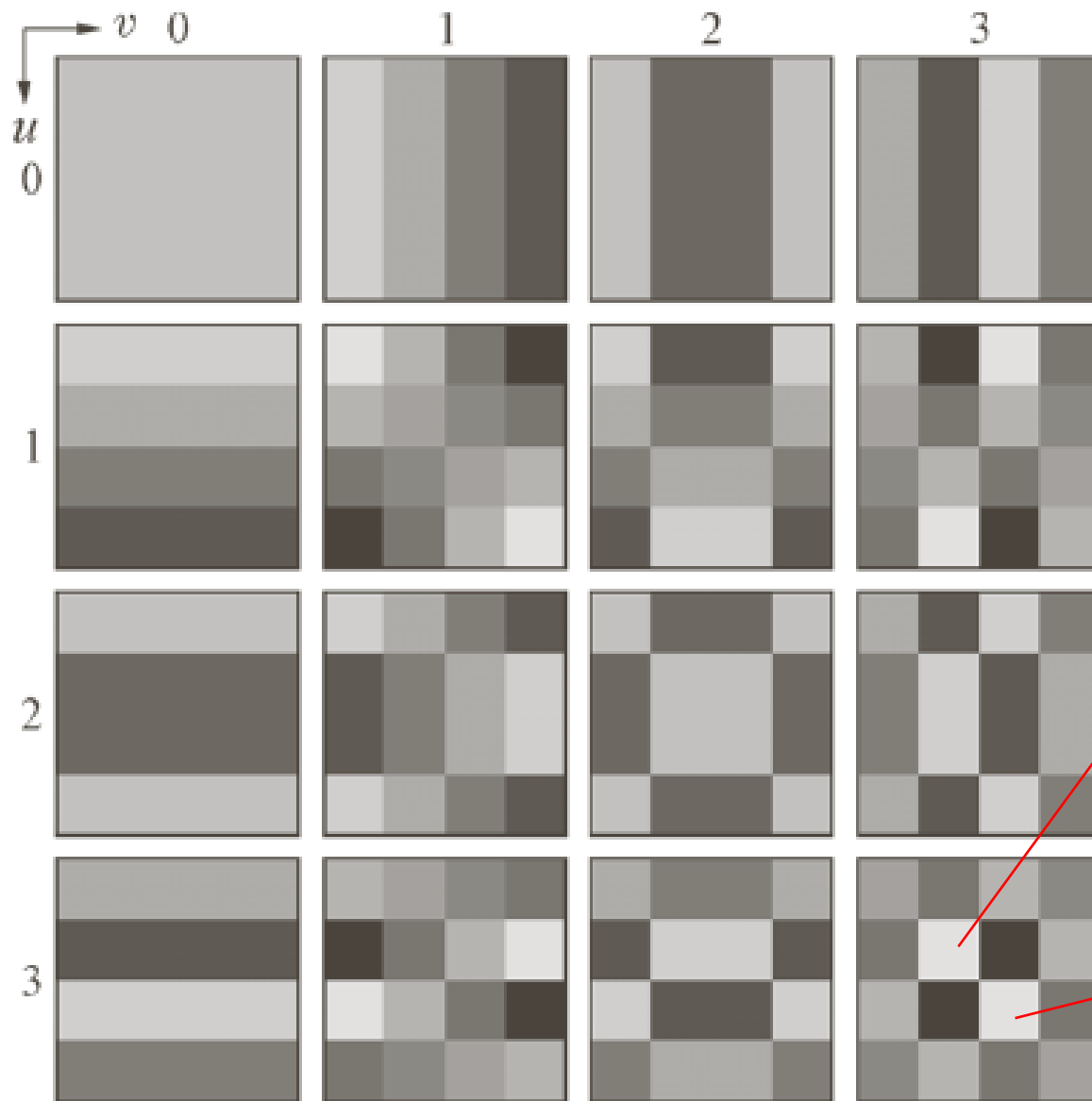
$$\begin{aligned} r(x, y, u, v) &= s(x, y, u, v) \\ &= \alpha(u)\alpha(v) \cos\left[\frac{(2x+1)u\pi}{2n}\right] \cos\left[\frac{(2y+1)v\pi}{2n}\right] \end{aligned}$$

$$\text{where } \alpha(u/v) = \begin{cases} \sqrt{\frac{1}{n}} & \text{for } u/v = 0 \\ \sqrt{\frac{2}{n}} & \text{for } u/v = 1, 2, \dots, n-1 \end{cases}$$



离散余弦变换(DCT)

FIGURE 8.23
Discrete-cosine
basis functions for
 $n = 4$. The origin
of each block is at
its top left.



$$\alpha(u)\alpha(v)\cos\left[\frac{(2x+1)u\pi}{2n}\right]\cos\left[\frac{(2y+1)v\pi}{2n}\right]$$

$u=3, v=3$
 $x=1, y=1$

$$\sqrt{\frac{2}{n}}$$

$u=3, v=3$
 $x=2, y=2$



离散余弦变换(DCT)

$$\alpha(u)\alpha(v)\cos\left[\frac{(2x+1)u\pi}{2n}\right]\cos\left[\frac{(2y+1)v\pi}{2n}\right]$$

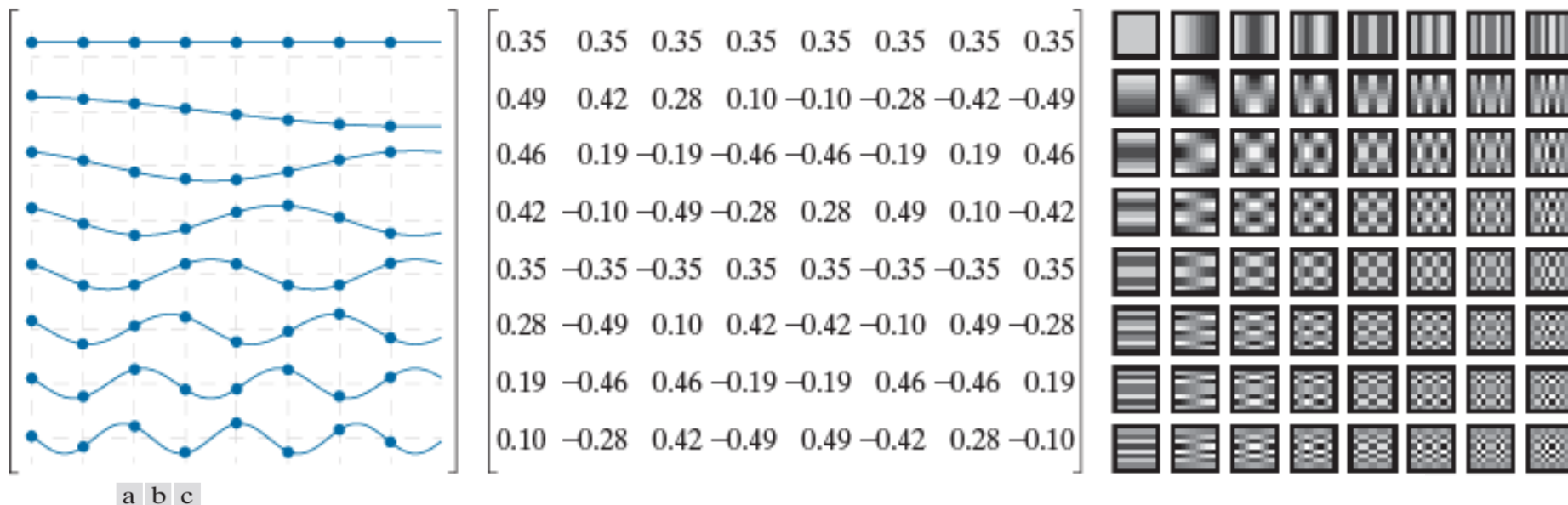
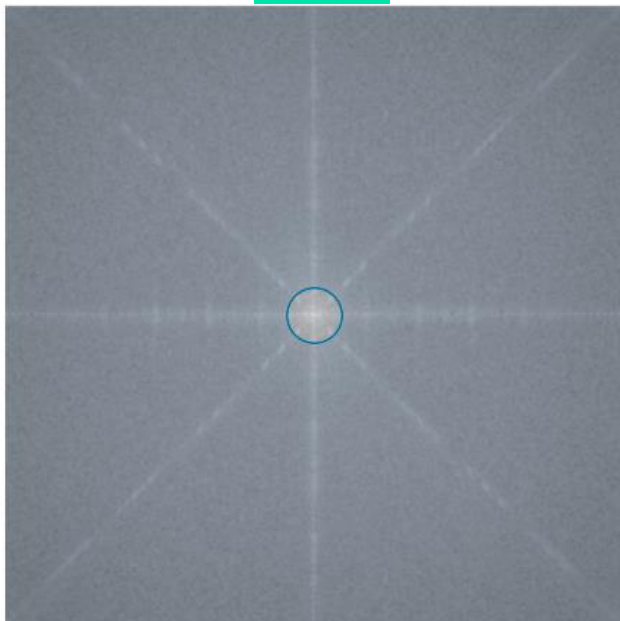
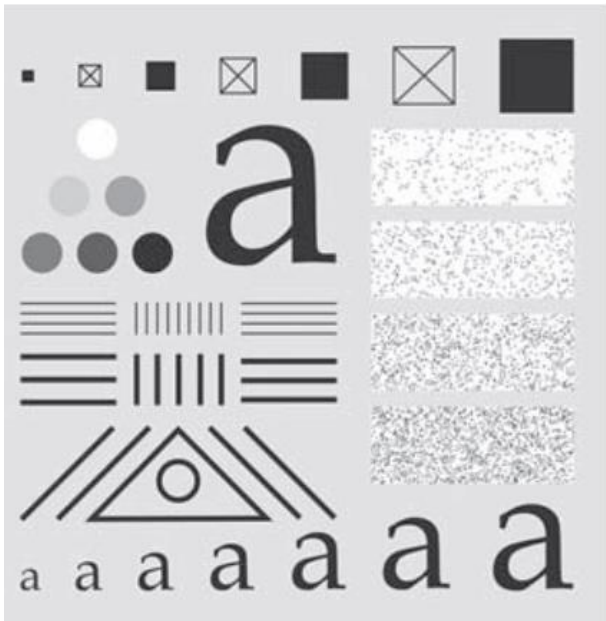


FIGURE 7.10 The transformation matrix and basis images of the discrete cosine transform for $N = 8$. (a) Graphical representation of orthogonal transformation matrix $\mathbf{A}_C$, (b) $\mathbf{A}_C$ rounded to two decimal places, and (c) basis images. For 1-D transforms, matrix $\mathbf{A}_C$ is used in conjunction with Eqs. (7-28) and (7-29); for 2-D transforms, it is used with Eqs. (7-35) and (7-36).



DFT



a	b	c
d	e	f
g	h	i

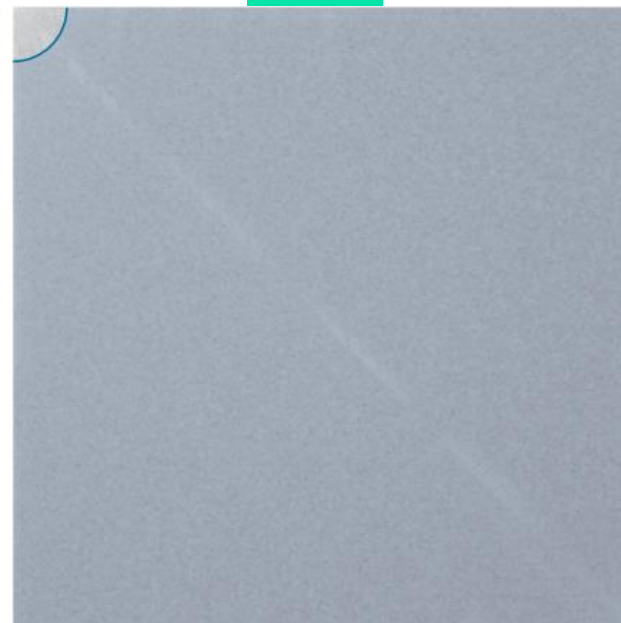
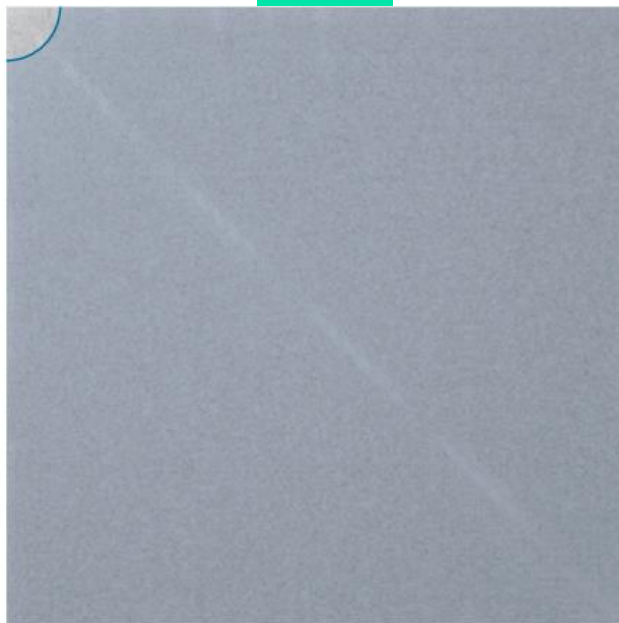
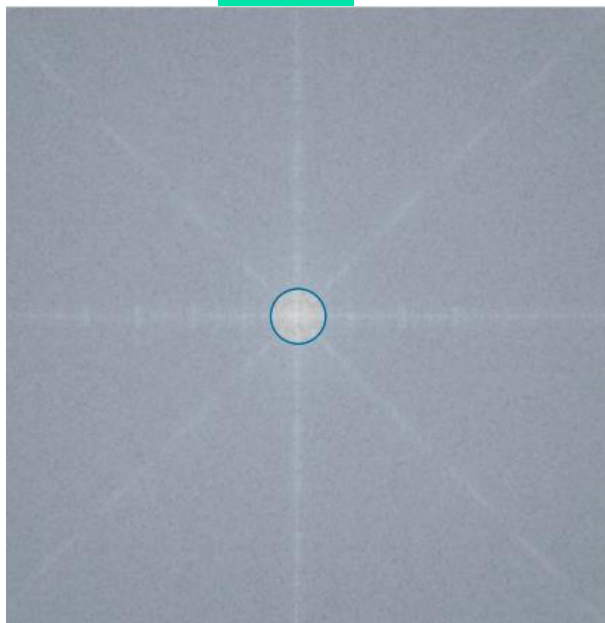
FIGURE 7.15 (a) Original image of the 688×688 test pattern from Fig. 4.41(a). (b) Discrete Fourier transform (DFT) of the test pattern in (a) after padding to size 1376×1376 . The blue overlay is an ideal lowpass filter (ILPF) with a radius of 60. (c) Result of Fourier filtering. (d)–(f) Discrete Hartley transform, discrete cosine transform (DCT), and discrete sine transform (DST) of the test pattern in (a) after padding. The blue overlay is the same ILPF in (b), but appears bigger in (e) and (f) because of the higher frequency resolution of the DCT and DST. (g)–(i) Results of filtering for the Hartley, cosine, and sine transforms, respectively.



DHT

DCT

DST



a	b	c
d	e	f
g	h	i

FIGURE 7.15 (a) Original image of the 688×688 test pattern from Fig. 4.41(a). (b) Discrete Fourier transform (DFT) of the test pattern in (a) after padding to size 1376×1376 . The blue overlay is an ideal lowpass filter (ILPF) with a radius of 60. (c) Result of Fourier filtering. (d)–(f) Discrete Hartley transform, discrete cosine transform (DCT), and discrete sine transform (DST) of the test pattern in (a) after padding. The blue overlay is the same ILPF in (b), but appears bigger in (e) and (f) because of the higher frequency resolution of the DCT and DST. (g)–(i) Results of filtering for the Hartley, cosine, and sine transforms, respectively.

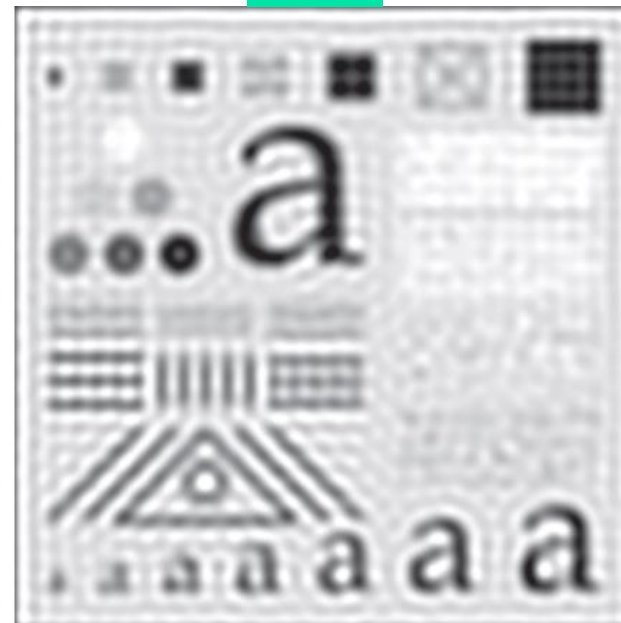


在这个例子中，计算傅里叶相关变换所需的总时间比相应的FFT计算长2到5倍。然而，所有的计算都花了不到一秒钟。

DHT

DCT

DST



a	b	c
d	e	f
g	h	i

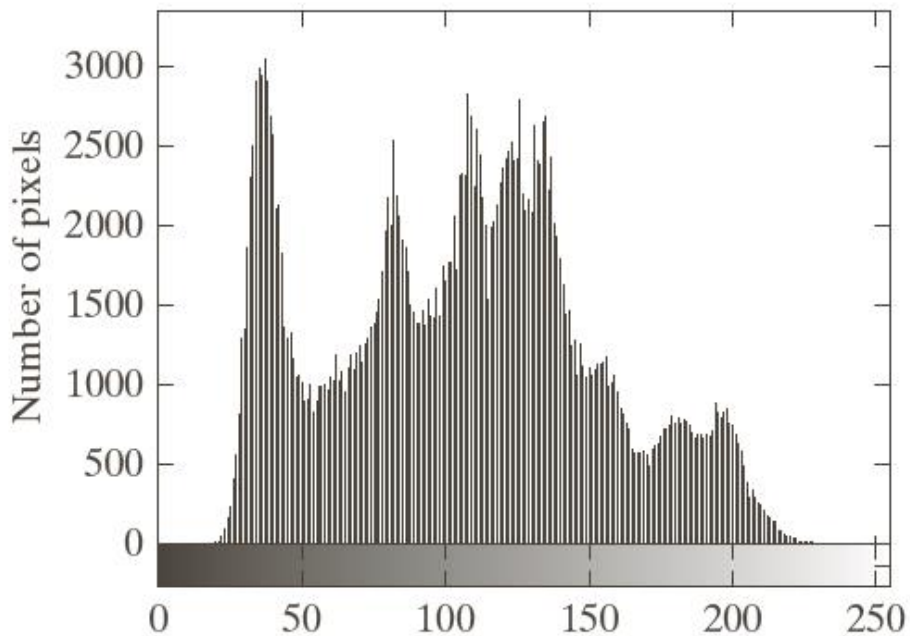
FIGURE 7.15 (a) Original image of the 688×688 test pattern from Fig. 4.41(a). (b) Discrete Fourier transform (DFT) of the test pattern in (a) after padding to size 1376×1376 . The blue overlay is an ideal lowpass filter (ILPF) with a radius of 60. (c) Result of Fourier filtering. (d)–(f) Discrete Hartley transform, discrete cosine transform (DCT), and discrete sine transform (DST) of the test pattern in (a) after padding. The blue overlay is the same ILPF in (b), but appears bigger in (e) and (f) because of the higher frequency resolution of the DCT and DST. (g)–(i) Results of filtering for the Hartley, cosine, and sine transforms, respectively.



图像与视频处理
Image and Video Processing
例子



FIGURE 8.9 (a)
A 512×512 8-bit
image, and (b) its
histogram.





在每一种情况下，50%的结果系数被截断，并对截断的系数阵列进行反变换。

DFT

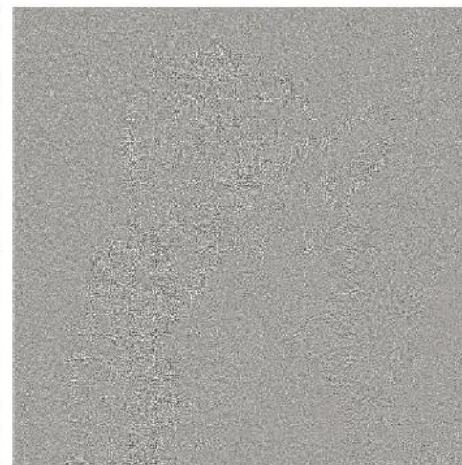
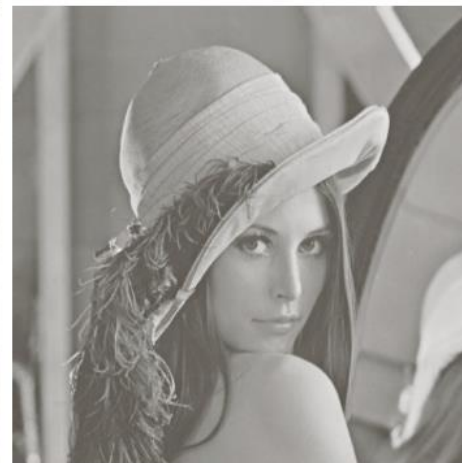


a	b	c
d	e	f

RMSE = 2.32



RMSE = 1.78



RMSE = 1.13

DCT

FIGURE 8.24 Approximations of Fig. 8.9(a) using the (a) Fourier, (b) Walsh-Hadamard, and (c) cosine transforms, together with the corresponding scaled error images in (d)–(f).



Given $T(u, v)$, $g(x, y)$ similarly can be obtained using the

generalized inverse discrete transform

给定 $T(u, v)$, $g(x, y)$ 同样可以用广义逆离散变换得到

$$g(x, y) = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(x, y, u, v)$$

for $x, y = 0, 1, 2, \dots, n-1$.

$$G = \begin{bmatrix} \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, 0, u, v) & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, 1, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, n-1, u, v) \\ \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(1, 0, u, v) & & \cdots & \\ & & \cdots & \\ \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, 0, u, v) & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, 1, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, n-1, u, v) \end{bmatrix}$$

$$\chi(u, v) = \begin{cases} 0 & \text{satisfy a specified truncation criterion} \\ 1 & \text{otherwise} \end{cases}$$

Base images



Given $T(u, v)$, $g(x, y)$ similarly can be obtained using the generalized inverse discrete transform 给定 $T(u, v)$, $g(x, y)$ 同样可以用广义逆离散变换得到

$$g(x, y) = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(x, y, u, v)$$

for $x, y = 0, 1, 2, \dots, n-1$.

$$G = \begin{bmatrix} g(0,0) \\ \vdots \\ g(0,n-1) \\ \vdots \\ g(n-1,0) \\ g(n-1,n-1) \end{bmatrix} = \begin{bmatrix} \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, 0, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(0, n-1, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, 0, u, v) & \cdots & \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(n-1, n-1, u, v) \\ \vdots & & \vdots & & \vdots & & \vdots \end{bmatrix} = \begin{bmatrix} \bar{T}^T \bar{S}(0,0) \\ \vdots \\ \bar{T}^T \bar{S}(0,n-1) \\ \vdots \\ \bar{T}^T \bar{S}(n-1,0) \\ \bar{T}^T \bar{S}(n-1,n-1) \end{bmatrix} = \bar{T}^T \begin{bmatrix} \bar{S}(0,0) \\ \vdots \\ \bar{S}(0,n-1) \\ \vdots \\ \bar{S}(n-1,0) \\ \bar{S}(n-1,n-1) \end{bmatrix} = \bar{T}^T \bar{S}$$

$$\chi(u, v) = \begin{cases} 0 & \text{satisfy a specified truncation criterion} \\ 1 & \text{otherwise} \end{cases}$$

Base images



$$\hat{T} = \begin{bmatrix} T(0,0)\chi(0,0) & & \\ & \ddots & \\ T(0,n-1)\chi(0,n-1) & & \\ & \ddots & \\ T(n-1,0)\chi(n-1,0) & & \\ T(n-1,n-1)\chi(n-1,n-1) \end{bmatrix} = \begin{bmatrix} \chi(0,0) & & \\ & \chi(0,n-1) & \\ & & \ddots \\ & & & \chi(n-1,n-1) \end{bmatrix} \vec{T} = \chi \vec{T}$$

$$\chi(u,v) = \begin{cases} 0 & \text{satisfy a specified truncation criterion} \\ 1 & \text{otherwise} \end{cases}$$

Base images



MSE

$$e_{ms} = E\{\|G - \hat{G}\|^2\}$$

$$= E\{\|\vec{T}^T \vec{S} - (\chi \vec{T})^T \vec{S}\|^2\}$$

$$= E\{\|\vec{T}^T (I - \chi) \vec{S}\|^2\}$$

$$= E\{\vec{T}^T (I - \chi) \vec{S} \vec{S}^T (I - \chi) \vec{T}\}$$

$$= E\{\vec{T}^T (I - \chi) \vec{T}\}$$

$$= \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} \sigma_{T(u,v)}^2 [1 - \chi(u, v)]$$

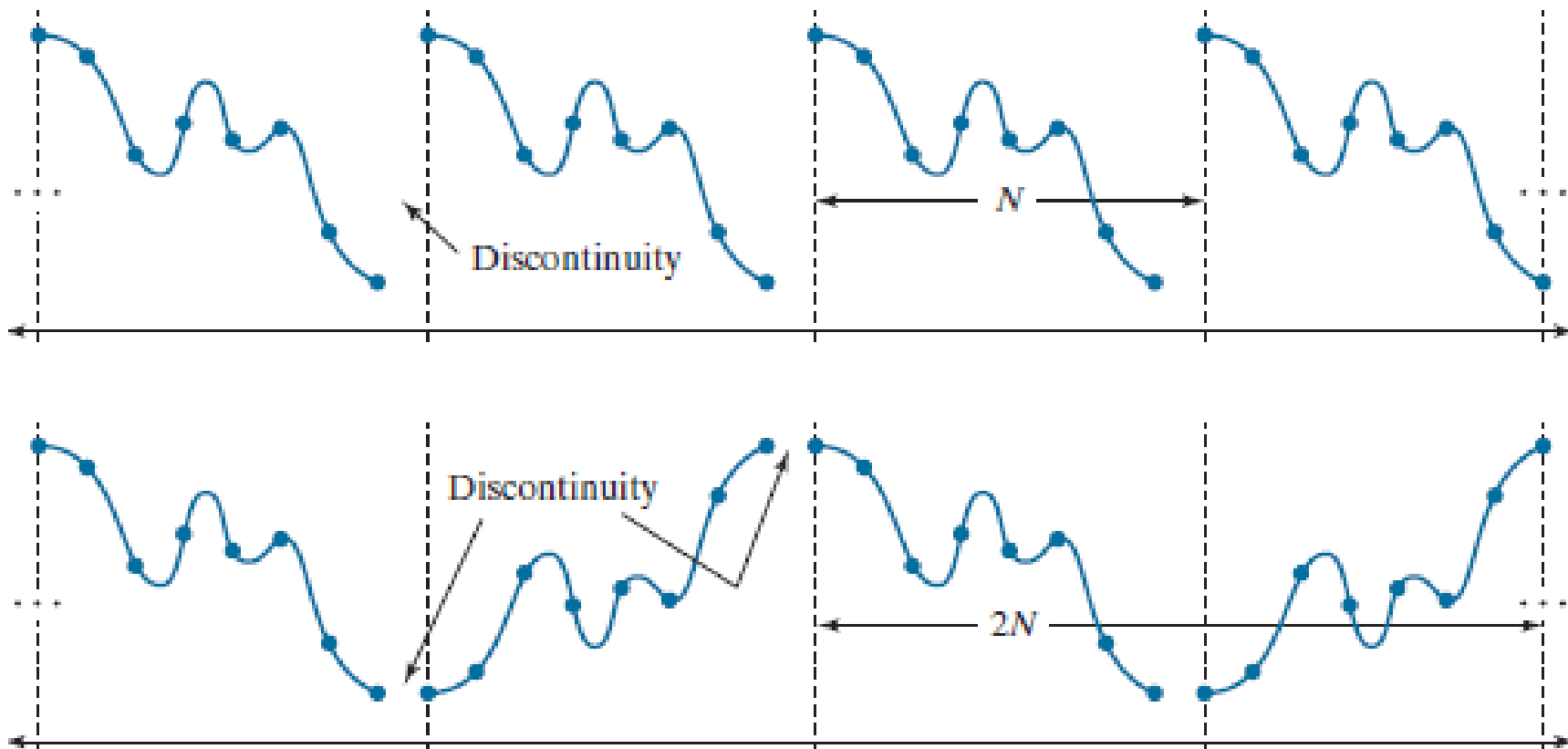
$$[1 - \chi(u, v)] = [1 - \chi(u, v)]^2$$



DFT&DCT

a
b

FIGURE 8.25 The periodicity implicit in the 1-D (a) DFT and (b) DCT.





子图像大小选择

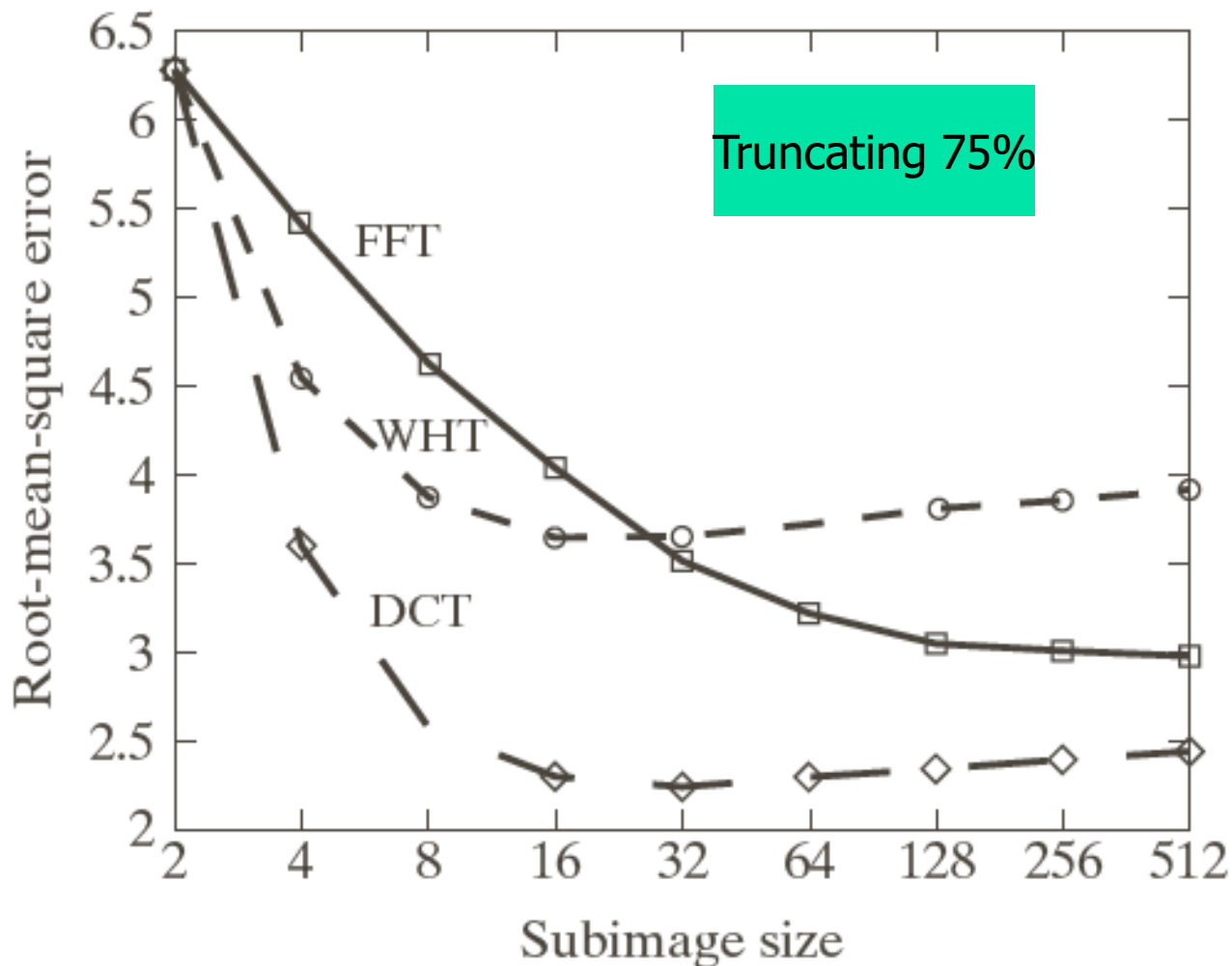


FIGURE 8.26
Reconstruction
error versus
subimage size.



子图像大小选择

25%DCT of 2*2

25%DCT of 4*4

25%DCT of 8*8



a b c d

FIGURE 8.27 Approximations of Fig. 8.27(a) using 25% of the DCT coefficients and (b) 2×2 subimages, (c) 4×4 subimages, and (d) 8×8 subimages. The original image in (a) is a zoomed section of Fig. 8.9(a).



位分配

对转换后的子图像的系数进行截断、量化和编码的整个过程通常称为**位分配**

区域编码

根据最大方差选择保留系数

-基于信息论的概念，将信息视为不确定性。

方差最大则携带最多的图像信息

阈值编码

根据最大幅度选择保留系数

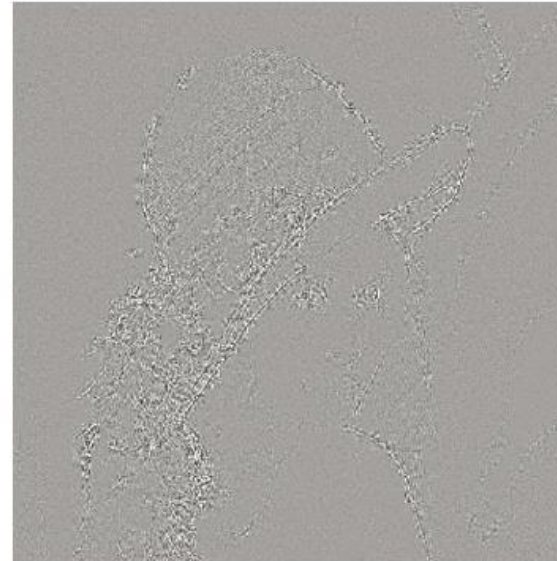
基于最大幅值对重构子图像质量的贡献最为显著



RMSE = 4.5

8 largest
Transformation
coefficients

Threshold

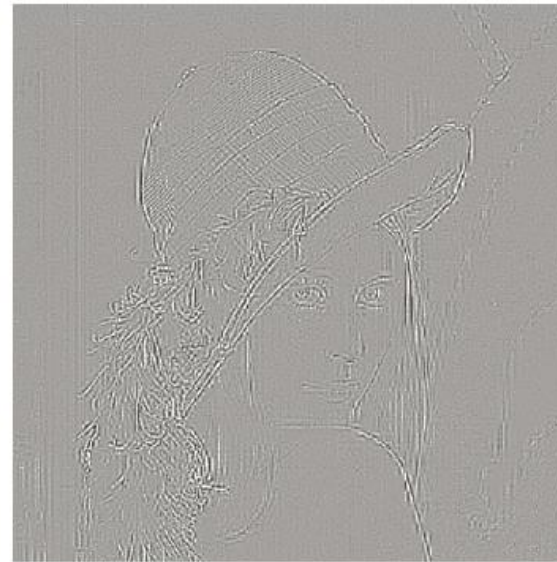
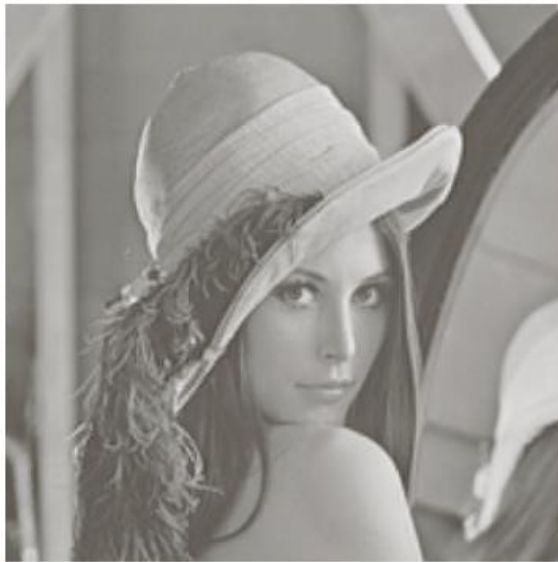


a	b
c	d

FIGURE 8.28
Approximations
of Fig. 8.9(a) using
12.5% of the
 8×8 DCT
coefficients:
(a) — (b) threshold
coding results;
(c) — (d) zonal
coding results. The
difference images
are scaled by 4.

8 largest
variance

Zonal



RMSE = 6.5

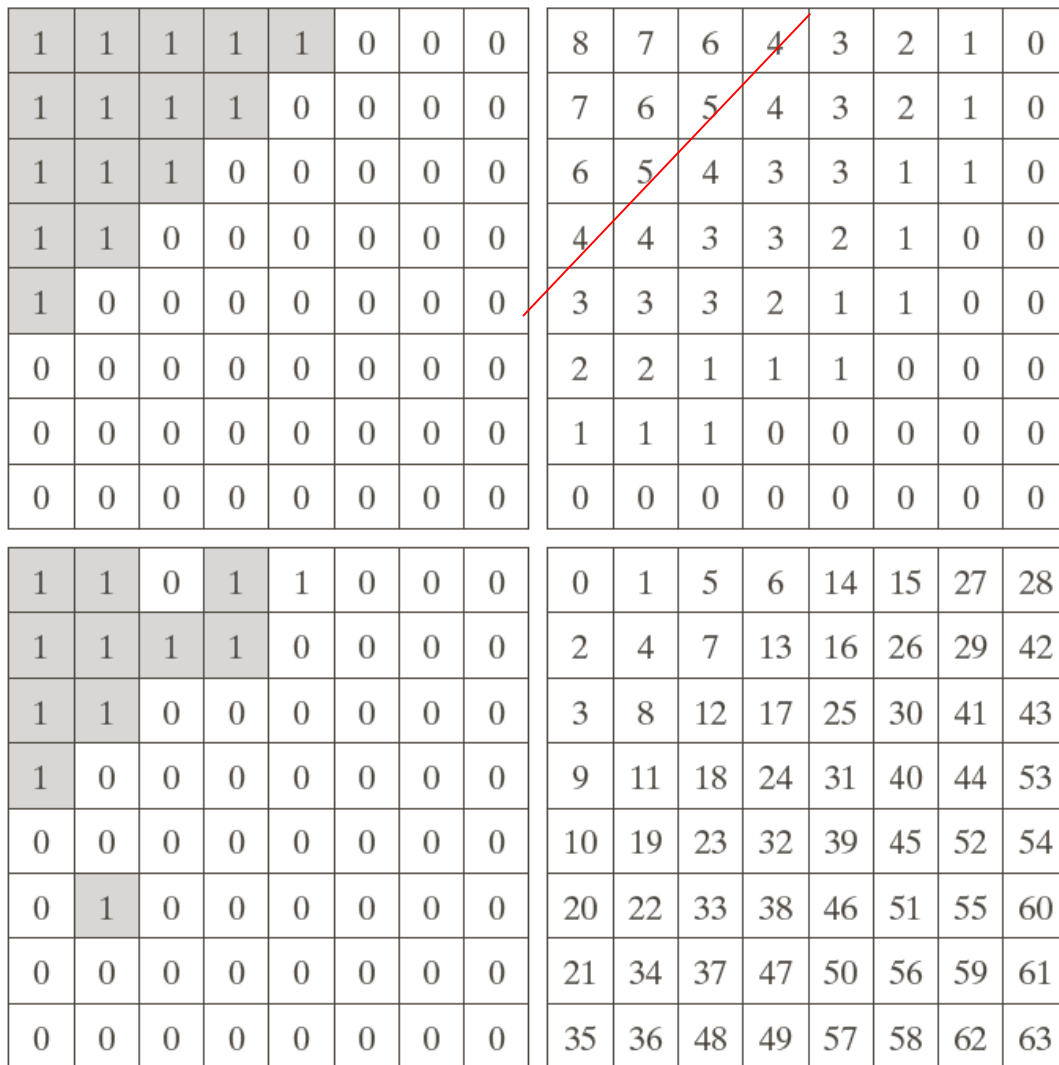


$$\log_2 \sigma_{T(u,v)}^2 \text{ bits}$$

a b
c d

FIGURE 8.29

A typical
(a) zonal mask,
(b) zonal bit
allocation,
(c) threshold
mask, and
(d) thresholded
coefficient
ordering
sequence. Shading
highlights the
coefficients that
are retained.



Zigzag order

最大方差DC系数:
瑞利密度
其他: 拉普拉斯
或者高斯密度

1.全局阈值
2.局部阈值
3.每个系数的
位置函数



阈值编码

$$\hat{T}(u, v) = \text{round} \left[\frac{T(u, v)}{Z(u, v)} \right]$$

$$Z = \begin{bmatrix} Z(0,0) & Z(0,1) & \dots & Z(0,n-1) \\ Z(1,0) & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ Z(n-1,0) & \dots & \dots & Z(n-1,n-1) \end{bmatrix}$$

Inverse :

$$\dot{T}(u, v) = \hat{T}(u, v)Z(u, v)$$

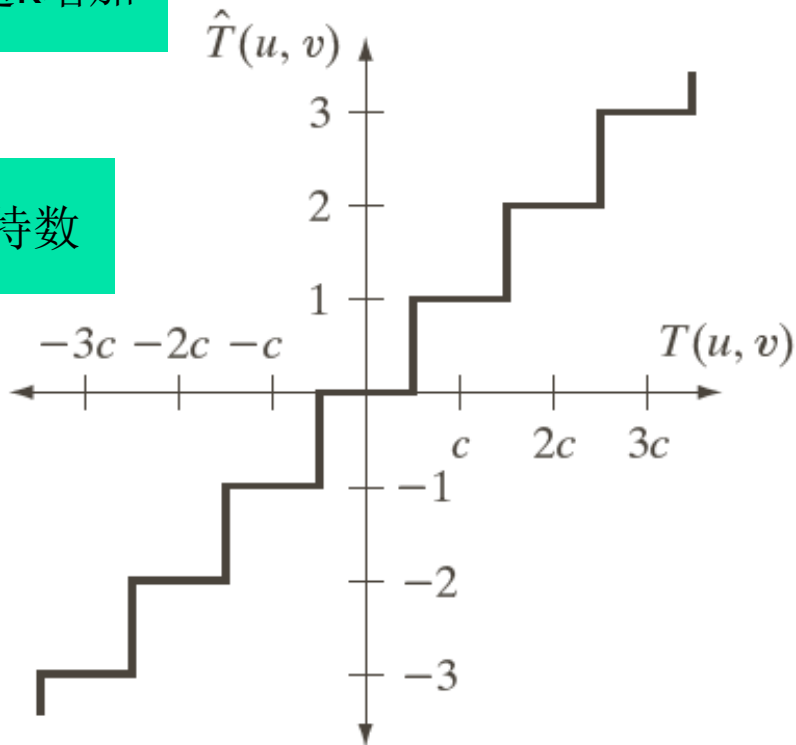


阈值编码

$$kc - \frac{c}{2} \leq T(u, v) < kc + \frac{c}{2}$$

VLC长度随k增加

C值控制比特数



a b

FIGURE 8.30

(a) A threshold coding quantization curve [see Eq. (8.2-29)]. (b) A typical normalization matrix.

Z

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

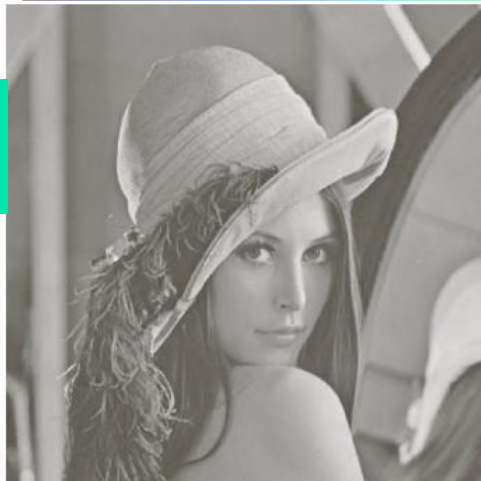


阈值编码

8*8 DCT

19:1, 2Z

12:1,
Z



30:1, 4Z



182:1
32Z

85:1, 16Z

FIGURE 8.31 Approximations of Fig. 8.9(a) using the DCT and normalization array of Fig. 8.30(b): (a) Z, (b) 2Z, (c) 4Z, (d) 8Z, (e) 16Z, and (f) 32Z.

8*8 DCT



关于JPEG压缩

- JPEG 代表 联合图像专家组
- 用于 24 位颜色文件。
- 适用于摄影图像。
- 尽管它是一种有损压缩技术，但它可以产生具有高压缩率的高质量图像。

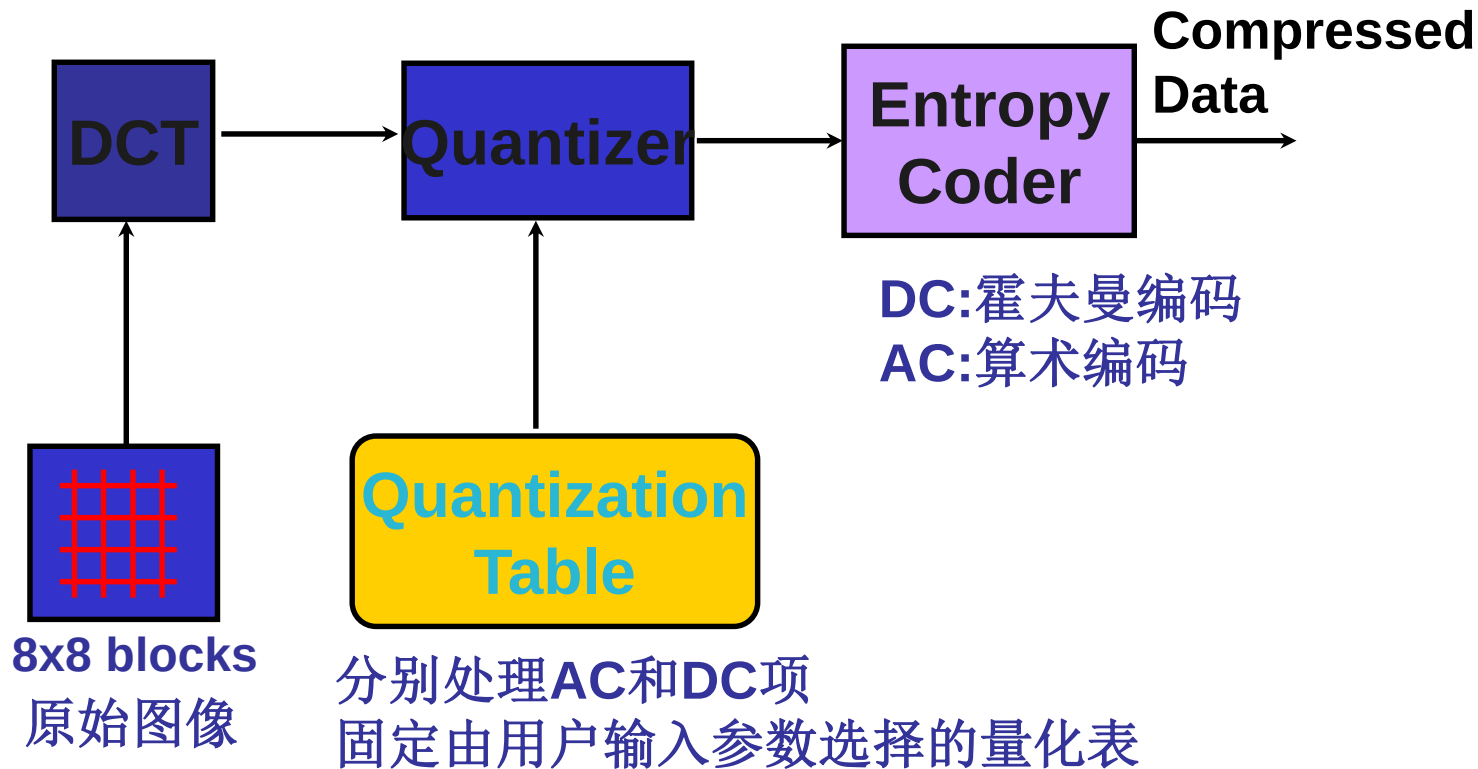


关于JPEG压缩

- 它定义了三种不同的编码系统：
 - 1. 有损基线编码系统，适用于大多数压缩应用
 - 2. 扩展编码系统，用于更大的压缩、更高的精度或渐进式重建应用
 - 3. 用于可逆压缩的无损独立编码系统



例子: JPEG





JPEG 压缩的步骤

1. (可选) 如果颜色以 RGB 模式表示, 请将其转换为 YUV。
2. 将文件分成 8×8 块。
3. 使用离散余弦变换将像素信息从空间域转换为频域。
4. 通过将每个系数除以一个整数值并四舍五入到最接近的整数来量化结果值。
5. 按锯齿形顺序查看结果系数。对以这种方式排序的系数进行运行长度编码。接下来是霍夫曼编码。



Step 1a: 将 RGB 转换为 YUV

- YUV 颜色模式根据颜色的亮度（亮度）和色度（色调）存储颜色。
- 人眼对色度的敏感度不如亮度敏感。
- JPEG 压缩不需要 YUV，但它提供了更好的压缩率。



RGB vs. YUV



- 将 RGB 转换为 YUV 是简单的算术。该公式基于红色、绿色和蓝色对亮度和色度因子的相对贡献。
- 根据目标监视器的不同，有几种不同的公式在使用。

例如:

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

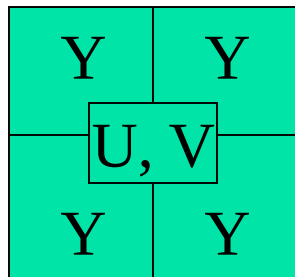
$$U = -0.1687 * R - 0.3313 * G + 0.5 * B + 128$$

$$V = 0.5 * R - 0.4187 * G - 0.813 * B + 128$$



Step 1b: 下采样

- 色度信息可以（可选）下采样。
- 表示法 4: 2: 0 表示对于每个 4 个像素块，有 4 个亮度信息（Y）样本，以及两个色度分量（U 和 V）各 1 个样本。





Step 2: 分为 8×8 块

- 请注意，使用 YUV 颜色时，Y 分量的每个块中有 64 个像素的信息（尽管 U 和 V 分量每个方向只有 16 个像素）。
- 如果图像没有均匀地分成 8×8 块，则会在末尾添加额外的像素并在压缩后丢弃。
- 通过减去 128 来“左”移。



离散余弦变换

- DCT 将数据从空间域转换为频域。
- 空间域显示其在空间中移动时颜色的幅度
- 频域显示图像文件中颜色的振幅从一个像素到下一个像素变化的速度。



Step 3: DCT



- 频域是数据的更好表示，因为它可以分离并丢弃对人类感知不是很重要的信息。
- 人眼对高频变化不是很敏感，尤其是在摄影图像中，因此高频数据在某种程度上可以被丢弃。



Step 3: DCT



- 颜色振幅信息可以被认为是一个波（二维）。
- 将波分解为其分量频率。
- 对于颜色数据的 8×8 矩阵，将获得频率分量的 8×8 矩阵的系数。



Step 4: 量化系数由 DCT 计算

- DCT 是无损的，因为反向 DCT 会准确地返回初始信息（忽略因使用浮点数而导致的舍入误差）。
- 来自 DCT 的值最初是浮点数。
- 它们通过量化更改为整数。



Step 4: 量化

- 量化涉及将每个系数除以 1 到 255 之间的整数并四舍五入。
- 选择量化表是为了将每个系数的精度降低到不超过必要的精度。
- 量化表随压缩文件一起携带。



Step 5: 按 “zigzag” 顺序排列

- 这样做是为了使系数按频率增加的顺序排列。
- 量化后频率系数越高，越有可能为 0。
- 这改进了运行长度编码的压缩。
- 进行运行长度编码和霍夫曼编码。



EXAMPLE 8.17:
JPEG baseline
coding and
decoding.



Image data

52	55	61	66	70	61	64	73
63	59	66	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	63	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94



EXAMPLE 8.17: JPEG baseline coding and decoding.



-128

52-128

-76	-73	-67	-62	-58	-67	-64	-55
-65	-69	-62	-38	-19	-43	-59	-56
-66	-69	-60	-15	16	-24	-62	-55
-65	-70	-57	-6	26	-22	-58	-59
-61	-67	-60	-24	-2	-40	-60	-58
-49	-63	-68	-58	-51	-65	-70	-53
-43	-57	-64	-69	-73	-67	-63	-45
-41	-49	-59	-60	-63	-52	-50	-34



EXAMPLE 8.17:
JPEG baseline
coding and
decoding.



DCT

−415	−29	−62	25	55	−20	−1	3
7	−21	−62	9	11	−7	−6	6
−46	8	77	−25	−30	10	7	−5
−50	13	35	−15	−9	6	0	3
11	−8	−13	−2	−1	1	−4	1
−10	1	3	−3	−1	0	2	−1
−4	−1	2	−1	2	−3	1	−2
−1	−1	−1	−2	−1	−1	0	−1



EXAMPLE 8.17: JPEG baseline coding and decoding.

量化

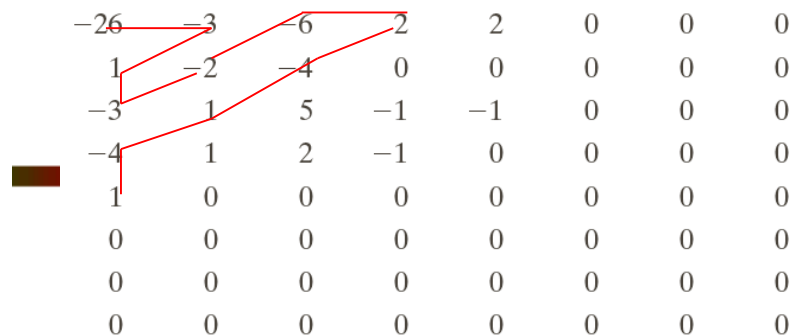
$\text{ROUND}(415/16)$

-26	-3	-6	2	2	0	0	0
1	-2	-4	0	0	0	0	0
-3	1	5	-1	-1	0	0	0
-4	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99



EXAMPLE 8.17: JPEG baseline coding and decoding.



$[-26 \ -3 \ 1 \ -3 \ -2 \ -6 \ 2 \ -4 \ 1 \ -4 \ 1 \ 1 \ 5 \ 0 \ 2 \ 0 \ 0 \ -1 \ 2 \ 0 \ 0 \ 0 \ 0 \ 0 \ -1 \ -1 \ \text{EOB}]$

假设紧靠其左侧的变换和量化子图像的 DC 系数为 **-17**。
DPCM 差异的结果是 $[-26 - (-17)] = -9$ 。

Table A.3: Category 4

Cate 4: 101, length: 7

$7-3=4$, generated from
the LSBs of the difference value

$-9=0111-1=0110$

1010110

K LSBs of positive difference
K LSBs of negative difference-1

1010110 0100 001 0100 0101 100001 0110 100011 001 100011 001
001 100101 11100110 110110 0110 11110100 000 1010

-3, category: 2, first 2bits 01, last 2 bits: 1101-1=1100

1, category: 1, first 2bits 00, last 1 bits: 0001=1

$C=512/92$
 $=5.6$



$[-26 \ -3 \ 1 \ -3 \ -2 \ -6 \ 2 \ -4 \ 1 \ -4 \ 1 \ 1 \ 5 \ 0 \ 2 \ 0 \ 0 \ -1 \ 2 \ 0 \ 0 \ 0 \ 0 \ 0 \ -1 \ -1 \ \text{EOB}]$

-26	-3	1	-3	-2	-6	2	-4	1	-4	1
-----	----	---	----	----	----	---	----	---	----	---

1010110 0100 001 0100 0101 100001 0110 100011 001 100011 001
001 100101 11100110 110110 0110 11110100 000 1010

1	5	02	00-1	2	00000-1	-1	EOB
---	---	----	------	---	---------	----	-----



EXAMPLE 8.17: JPEG baseline coding and decoding.

Huffman decoder



-26	-3	-6	2	2	0	0	0
1	-2	-4	0	0	0	0	0
-3	1	5	-1	-1	0	0	0
-4	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0



EXAMPLE 8.17: JPEG baseline coding and decoding.

Dequantize

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

-416	-33	-60	32	48	0	0	0
12	-24	-56	0	0	0	0	0
-42	13	80	-24	-40	0	0	0
-56	17	44	-29	0	0	0	0
18	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0



EXAMPLE 8.17: JPEG baseline coding and decoding.



IDCT

−70	−64	−61	−64	−69	−66	−58	−50
−72	−73	−61	−39	−30	−40	−54	−59
−68	−78	−58	−9	13	−12	−48	−64
−59	−77	−57	0	22	−13	−51	−60
−54	−75	−64	−23	−13	−44	−63	−56
−52	−71	−72	−54	−54	−71	−71	−54
−45	−59	−70	−68	−67	−67	−61	−50
−35	−47	−61	−66	−60	−48	−44	−44



EXAMPLE 8.17: JPEG baseline coding and decoding.

	58	64	67	64	59	62	70	78
+128	56	55	67	89	98	88	74	69
	60	50	70	119	141	116	80	64
	69	51	71	128	149	115	77	68
	74	53	64	105	115	84	65	72
	76	57	56	74	75	57	57	74
	83	69	59	60	61	61	67	78
	93	81	67	62	69	80	84	84
	52	55	61	66	70	61	64	73
	63	59	66	90	109	85	69	72
	62	59	68	113	144	104	66	73
	63	58	71	122	154	106	70	69
	67	61	68	104	126	88	68	70
	79	65	60	70	77	63	58	75
	85	71	64	59	55	61	65	83
	87	79	69	68	65	76	78	94



EXAMPLE 8.17: JPEG baseline coding and decoding.

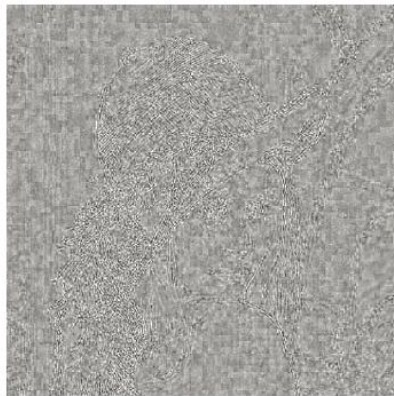


差异

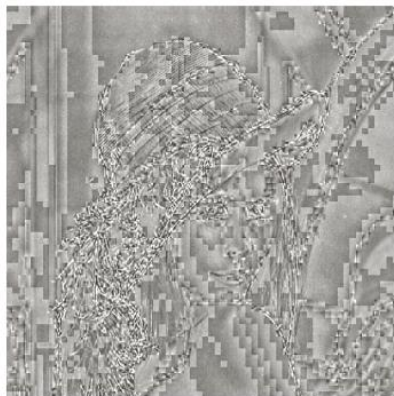
-6	-9	-6	2	11	-1	-6	-5
7	4	-1	1	11	-3	-5	3
2	9	-2	-6	-3	-12	-14	9
-6	7	0	-4	-5	-9	-7	1
-7	8	4	-1	6	4	3	-2
3	8	4	-4	2	6	1	1
2	2	5	-1	-6	0	-2	5
-6	-2	2	6	-4	-4	-6	10



25:1



52:1



a	b	c
d	e	f

FIGURE 8.32 Two JPEG approximations of Fig. 8.9(a). Each row contains a result after compression and reconstruction, the scaled difference between the result and the original image, and a zoomed portion of the reconstructed image.



JPEG at 0.125 bpp (扩大)





JPEG2000 at 0.125 bpp





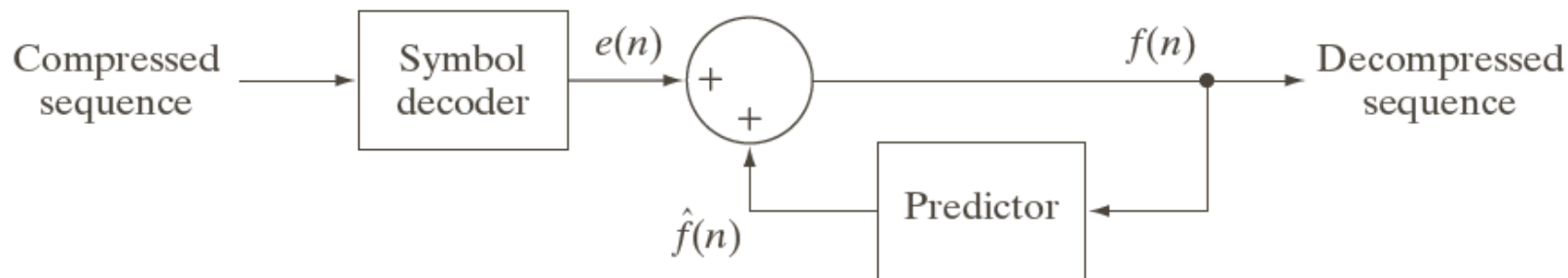
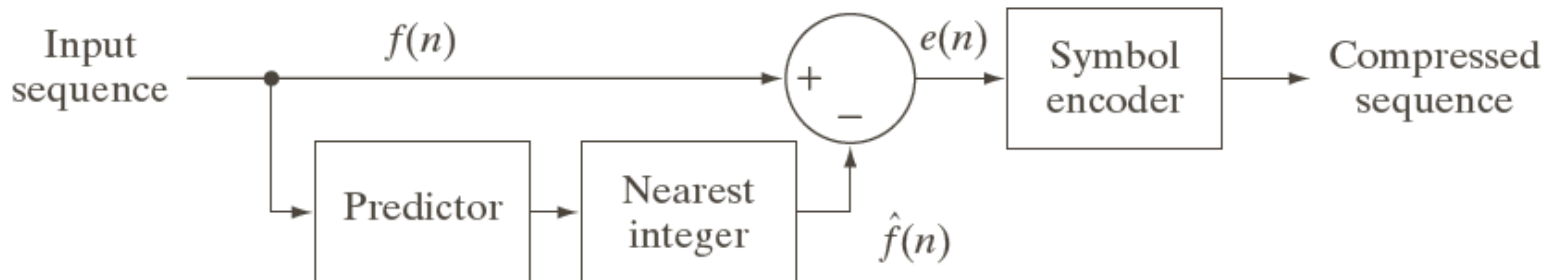
无损预测编码

a
b

FIGURE 8.33

A lossless
predictive coding
model:

(a) encoder;
(b) decoder.



$$\hat{f}(n) = \text{round}\left[\sum_{i=1}^m \alpha f(n-i)\right]$$

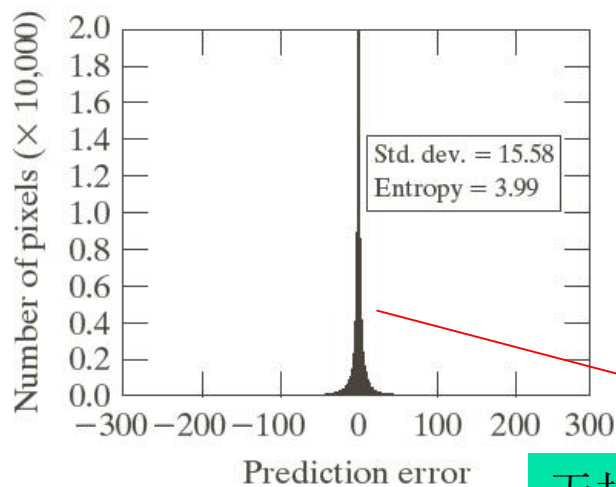
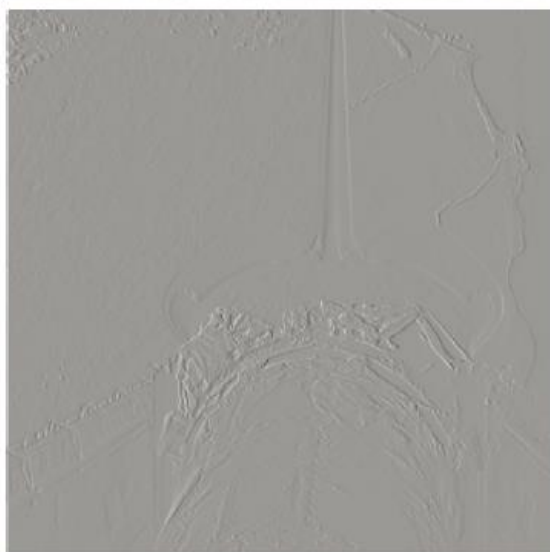
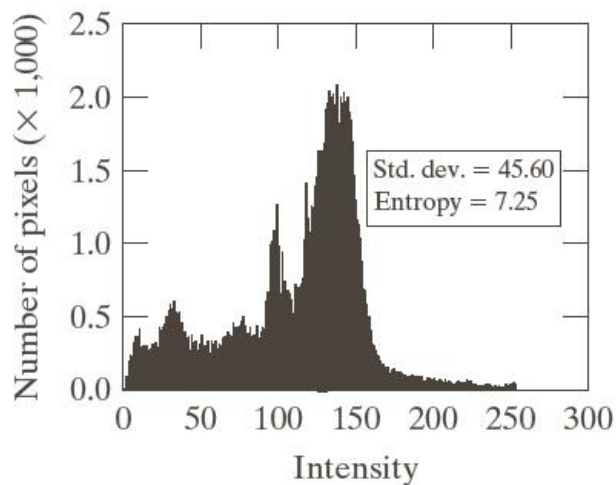
$$\hat{f}(x, y) = \text{round}\left[\sum_{i=1}^m \alpha f(x, y-i)\right]$$

$$\hat{f}(x, y) = \text{round}\left[\sum_{i=1}^m \alpha f(x, y, t-i)\right]$$



a	b
c	d

图8.34



(a) 从轨道航天飞机上看地球。(b) (a) 的强度直方图。(c) 由等式产生的预测误差图像。(8.2-34)。(d) 预测误差的直方图。(原图

$$\hat{f}(x, y) = \text{round}[\alpha f(x, y-1)]$$

$$p_e(e) = \frac{1}{\sqrt{2}\sigma_e} e^{\frac{-\sqrt{2}|e|}{\sigma_e}}$$

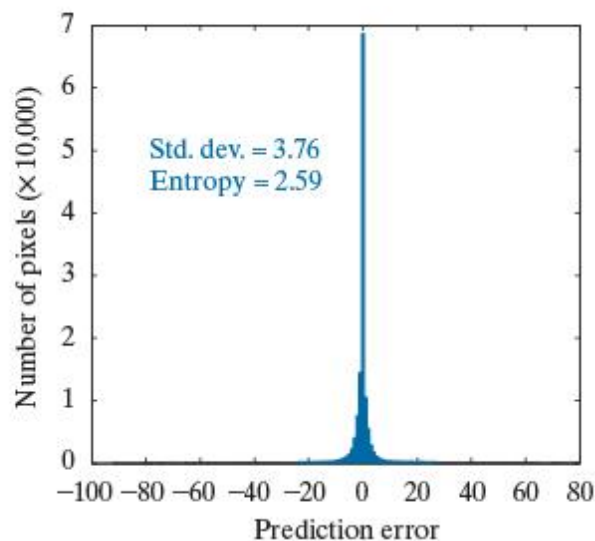
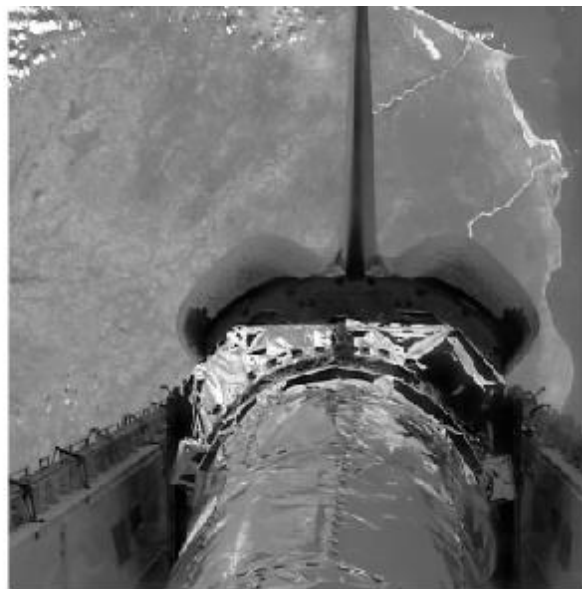
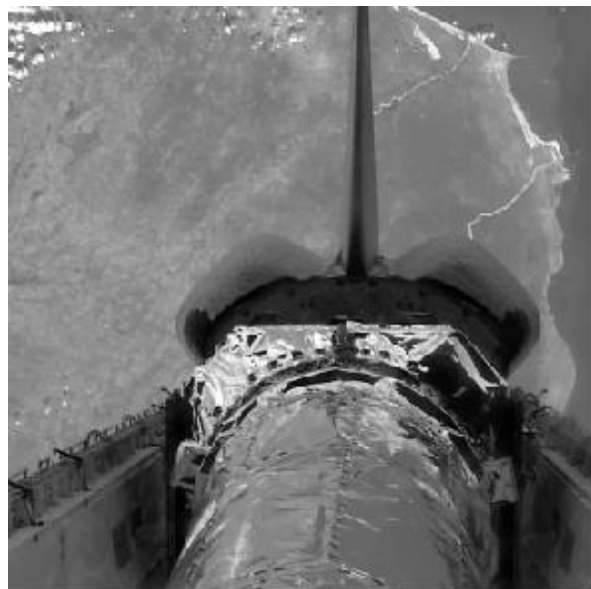
无损预测编码错误：
零平均不相关的拉普拉斯



a	b
c	d

图8.35

(a) 和 (b) 从一个轨道航天飞机视频的两个地球视图。(c) 由等式产生的预测误差图像。(8.2-36). (d) 预测误差的直方图。

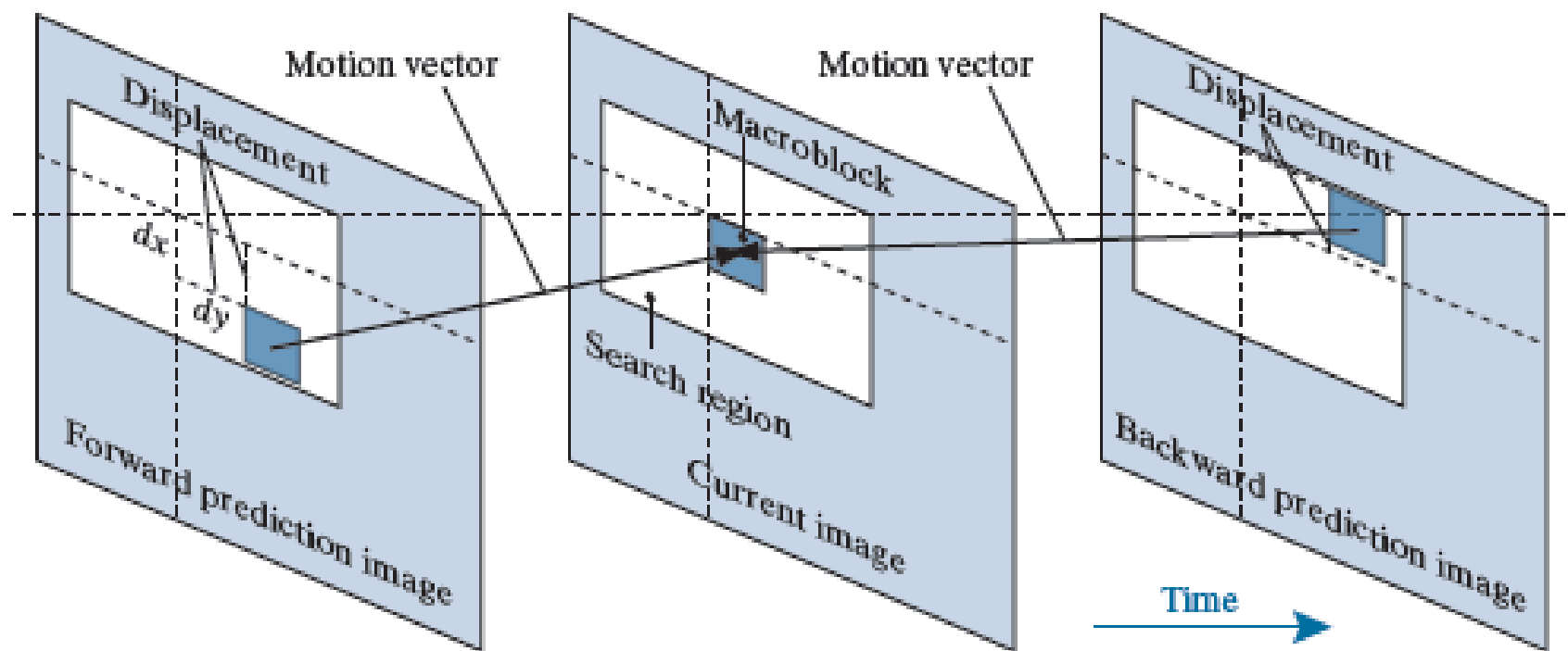


$$\hat{f}(x, y, t) = \text{round}[\alpha f(x, y, t-1)]$$



运动补偿的预测残差

图8.36宏块运动规范。





视频处理



块匹配算法

·概述:

- 假设一个块中的所有像素都经历了平移, 由单个MV表示
- 通过最小化该块上的DFD误差, 独立地估计每个块的MV

·最小化函数:

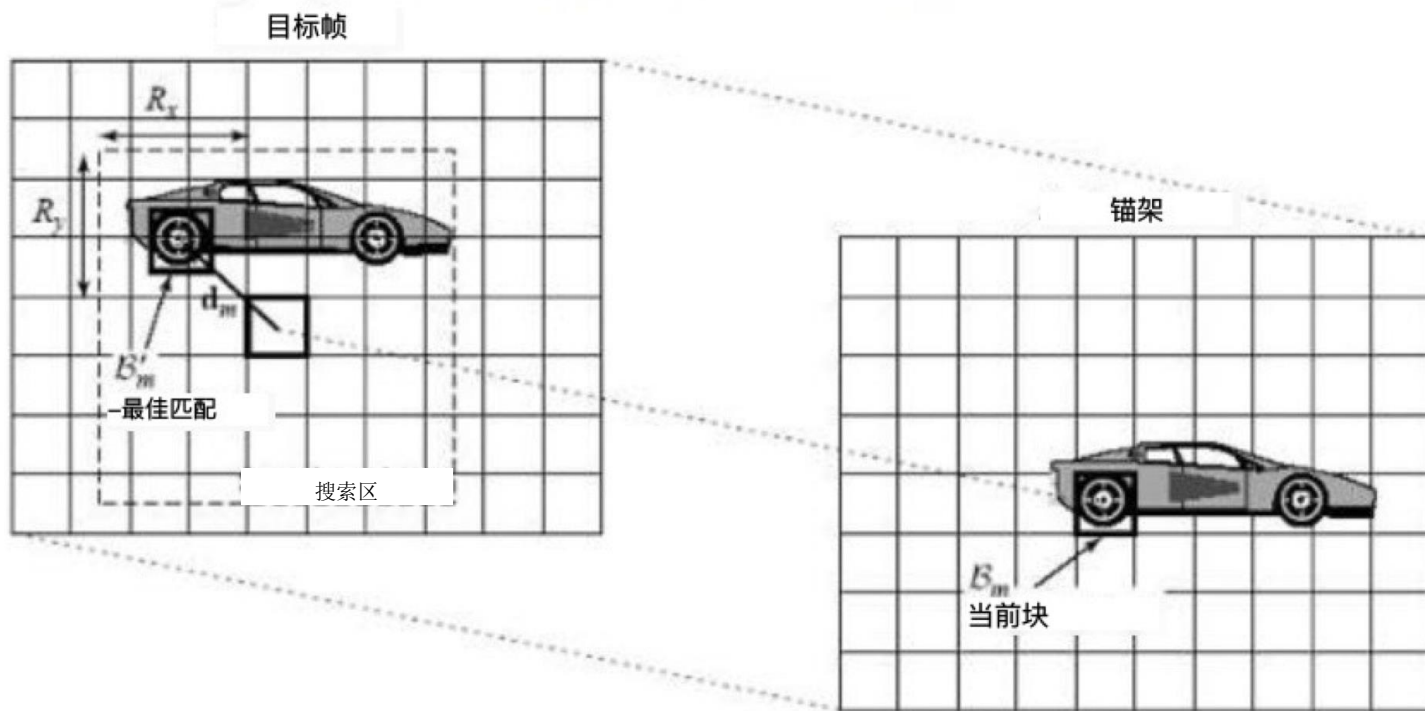
$$E_{\text{DFD}}(\mathbf{d}_m) = \sum_{\mathbf{x} \in B_m} |\psi_2(\mathbf{x} + \mathbf{d}_m) - \psi_1(\mathbf{x})|^p \rightarrow \min$$

·优化方法:

- 使用MAD标准 ($p=1$) 进行穷举搜索 (可行, 因为一次只需要搜索一个MV)
- 快速搜索算法
- 整数与分数像素精度搜索



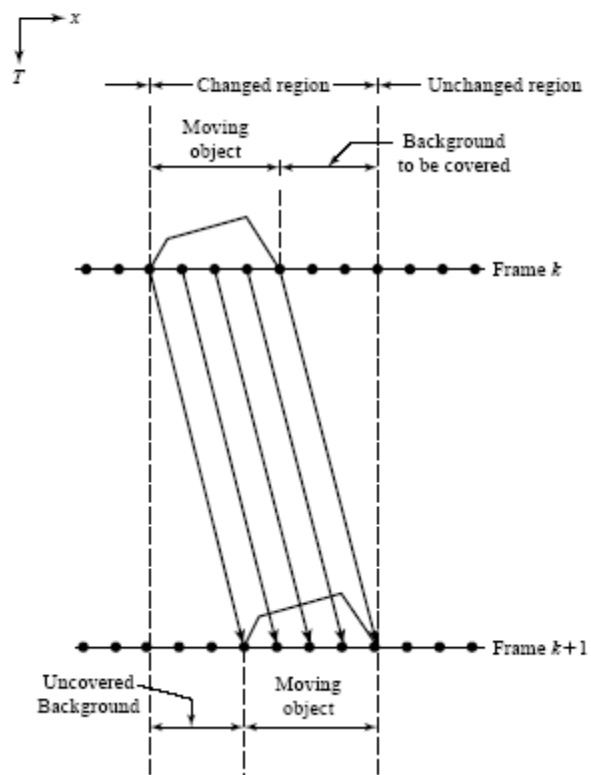
穷举块匹配法





遮挡效应

闭塞区运动不确定





积分-Pel EMBA的复杂性

- 假设
 - 图像大小: $M \times M$
 - 块大小: $N \times N$
 - 搜索范围: 每个维度中的 $(-R, R)$
 - 搜索步长: 1像素 (假设为整数MV)
- 操作计数 (一个操作=1 “-”, 1 “+”, 1 “*”) :
 - 每个候选位置: N^2
 - 每个块通过所有候选人: $(2R+1)^2 N^2$
 - 整个帧: $(MN^2 (2R+1)^2 N^2 = M^2 (2R+1)^2$
 - 与块大小无关!
- 例子: $M=512$, $N=16$, $R=16$, 30帧/秒
 - 操作总数= 2.85×10^8 每帧 = 8.55×10^9 秒/秒
- 适用于超大规模集成电路实现的规则结构
- 对纯软件实施的挑战



用于脚本Integer-pel EMBA 的示例Matlab脚本

```
%f1: anchor frame; f2: target frame, fp: predicted image;
%mvx,mvy: store the MV image
%widthxheight: image size; N: block size, R: search range

for i=1:N:height-N,
    for j=1:N:width-N %for every block in the anchor frame
        MAD_min=256*N*N;mvx=0;mvy=0;
        for k=-R:1:R,
            for l=-R:1:R %for every search candidate
                MAD=sum(sum(abs(f1(i:i+N-1,j:j+N-1)-f2(i+k:i+k+N-1,j+l:j+l+N-1))));
                % calculate MAD for this candidate
                if MAD<MAD_min
                    MAD_min=MAD,dy=k,dx=l;
                end;
            end;end;
        fp(i:i+N-1,j:j+N-1)= f2(i+dy:i+dy+N-1,j+dx:j+dx+N-1);
        %put the best matching block in the predicted image
        iblk=(floor)(i-1)/N+1; jblk=(floor)(j-1)/N+1; %block index
        mvx(iblk,jblk)=dx; mvy(iblk,jblk)=dy; %record the estimated MV
    end;end;
```

Note: A real working program needs to check whether a pixel in the candidate matching block falls outside the image boundary and such pixel should not count in MAD. This program is meant to illustrate the main operations involved. Not the actual working matlab script.

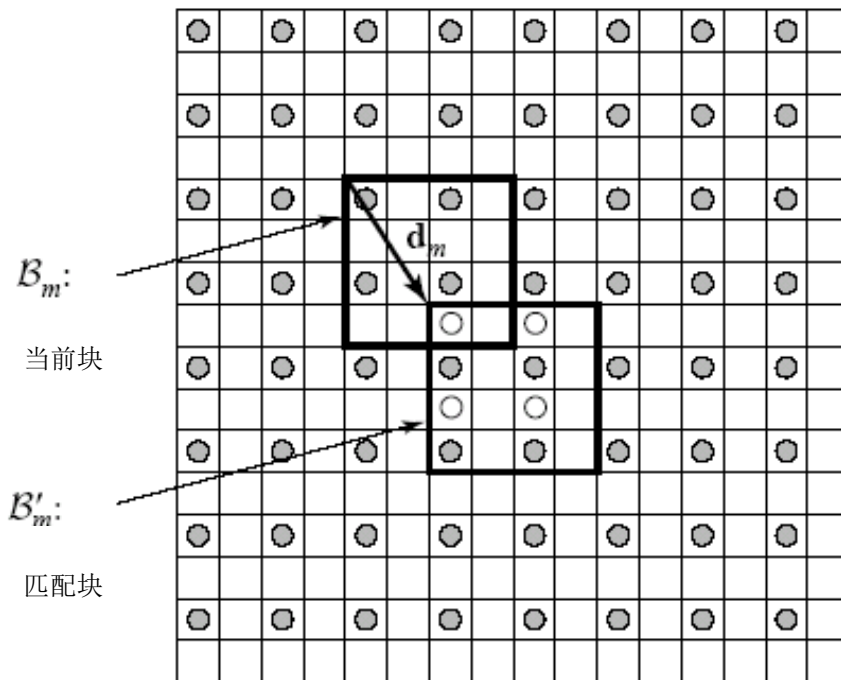


分数精度EBMA

- 真正的MV可能并不总是像素的倍数。要允许子像素MV，搜索步长必须小于1像素
- 半像素EBMA：步长=两个维度上的1/2像素
- 难度：
 - 目标帧只有整数像素
- 解决办法：
 - 在搜索前将目标帧内插2倍
 - 双线性插值通常使用
- 复杂性：
 - 4倍的整数像素，加上额外的内插运算。
- 快速算法：
 - 先以整数精度进行搜索，然后以半像素精度在小搜索区域中进行细化。



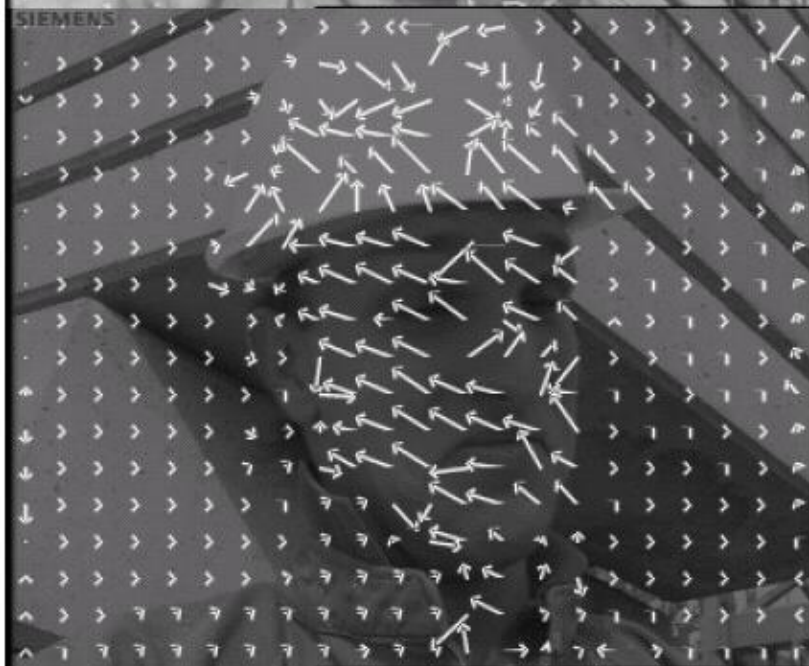
半像素精度EBMA





目标框

动作域



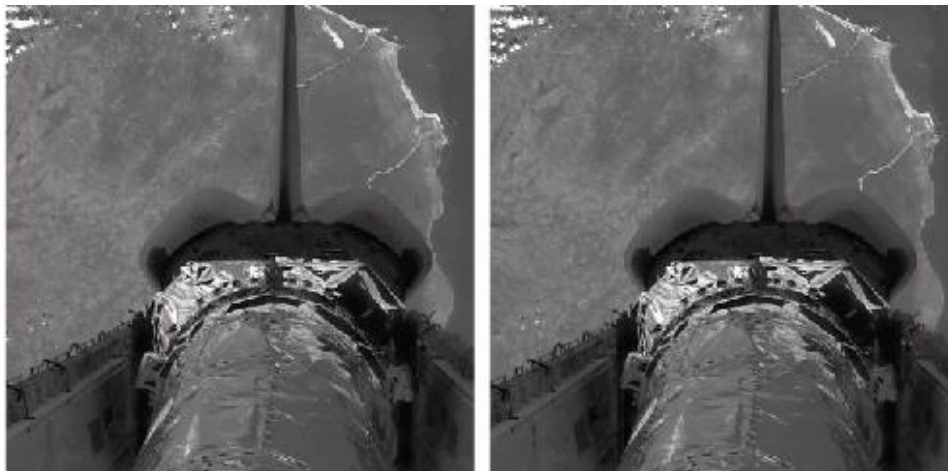
锚定框架

(29.86dB)

预测锚定框架

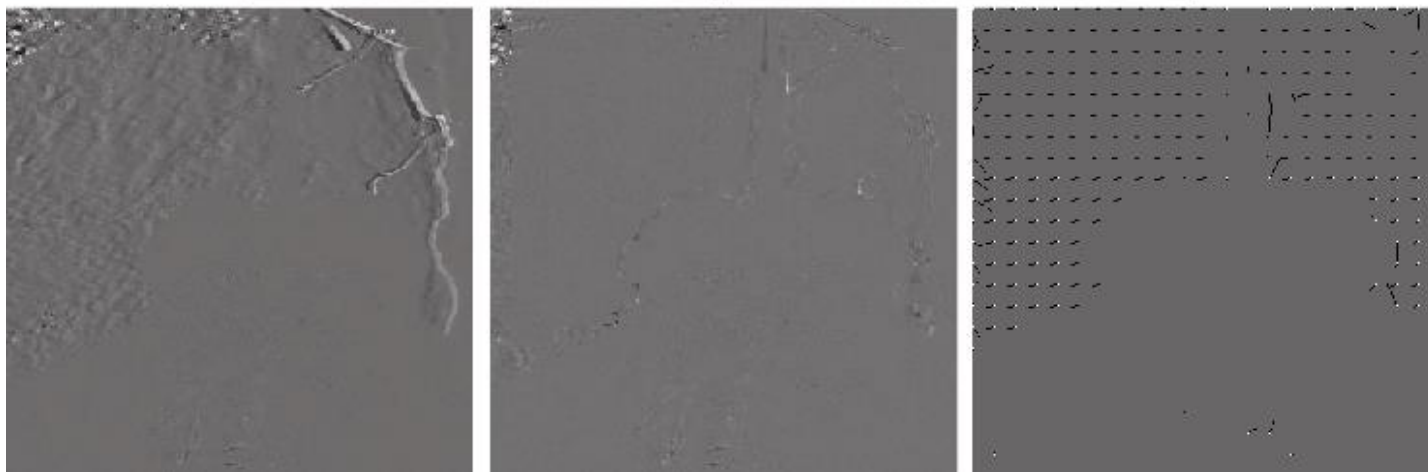


参考框架，当前框架



MB=16*16
RANGE=+/-16

Stan. V=
12.73
H=4.17
Bits/pixel
C=8/4.17
=1.92



a b
c d e

Stan. V=5.63 H=3.04, C=8/3.04=2.63

图8.37 (a) 和 (b) 在绕轨道运行的太空穿梭视频中，相隔13帧的两幅地球照片 (c) 没有运动补偿的预测误差图像。(d) '带运动补偿的预测残差。(e) 与 (d) 相关联的运动矢量。(d) 中的白点表示所描绘的运动矢量的箭头。(原始图片由美国宇航局提供)



MB=8*8,+/-8

No, delta=12.7, H=4.17

Integer delta=4.4, H=3.34

a	b
c	d

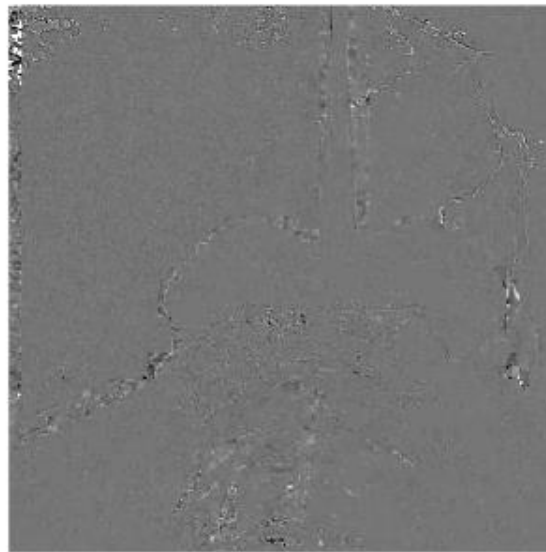
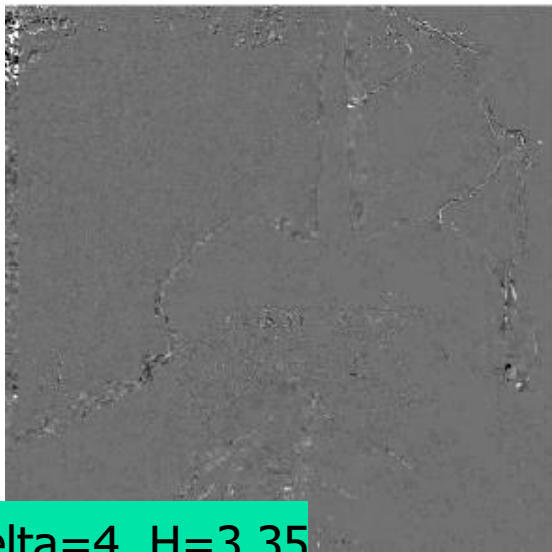
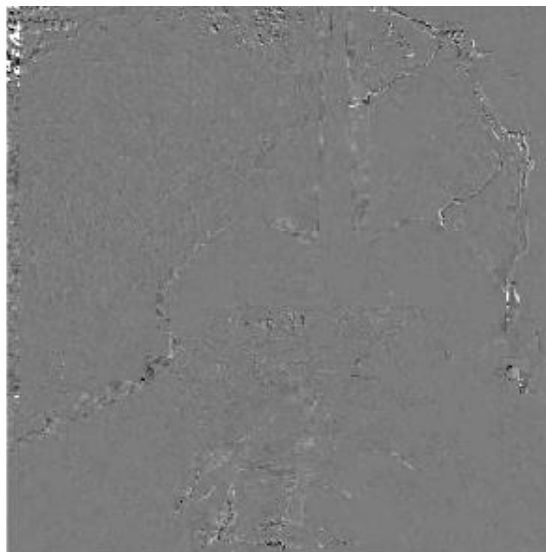
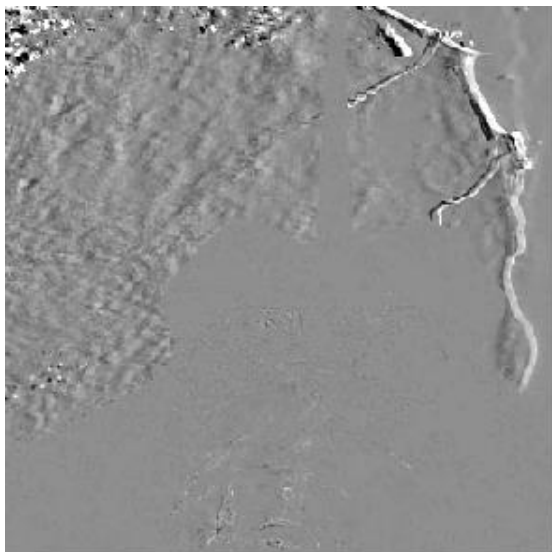


图8.38

亚像素运动补偿
预测残差：(a)
无运动补偿；
(b) (b) 单
像素精度；(c)
像素精度；和
(d) 像素精度
。(所有预测误差都被缩放到全
强度范围，然后
乘以2以增加其
可见性。)

1/2 delta=4, H=3.35

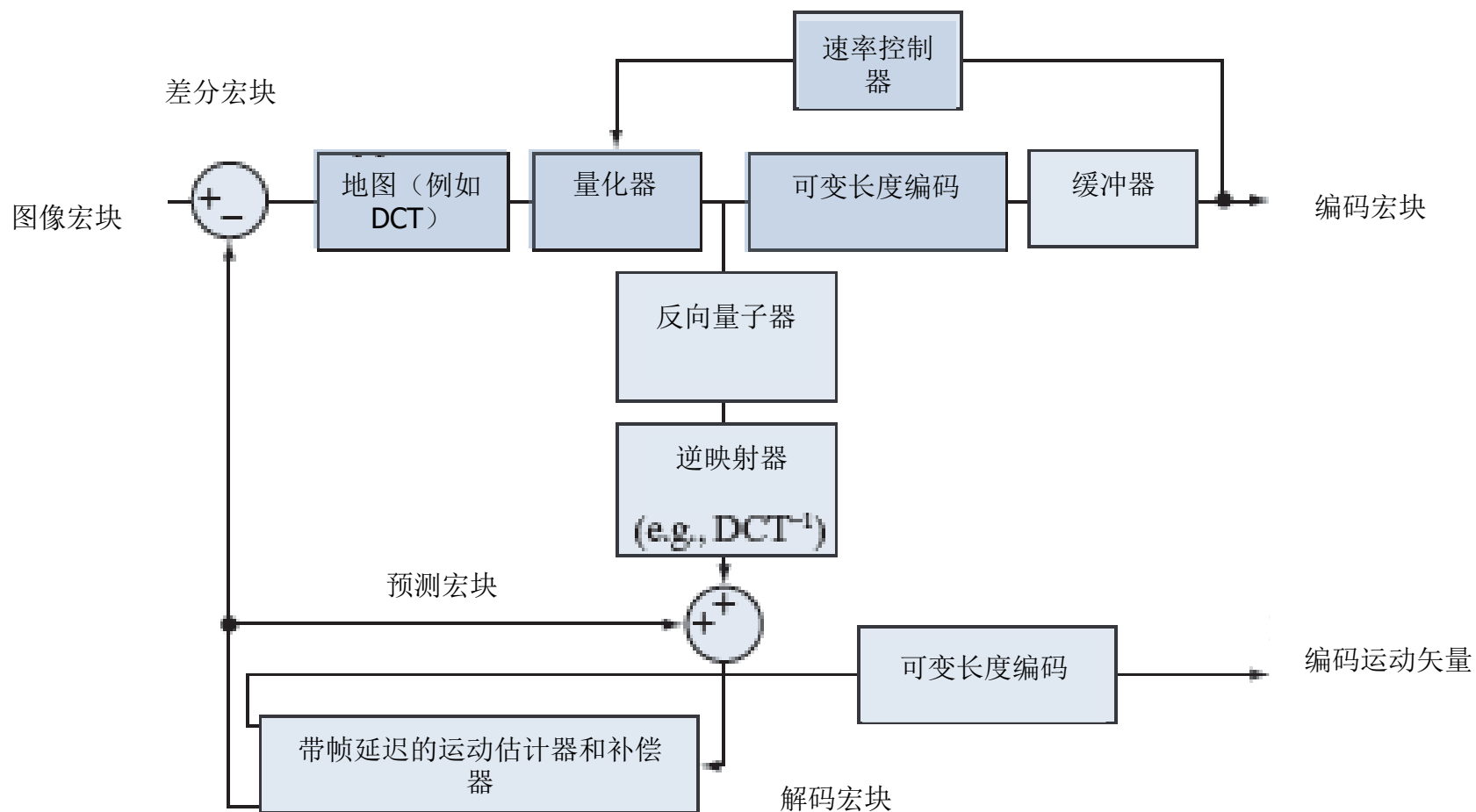
1/4 delta=3.8, H=3.34



参数	H.262				H.263	MPEG-4	VC-1 WMV-9	H.264 MPEG-4 AVC
	表8.11	video压缩标准中的预测编码。						
运动矢量精度	1	1/2	1/2	1/2	1/4	1/4	1/4	
宏块大小	16 × 16	16 × 16	16 × 16 16 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 16 × 8 8 × 16 8 × 8 8 × 4 4 × 8 4 × 4
转换	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 8 × 4 4 × 8 4 × 4 Integer DCT	4 × 4 8 × 8 Integer
帧间预测	P	P, B	P, B	P, B	P, B	P, B	P, B	P, B
I-帧内预测	No	No	No	No	No	No	No	Yes



图8.39典型的运动补偿视频编码器。





Frame 0021



Frame 0044



Frame 0201



Frame 0266



Frame 0424



Frame 0801



Frame 0959



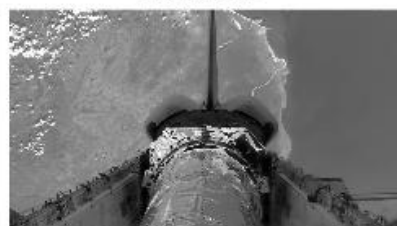
Frame 1023



Frame 1088



Frame 1224



Frame 1303



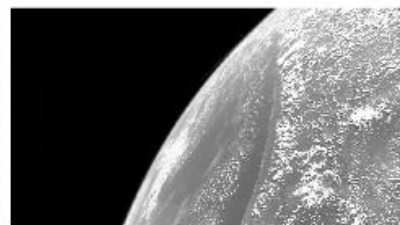
Frame 1304



Frame 1595



Frame 1609



Frame 1652

图8.40来自1829帧的15帧，1分钟的NASA视频。原始视频是HID全彩（美国宇航局提供）

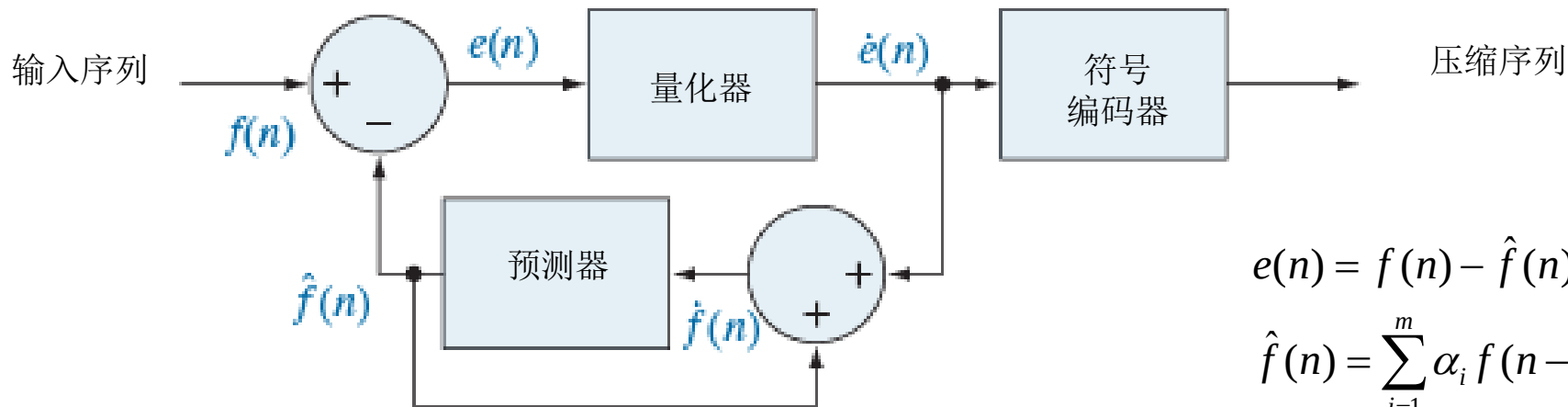


有损预测编码

a
b

图8.41

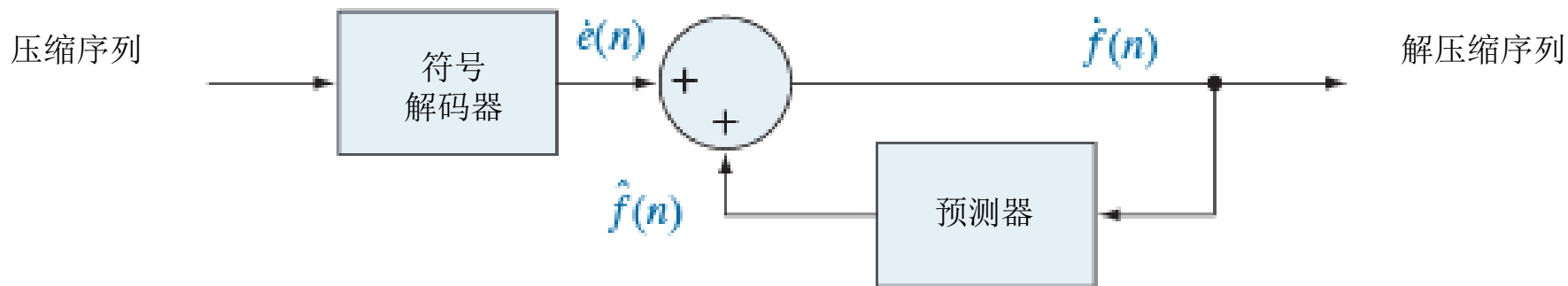
一个无损的
预言性的
编码模式：
(a) 编码器：
(b) 解码器。

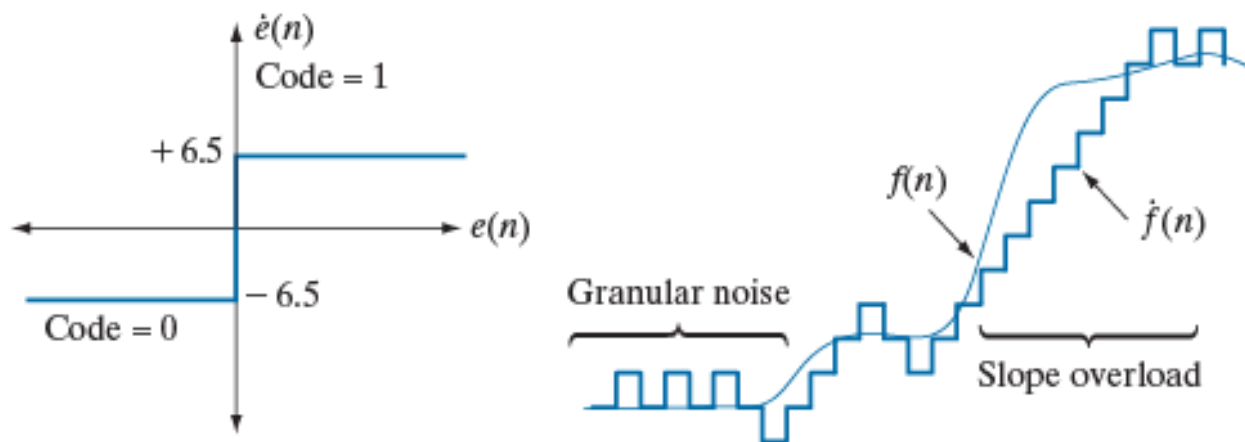


$$e(n) = f(n) - \hat{f}(n)$$

$$\hat{f}(n) = \sum_{i=1}^m \alpha_i f(n-i)$$

$$\dot{f}(n) = \dot{e}(n) + \hat{f}(n)$$





a b
c

图8.42 Delta调制的一个例子。

Input		Encoder				Decoder		Error
n	$f(n)$	$\hat{f}(n)$	$e(n)$	$\dot{e}(n)$	$\dot{f}(n)$	$\hat{f}(n)$	$\dot{f}(n)$	$f(n) - \dot{f}(n)$
0	14	—	—	—	14.0	—	14.0	0.0
1	15	14.0	1.0	6.5	20.5	14.0	20.5	− 5.5
2	14	20.5	− 6.5	− 6.5	14.0	20.5	14.0	0.0
3	15	14.0	1.0	6.5	20.5	14.0	20.5	− 5.5
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
14	29	20.5	8.5	6.5	27.0	20.5	27.0	2.0
15	37	27.0	10.0	6.5	33.5	27.0	33.5	3.5
16	47	33.5	13.5	6.5	40.0	33.5	40.0	7.0
17	62	40.0	22.0	6.5	46.5	40.0	46.5	15.5
18	75	46.5	28.5	6.5	53.0	46.5	53.0	22.0
19	77	53.0	24.0	6.5	59.6	53.0	59.5	17.5
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.

$$e(n) = f(n) - \hat{f}(n)$$

$$\hat{f}(n) = \alpha_1 \dot{f}(n-1)$$

$$\dot{f}(n) = \dot{e}(n) + \hat{f}(n)$$

$$\dot{e}(n) = \begin{cases} 6.5 & e(n) > 6.5 \\ -6.5 & \text{otherwise} \end{cases}$$



最优预测器

■ 最小化均方预测误差

$$E\{e^2(n)\} = E\{[f(n) - \hat{f}(n)]^2\}$$

s.t.

$$\dot{f}(n) = \dot{e}(n) + \dot{\hat{f}}(n) \approx e(n) + \hat{f}(n) = f(n)$$

$$\hat{f}(n) = \sum_{i=1}^m \alpha_i f(n-i)$$

■ 最小化

$$E\{e^2(n)\} = E\{[f(n) - \sum_{i=1}^m \alpha_i f(n-i)]^2\}$$



最优预测器

$$\alpha = R^{-1}r$$

$$\alpha = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_m \end{bmatrix}, r = \begin{bmatrix} E\{f(n)f(n-1)\} \\ E\{f(n)f(n-2)\} \\ \vdots \\ E\{f(n)f(n-m)\} \end{bmatrix}$$

$$R = \begin{bmatrix} E\{f(n-1)f(n-1)\} & E\{f(n-2)f(n-1)\} & \dots & \dots & E\{f(n-m)f(n-1)\} \\ E\{f(n-1)f(n-2)\} & & \dots & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ E\{f(n-1)f(n-m)\} & E\{f(n-2)f(n-m)\} & \dots & \dots & E\{f(n-m)f(n-m)\} \end{bmatrix}$$



最优预测器

$$\hat{f}(x, y) = 0.97 f(x-1, y)$$

$$\hat{f}(x, y) = 0.5 f(x, y-1) + 0.5 f(x-1, y)$$

$$\hat{f}(x, y) = 0.75 f(x, y-1) + 0.75 f(x-1, y) - 0.5 f(x-1, y-1)$$

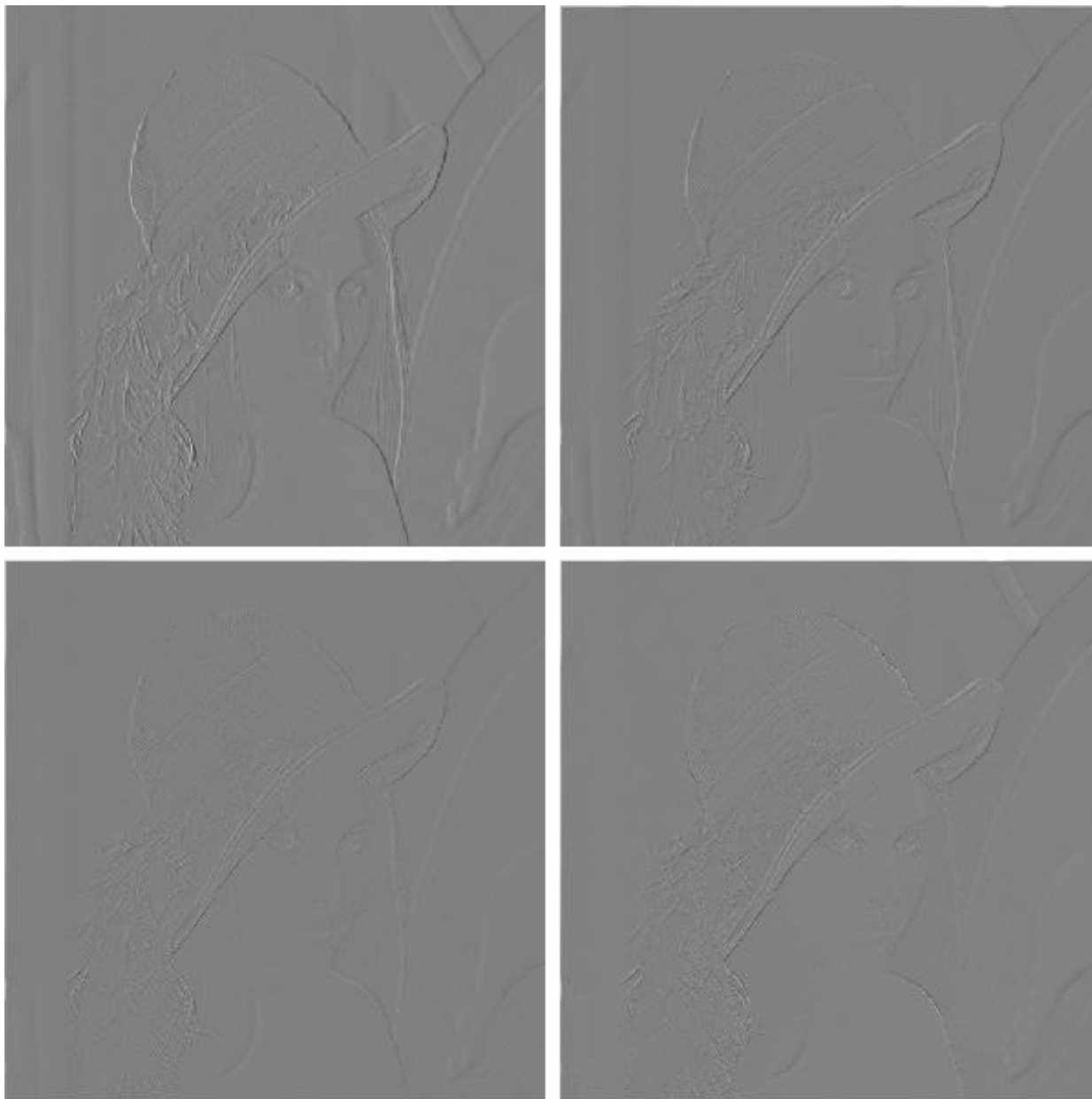
定义一套相对稳健的阿尔法，在广泛的图像范围内提供令人满意的性能。

$$\hat{f}(x, y) = \begin{cases} 0.97 f(x, y-1) & \Delta h \leq \Delta v \\ 0.97 f(x-1, y) & \text{otherwise} \end{cases}$$

$$\Delta h = |f(x-1, y) - f(x-1, y-1)|$$

$$\Delta v = |f(x, y-1) - f(x-1, y-1)|$$

最后一个旨在通过计算图像方向属性的局部测量，并选择专门针对测量行为的预测器，来改善边缘渲染。



a	b
c	d

图8.43

四种Linear预测
技术的比较。

预测器误差
9.1, 9.7



最优量化

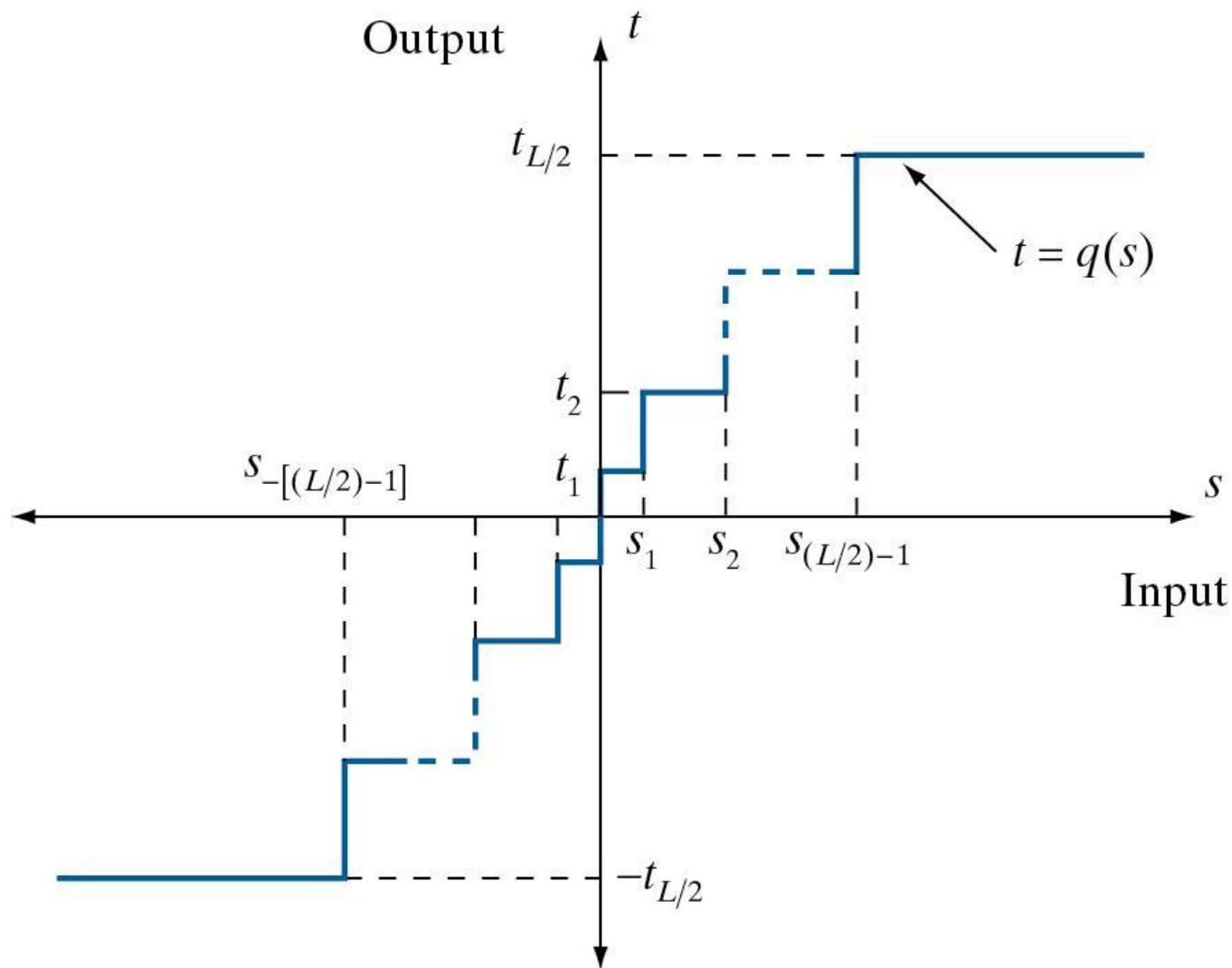


图8.44

一个典型的
量化函数。



最优量化

■ 最小化

$$E\{(s_i - t_i)^2\}$$

$p(s)$ is even function

obtain:

$$\int_{s_{i-1}}^{s_i} (s - t_i) p(s) ds, i = 1, 2, \dots, L/2$$

$$s_i = \begin{cases} 0 & i = 0 \\ \frac{t_i + t_{i+1}}{2} & i = 1, 2, \dots, L/2 - 1 \\ \infty & i = L/2 \end{cases}$$

$$s_{-i} = -s_i, t_{-i} = -t_i$$

$$t_i - t_{i-1} = s_i - s_{i-1} = \theta$$

重建水平是 $p(s)$ 下的区域在指定决策区间内的中心值

决策层处于重建层的中间位置

q 是奇数函数



- 如果在一般的有损预测编码器中使用一个利用可变长度编码的符号编码器
- 步长的最佳统一量化器将提供比固定长度编码的Lloyd-Max量化器更低的码率，且输出保真度相同



- 缓慢变化的区域可以被精细地量化，而快速变化的区域则被更粗略地量化。
- 减少颗粒噪声和斜率过载，同时只需要在编码率上有一个最小的增加。权衡之下，量化器的复杂性增加了。



JPEG2000简介



JPEG 2000标准化

- JPEG 是 ‘Joint Photographic Experts Group’ （ "联合摄影专家组 "） 的缩写
- 网址: www.jpeg.org
- ISO/IEC JTC 1/SC 29/WG 1



JPEG2000的目标

- 卓越的低比特率压缩
- 在一个单一的代码流中进行无损和有损压缩
- 可以处理大型图像。64K*64K
- 按像素精度和分辨率进行渐进式传输
- 感兴趣区域 (ROI) 编码能力
- 在嘈杂的无线环境中传输



JPEG 2000 应用

- 互联网
- 手机
- 印刷业
- 扫描
- 数码摄影
- 遥感
- 传真
- 医疗
- 数字图书馆
- 电子商务



图像与视频处理 Image and Video Processing

JPEG 2000与JPEG基线的对比 (在0.125 bpp时)



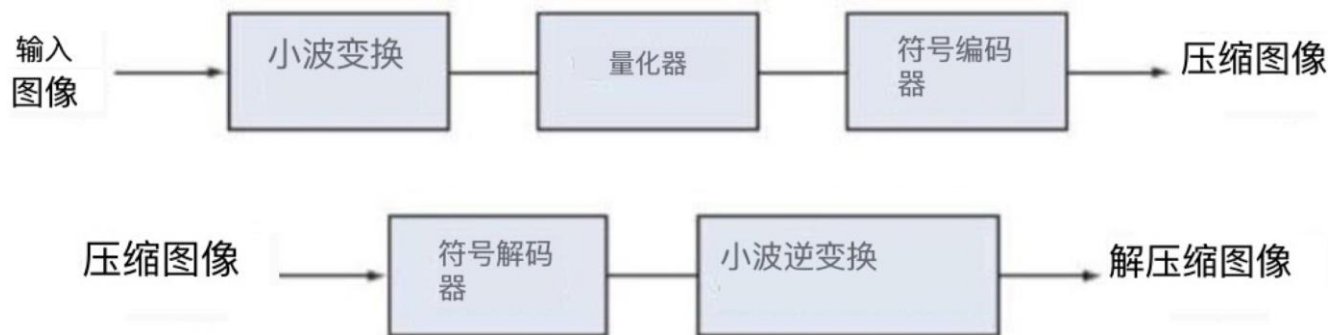


JPEG 2000与JPEG基线的对比 (在0.25 bpp时)





JPEG 2000编码方案



a
b

图8.45一种小波编码系统：

(a) 编码器：
(b) 解码器。

TABLE 8.14

当截断变换时，小波变换滤波器抽头和零系数低于1.5

小波	滤嘴 (缩放+小波)	零系数
Haar	2 + 2	33.3%
Daubechies	8 + 8	40.9%
Symlet	8 + 8	41.2%
Biorthogonal	17 + 11	42.1%

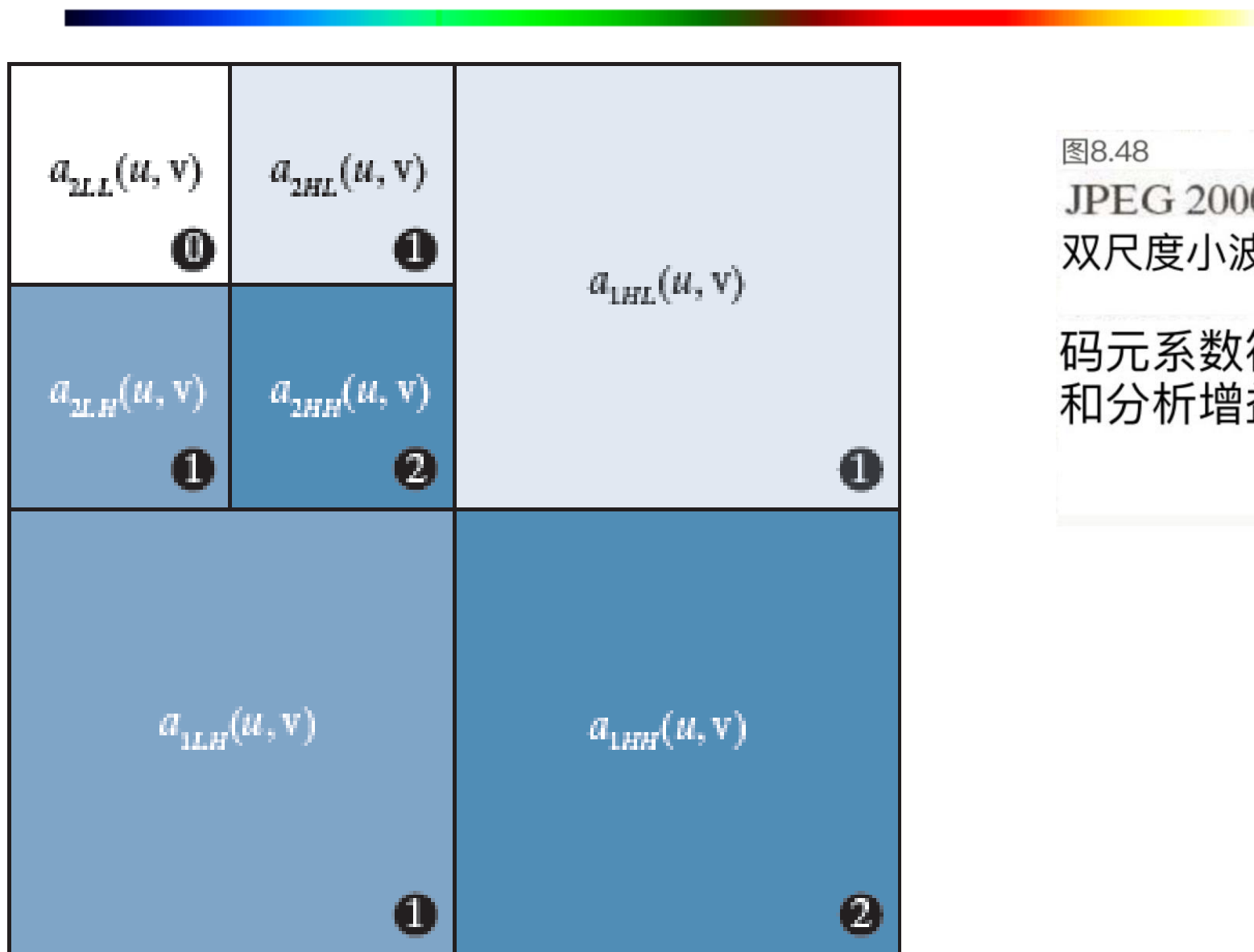


图8.48

JPEG 2000

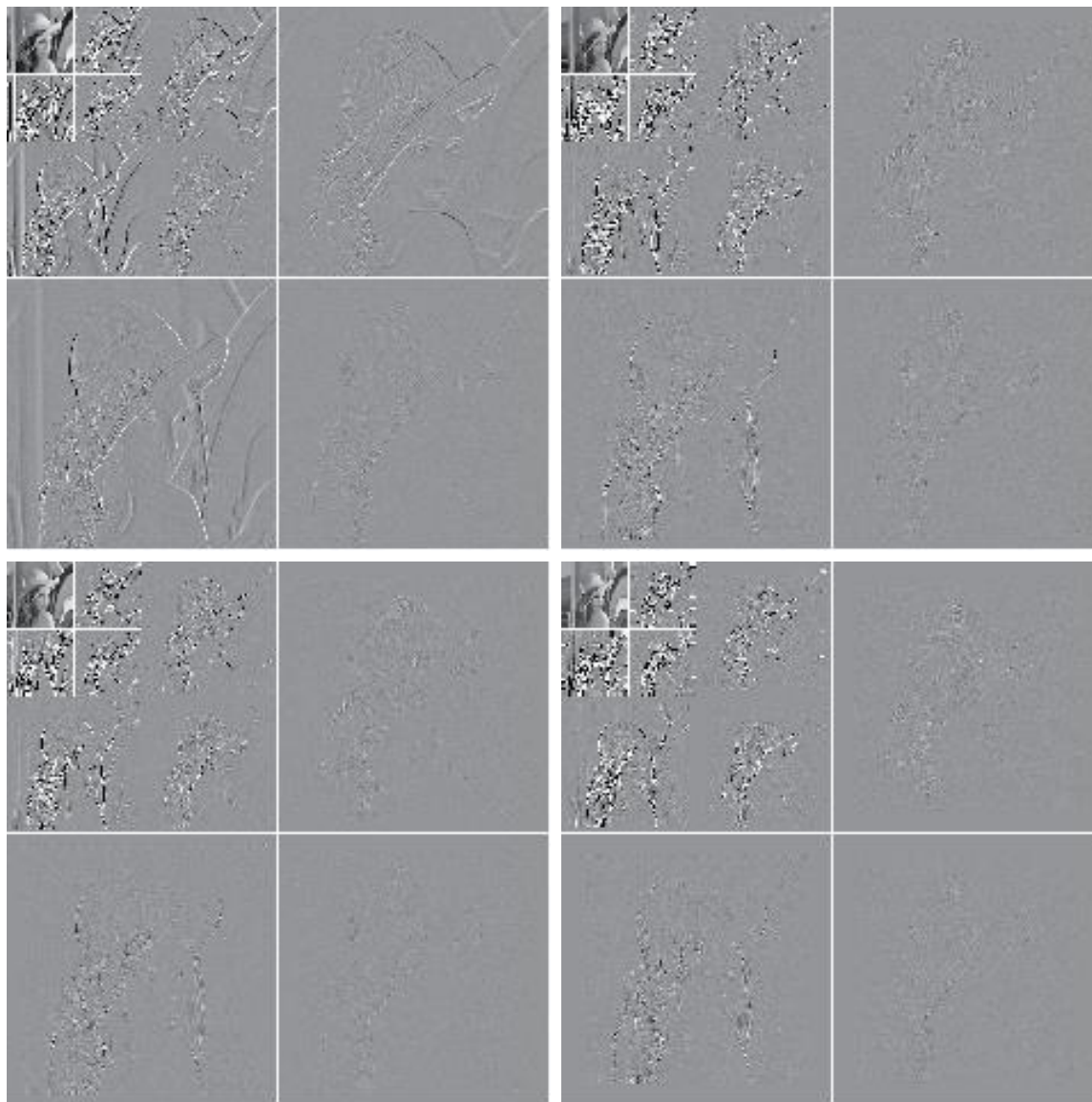
双尺度小波变换

码元系数符号
和分析增益。



Haar

Daubechies9/7



a b
c d

FIGURE 8.46

图8.9 (a) 的三
尺度小波变换

关于:

(a) Haar wavelets,
(b) Daubechies
wavelets,
(c) symlets, and
(d) Cohen-
Daubechies
Feauveau
biorthogonal
wavelets.

CohenDaubechies

Haar



小波选择



$<1.5, 0$

表8.13小波变换滤波器抽头和

在图8.46中截断1.5下的变换时系数归零。

小波	滤嘴 (缩放+小波)	零系数
Haar (see Ex. 7.10)		33.8%
Daubechies (see Fig. 7.8)		40.9%
Symlet (see Fig. 7.26)		41.2%
Biorthogonal (see Fig. 7.39)		42.1%

4-28乘法
和加法/系数



TABLE 8.15

不可逆9-7小波变换的低通和高通分析滤波器的脉冲响应

Haar, 9/7

滤嘴

高通小波系数

低通小波系数

0	−1.115087052456994	0.6029490182363579
±1	0.5912717631142470	0.2668641184428723
±2	0.05754352622849957	−0.07822326652898785
±3	−0.09127176311424948	−0.01686411844287495
±4	0	0.02674875741080976



分解水平的选择

TABLE 8.14

分解级别对
512*512的图像
8.9 (a) 的小波
编码的影响

分解级别 (比例尺或滤波器组迭 代)	近似系数图像	截断系数 (%)	重建误差 (均方根)
1	256×256	74.7%	3.27
2	128×128	91.7%	4.23
3	64×64	95.1%	4.54
4	32×32	95.6%	4.61
5	16×16	95.5%	4.63

Th=5

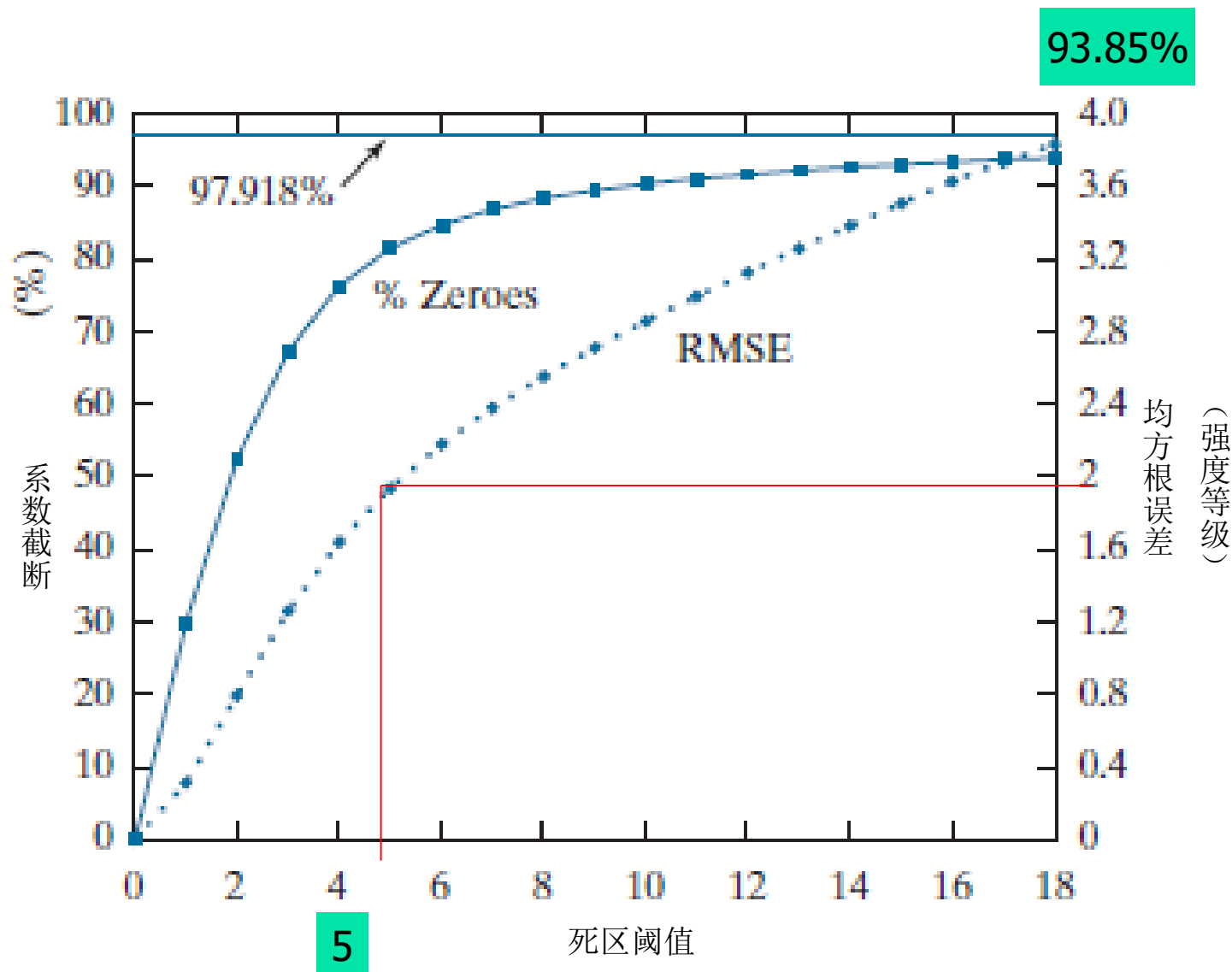


图8.47
死区间选择对小
波编码的影响

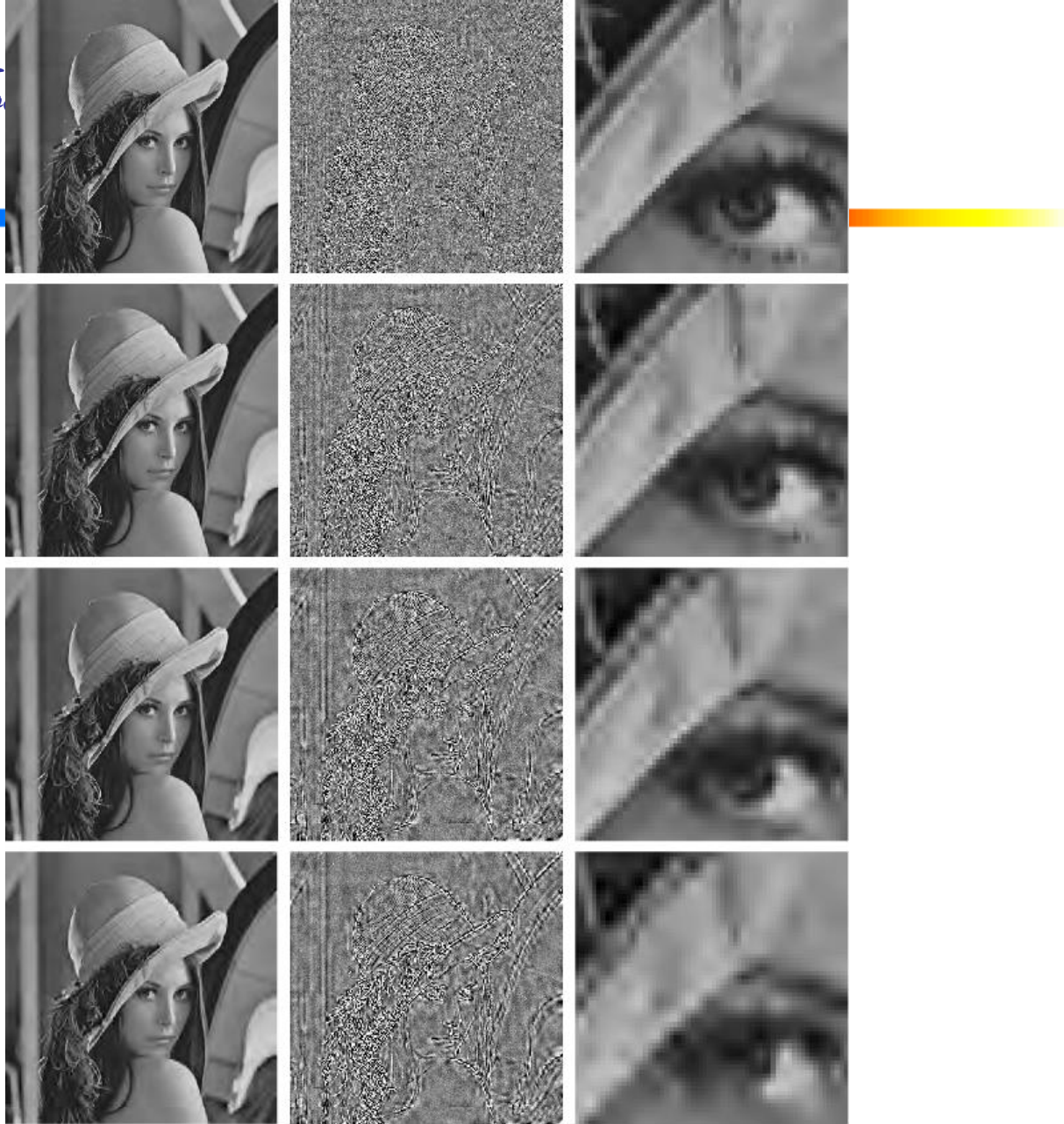


图8.49图8.48的四个JPEG-2000近似。8.9(a). 每行包含压缩和重建后的结果，结果和原始图像之间的密封差，以及重建图像的缩放部分。（将第1行和第2行中的结果与图2中的JPEG结果进行比较。8.32.）



例如：原始自行车
车图像，8bpp，
2048x2560





例子：按质量递增
，视觉无损的
bike_full, 3.065
bpp, PSNR =
49.33 dB





例子：按尺寸（空间分辨率）逐级递进

ISO 400



bike_half, 0.71 bpp, 32.9 dB

ISO 400



bike_quat

ISO 400



bike_oct



bike_hex



图像与视频处理
Image and Video Processing

原始图像fr1782, 8bpp, 640x480





图像与视频处理

Image and Video Processing

例子：视觉上无损的图像（3.19 bpp）。





例子：按质量渐进 (0.20 bpp)





图像与视频处理

Image and Video Processing

例子：按质量渐进 (0.10 bpp)





图像与视频处理

Image and Video Processing

例子：按质量渐进 (0.05 bpp)





0.125 bpp的JPEG (放大)





0.125 bpp的JPEG2000





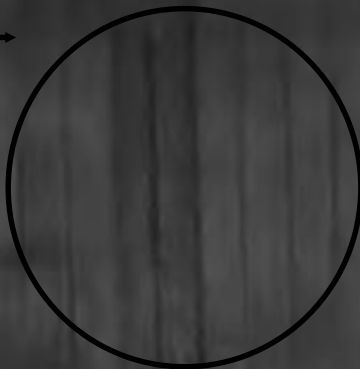
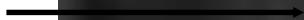
感兴趣的区域 (ROI) (1)

- 以比图像其他部分更高的保真度重建图像的特定部分
- 两种类型：
 - 编码过程中的ROI：当编码前已知所需区域的几何形状时非常有用。
 - 解码时的ROI：例如在交互式浏览远程数据库中的图像时非常有用。



例子：ROI；图像压缩为0.05bpp

ROI

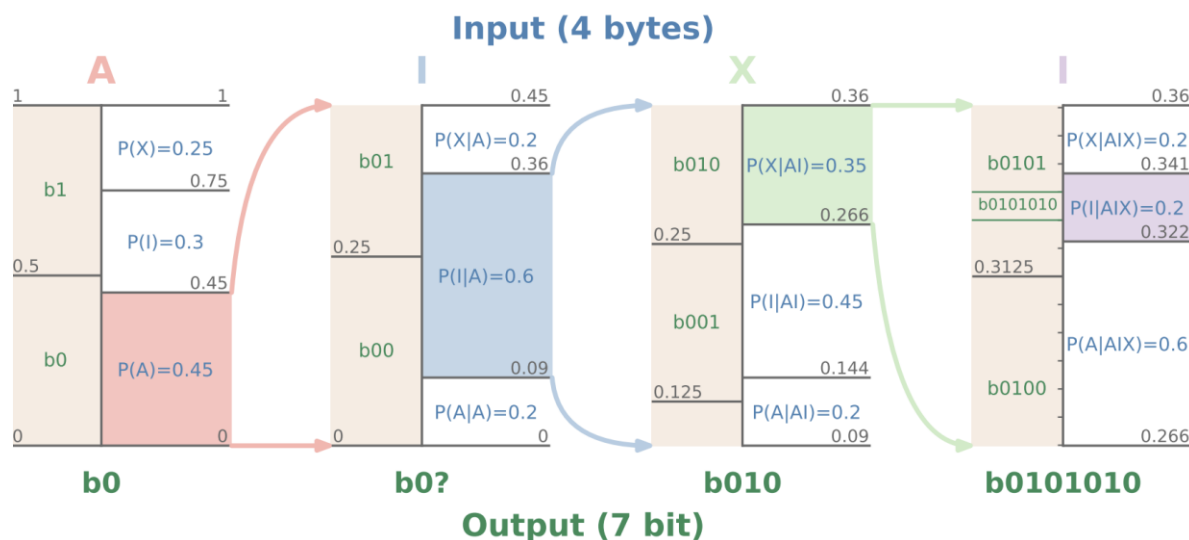




Language Modeling Is Compression

数据的压缩和信息中每个字符出现的概率有关系

- 在实际场景中，每个字符出现的概率是不同的，比如汉语中的“稿”和“的”出现的概率明显不同，“的”出现的概率会高很多，在这种情况下，如果把出现概率高的字符编码成更短的二进制数，把出现概率低的字符编码成相对长的二进制数，可以节省整体编码的二进制数。
- 大语言模型本质上是一个概率预测模型，即在给定上下文的情况下，模型给出字符表中各个字符出现在下一个位置的概率，然后我们通常选取概率最高的那个字符作为预测值，所以预测模型依赖一个概率分布。
- 通常说大模型在训练过程中学习“知识”，这个“知识”是什么呢，其实就是概率分布。



Google DeepMind, ICLR2024, <https://arxiv.org/abs/2309.10668>, 2024.3



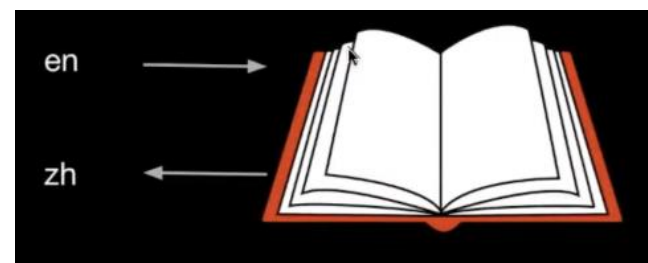
Language Modeling Is Compression

找到解决感知问题的最小描述长度

- 生成模型是无损压缩器
- 大语言模型是 SOTA 的无损文本压缩器

Minimum Description Length

- ✓ Think about Searle 's Chinese Room thought experiment. If a computer program translates en $\leftrightarrow$ zh via a series of rules, does it understand translation?
- ✓ if the computer uses a lookup of direct translations of all possible phrases, it has the least understanding of translation.

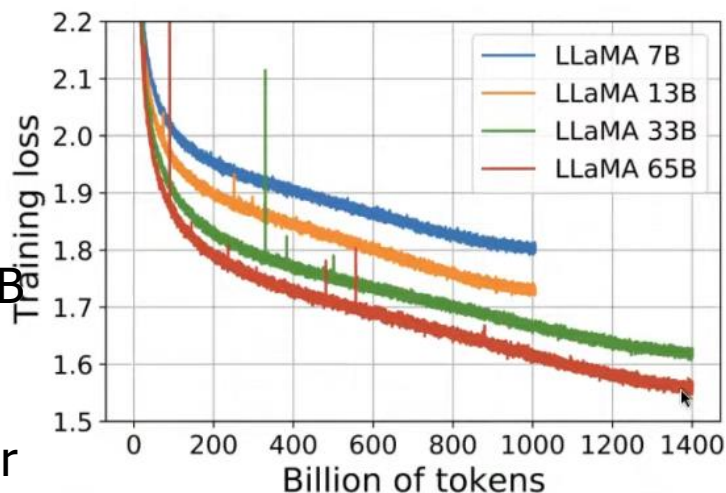




Language Modeling Is Compression

- 用于初始化和训练模型的代码约为 1MB，粗略地计算负对数似然大约是 400GB，而用于训练的原始数据是 5.6TB 的文本，因此该模型的压缩率为 14 倍。
- Hutter Prize 上最好的文本压缩器能实现 8.7 倍的压缩。创建更强大的模型，为我们的训练数据提供更低的无损压缩率。

- ✓ Compression stats:
Original dataset:
5.6TB
raw text.
- ✓ Compressed: 397.3GB
- ✓ compression
rate: 14X
- ✓ Best text compressor
on HutterPrize: 8.7x
compression.



- So let's put this together, Satya wants to send a dataset of the world's knowledge D to Sundar. Both have access to supercomputers, but there's





Language Modeling Is Compression

■ 压缩有可能是正确的，但效率不高

- 1) 像素级图像进行建模是十分昂贵的，因此研究者尝试使用语义信息进行建模。
- 2) 像素级的视频建模更是复杂

■ 很多有用的信息是不可观察的

Alphazero，你可能只观察到了他可以进行围棋的自我博弈，但忽略了在训练它时用到了很多其他游戏的中间搜索结果。它看起来更像是一个代理人，可以通过搜索中间结果实现不同行为。

Compressing everything may be correct but inefficient:

- pixel-level image modelling is expensive
- Pixel-level video modelling seems positively crazy

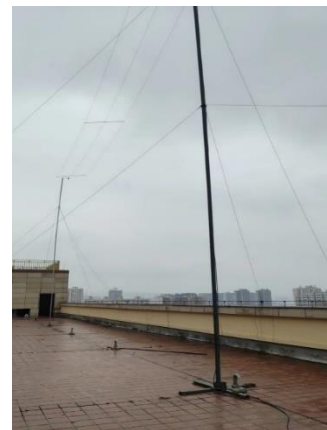
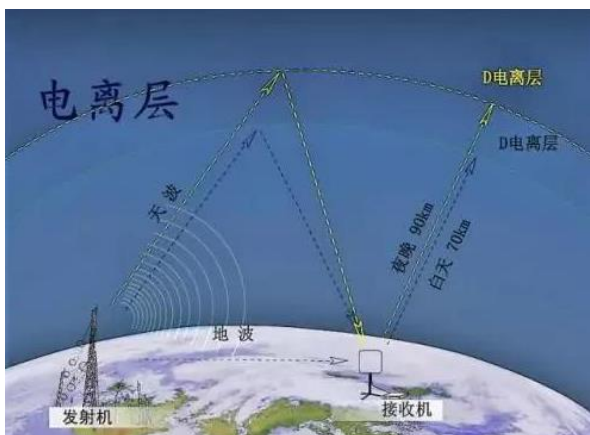
Lots of useful information in the world that is not observable:

- Why on-policy behaviour is still incredibly important.
- E.g.projects such as Alphazero, Cicero etc.



短波信道图像传输需求

- 短波通讯系统无须中继站就可以长距离通讯，其装置体积小、技术成熟，对天灾、战争的抗毁能力强；
- 短波通讯劣势：传输速率和带宽极低。每个短波电台只占用3.7kHz的频率宽度，单载波短波数据传输速率不高于9.6Kbit/s（真实应用常不高于1000bits/s=125B/s）；
- 短波信道实时传输图像视频被业界公认为不可行，**多年来国内外一直未有大的突破。**





智能手机卫星通信图像传输需求

- 卫星通信将成为旗舰智能手机标配，如苹果iPhone 14、华为Mate60；
- 地面蜂窝网络仅覆盖地球表面10%，易受地形和地质灾害影响，偏远地区、海上、灾害现场等迫切需要应急通信服务，包括对应急现场借助图像或视频进行感知；
- 北斗三号为手机支持卫星通信提供了可能，但对民用只开放短报文功能，带宽很窄，每分钟只支持传输几百个字节；**要想利用北斗三号短报文信道传输图像，现有编解码算法无法满足图像质量和超高压缩比的双重要求。**





智能体远距离高压压缩比图像传输需求

- 当前智能体（机器人、无人机群、无人车、无人艇）要实现远距离目标识别、导航定位、态势感知、远程自动控制等功能受带宽限制较大；
- 特别是无人机群，受制于传输带宽，无法同时接收机群高分辨率、高帧频的图像视频数据进行协同。



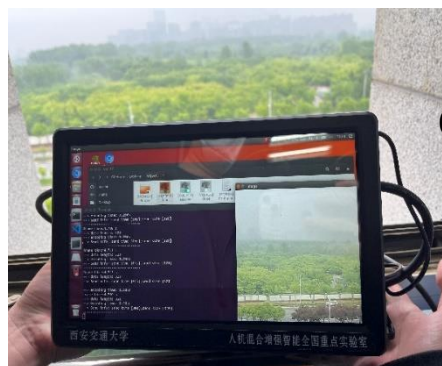


极低码流智能图像传输技术的突破

- 2023年5月，本团队依托西安交大“人机混合增强智能全国重点实验室”在西安-宝鸡两地之间成功开展短波信道极低码流智能图像实时传输实验。发射和接收端都采用自研的“极低码流智能图像传输设备”，通过RS232串口与短波电台通讯（传输带宽为600bps，即75个字节/s），实现彩色图像实时传输。
- 短波电台提供方-某大型通讯企业资深专家评价：**短波通信领域的革命性技术。**



西安发射端



600bps



宝鸡接收端



短波信道实时传输实验现场

西安实验室-发射端



宝鸡实验室-接收端





短波信道实时传输效果 (带宽600bps, 接收端)





极低码流智能图像传输技术的突破

- 2023年9月，本团队利用智能终端（算法移植国产SoC）+北斗数传终端（串口连接），通过**北斗短报文信道**，实现低成本、超高压缩比的图像发送。

北斗短报文数传终端应用示意图

